THE CODING BRIDGE

Algorithmic Coding Patterns

From knowing the algorithm to writing it in Python

Pop. Process. Expand. Enqueue.
State. Invariant. Update.
The small decisions that make an algorithm executable.

A companion to the 150 algorithm pseudocode notes
Pattern-first · Python-focused · Designed for transfer

Before you begin

This book has two reading paths. The algorithms primer in Part I refreshes the structures and algorithms with defined problems, initialized Python examples, calls you can run, and small changes of requirements that change the solution. Read only the topics you need. Part II assumes you recognize those algorithms and teaches the layer between a name and a working program: what the loop owns, what a variable means right now, when a condition is checked, how an answer survives later iterations, and which state belongs in a helper, a call frame, or an object.

A source reference such as #39 means entry 039 under `books/algorithms/pseudo/`. The matching runnable implementation lives under `books/algorithms/python/`. Read a source entry only when you want a complete algorithm; the sketches here deliberately show the *coding move* rather than copy the full solution. The index at the end routes every one of the 150 entries to a move in this book.

To use a primer chapter, start from its problem contract, run its example, and ask how a changed input or requirement affects the data structure. To use a coding-pattern chapter, cover the sketch and say aloud: "My state is ...; before every iteration ...; this action preserves it ...; I record the answer ...". Then code only the spine. The "break test" after a pattern is a tiny counterexample to a plausible wrong order. Change the input and try to break your own version. The companion Jupyter notebook puts runnable examples next to the reading; intentionally incomplete pattern sketches remain reading examples, not failing code cells.

The examples use ordinary Python. A `deque` is a queue; `popleft()` removes its front in constant time. A `set` or `dict` gives expected constant-time membership. A `list` is also a stack with `append()` and `pop()`. `heapq` maintains a min-heap on a list. Nothing depends on clever syntax: when the implementation becomes tricky, the contract gets simpler, not shorter.

Contents

Before you begin i

I Algorithms primer 1

How to use the algorithms primer 2

1 Sequences: positions, spans, and remembered sums 3

1.1 An array's position promise and a dynamic array's bill . . . 3
1.2 Fixed-length windows: exchange one item . . . 3
1.3 Variable-length windows: repair a condition . . . 4
1.4 Two pointers need a reason to move . . . 5
1.4.1 If sorting is not promised . . . 6
1.5 Prefix sums: subtract away an earlier span . . . 6
1.6 Prefix sums plus a hash map: count unknown starts . . . 7

2 Links and worklists: order without indexes 9

2.1 Singly linked lists: keep the next link before changing it . . . 9
2.2 Two-way links and permanent boundary nodes . . . 10
2.3 Fast and slow pointers: distance is state . . . 11
2.4 Stack versus queue: which promise gets served? . . . 12
2.5 A monotonic stack finishes earlier questions . . . 12
2.6 A queue schedules processing; it is not the answer . . . 12
2.7 Adapters: paying for the order you did not get . . . 13

3 Recursion and the search tree 15

3.1 Make the base case part of the contract . . . 15
3.2 Repeated subproblems: cache or carry two values . . . 15
3.3 A tree frame returns one subtree answer . . . 16
3.4 A maze has cycles: decide who owns "visited" . . . 17
3.5 Backtracking: one shared path, different answer events . . . 18
3.6 N-Queens: name a constraint so it is cheap to test . . . 20

4 Trees as traversals and indexes **21**

4.1 Binary trees: order of visits is an answer choice . . . 21
4.2 BSTs: a live catalog of available prices . . . 22
4.3 Segment trees: sums of changing intervals . . . 24
4.4 Quadtrees: stop when a square is uniform . . . 25
4.5 Tries: a path is a string prefix . . . 27

5 Hashing: remember equality without rescanning **29**

5.1 Choose the remembered fact . . . 29
5.2 Hashable keys and canonical forms . . . 30
5.3 Collisions, equality, and growth . . . 31
5.4 Consistent hashing: map to the next point . . . 32
5.5 Bloom filters: a qualified answer in few bits . . . 33
5.6 Try it: from a question to a call . . . 34

6 Sorting: choose the guarantee you need **36**

6.1 Insertion sort: grow a settled prefix . . . 36
6.2 Merge sort: settle two ordered halves . . . 37
6.3 Quicksort: make a pivot final . . . 37
6.4 Heap sort: put each maximum at the end . . . 38
6.5 When can sorting be linear? . . . 39
6.5.1 Counting sort: pay for the integer universe . . . 39
6.5.2 Bucket sort: pay for a distribution assumption . . . 40
6.5.3 Radix sort: preserve lower-digit decisions . . . 41
6.6 Try it: changing the contract changes the tool . . . 42

7 Graphs: decide what a route must accomplish **43**

7.1 Nodes, edges, and neighbors . . . 43
7.2 Depth-first versus breadth-first search . . . 44
7.3 Components and islands: restart outside the loop . . . 45
7.4 Dijkstra: first *popped* minimum, not first offer . . . 46
7.5 A minimum spanning tree is not a shortest route . . . 46
7.6 Topological order and two kinds of cycle check . . . 48
7.7 Eulerian trails: spend each edge exactly once . . . 49

8 Bits and strings: decide what one position means **52**

8.1 Bits as switches, with an explicit width . . . 52
8.2 Counting set bits and enumerating chosen items . . . 52

8.3 An exact bitmap or a set? . . . 53
8.4 Text units, anagrams, and character counts . . . 54
8.5 Knuth–Morris–Pratt (KMP): reuse a matched start . . . 54
8.6 Tries share starts; stacks close obligations . . . 55
8.7 Kadane's scan: a best ending and a best ever . . . 56

9 Dynamic programming and common problem moves 59
9.1 One-dimensional DP: choose the last item or skip it . . . 59
9.2 Two dimensions: give each prefix pair a meaning . . . 60
9.3 Zero-one knapsack: one item, one decision . . . 61
9.4 A straight run: start only where a run starts . . . 62
9.5 Candy: satisfy constraints from both directions . . . 63
9.6 Trapped rainwater: settle the side with a known bound . . . 63
9.7 A reachability boundary instead of a DP table . . . 64
9.8 Try the definitions . . . 65

II Coding patterns 66

10 State, invariant, update 67
10.1 Begin with a contract, not a loop . . . 67
10.2 The state–invariant–update sentence . . . 67
10.3 The blank-page S/I/U worksheet . . . 68
10.4 POPREXEN: pop, process, expand, enqueue . . . 69
10.5 Distinguish frontier, answer, and evidence . . . 70
10.6 Draft the spine before filling in helpers . . . 70

11 Where a condition belongs 71
11.1 Guard before the first unsafe read . . . 71
11.2 Admit at discovery, not at processing . . . 72
11.3 Reject stale work at pop time . . . 72
11.4 Record while a condition still holds . . . 73
11.5 Test termination with the state it proves . . . 73
11.6 Restore on unwind, even after success . . . 74

12 Who owns the state? 75
12.1 Local, call-frame, shared, or member state . . . 75
12.2 Visited is a state machine, not always a set . . . 75

12.3 A helper needs a return contract . . . 76
12.4 Mutability and identity: snapshot the result . . . 76
12.5 Carry, derive, or cache? . . . 77

13 One pass, with a memory of the past **78**
13.1 Look up before storing the current item . . . 78
13.2 Prefix summaries and the deliberately empty prefix . . . 78
13.3 Work only at the start of a run . . . 79
13.4 Fold a prefix into “current” and “best” . . . 80
13.5 Two passes when the future is unavailable . . . 80
13.6 A candidate is not automatically an answer . . . 81

14 Move the edges of a sequence **83**
14.1 Converging pointers: justify the discarded side . . . 83
14.2 Expand right, repair left . . . 83
14.3 Use one deficit with a clear sign . . . 84
14.4 Fixed width: add, evict, then inspect . . . 85
14.5 Sort, then keep only the active frontier . . . 85
14.6 Rewire a link only after saving the future . . . 86

15 Search a boundary, not a lucky position **87**
15.1 Half-open search for the first true index . . . 87
15.2 Move the predicate into a helper . . . 87
15.3 Partition search: certify both sides . . . 88
15.4 Sorted halves, but a moving origin . . . 89
15.5 Partition in place with region invariants . . . 89
15.6 Choose a representation for ties and order . . . 90

16 Stacks, deques, and heaps: keep only useful work **91**
16.1 A stack of unfinished obligations . . . 91
16.2 Monotonic stack: each pop completes an answer . . . 92
16.3 A sentinel forces the remaining work to finish . . . 92
16.4 Monotonic deque: age and quality are separate . . . 93
16.5 Heap: the top is an appointment, not the whole answer . . . 94
16.6 K-way merging: a heap of frontiers . . . 94
16.7 Two heaps: encode a dividing line . . . 95
16.8 Lazy deletion: one authority, many copies . . . 95

17 Recursive contracts and backtracking **97**
17.1 Decide whether information travels down or up . . . 97
17.2 Postorder: return one arm, record the whole path . . . 97
17.3 Backpropagation: wait, combine, propagate . . . 98
17.3.1 When backpropagation means gradients . . . 99
17.4 Backtracking: choose, descend, undo . . . 100
17.5 Path marks are loans, not global facts . . . 100
17.6 Memoization: the key must identify the entire future . . . 101
17.7 Pick the search tree before writing the helper . . . 102
17.8 Prune before choosing; undo only what was chosen . . . 103
17.9 Python recursion has a real depth limit . . . 103

18 Graph code is frontier code **105**
18.1 First write the node and neighbor contract . . . 105
18.2 DFS: enter, resume, exit . . . 105
18.3 BFS: an arrival gets a permanent layer . . . 106
18.4 BFS levels: freeze the current wave . . . 107
18.5 BFS with more than one shortest parent . . . 108
18.6 BFS configurations: visit the whole future . . . 109
18.7 Multi-source BFS: start all waves at once . . . 110
18.8 DFS colors: separate “on my path” from “done” . . . 111
18.9 Low-link: can a subtree escape above its parent? . . . 112
18.10 Topological order: a count of unfinished prerequisites . . . 113
18.11 Union-find: the root is the group identity . . . 113
18.12 Dijkstra: propose, relax, reject stale routes . . . 114
18.13 MST: what has become connected? . . . 115

19 Data shape decides the structure **116**
19.1 Choose by operation, not by the noun in the story . . . 116
19.2 When a hash map is the right move . . . 116
19.3 Strings: choose the unit and the query . . . 117
19.3.1 Fixed-width windows keep a signed count delta . . . 117
19.3.2 Reuse a prefix, or verify a hash candidate . . . 118
19.3.3 Trie paths and token boundaries . . . 119
19.4 Parsing: a cursor is part of the state . . . 121
19.5 Grids: coordinates, neighbors, and marking policy . . . 121
19.6 Simultaneous updates need an old state and a new state . . . 122

19.7 Bitmasks and bitmaps: name the universe before shifting . . . 122
19.8 Mutable range queries: precompute or maintain a tree . . . 124
19.9 Approximation and indexing: state the answer contract . . . 125

20 Under the hood: data structures by workload 126
20.1 Hashing: from arbitrary key to a bucket . . . 126
20.2 Open addressing: a deleted slot is not an empty slot . . . 126
20.3 Two-way indexing: a map and an array . . . 127
20.4 Circular buffers: preserve order without shifting . . . 128
20.5 Bounded keys: count instead of compare . . . 129
20.6 Sorted array, heap, or range tree? . . . 129
20.7 A text cursor is a boundary between two stacks . . . 129
20.8 Bounded memory changes the error contract . . . 130

21 Greedy, dynamic state, and exact arithmetic 131
21.1 Greedy: name the choice you can never regret . . . 131
21.2 Collapse a BFS layer to two integers . . . 131
21.3 A failed candidate can eliminate a whole prefix . . . 132
21.4 DP: define the subproblem before the table . . . 132
21.5 Write in place with a reader and a scribe . . . 133
21.6 Geometry: use exact signs, not approximate slopes . . . 134
21.7 Sieve: spend work only where it proves something . . . 134
21.8 Halving: make progress by consuming a bit . . . 135
21.9 Sampling and weights: respect the boundary of a draw . . . 136

22 Design problems: invariants that survive every call 137
22.1 Make the operation table the class blueprint . . . 137
22.2 Fast lookup plus fast order requires two structures . . . 137
22.3 Frequency buckets: exploit unit-size changes . . . 138
22.4 Time is an input and an expiration boundary . . . 139
22.5 History: make “as of” a search in sorted writes . . . 139
22.6 Undo logs: preserve the first before-image . . . 140
22.7 Iterators: normalize before answering . . . 140
22.8 Identity, placement, and distributed counters . . . 141

23 Prove the code with a trace, then try to break it 142
23.1 The trace is a test of the invariant . . . 142
23.2 Find the smallest input that distinguishes two orders . . . 142

23.3 Check output shape, not only a value . . . 143
23.4 Use a deliberately slow oracle on small cases . . . 143
23.5 Python details that change the proof . . . 144
23.6 Count transitions, not loop nesting . . . 145
23.7 The final blank-page checklist . . . 145

24 Translation labs: from idea to the next line 146
24.1 The BFS diamond: who claims D? . . . 146
24.2 DFS: what is waiting while a child runs? . . . 146
24.3 Two fires, one queue . . . 147
24.4 One cell, two futures . . . 148
24.5 Two prerequisites, one ready course . . . 148
24.6 A cheaper proposal arrives late . . . 149
24.7 Connectivity: compare roots, not neighbors . . . 149
24.8 Warmer day: pop to finish, not to forget . . . 150
24.9 Histogram: the popped bar finds its width . . . 150
24.10 Top-K: why a min-heap keeps large values . . . 150
24.11 Return an aggregate, then send context forward . . . 151
24.12 Backpropagation through a shared input . . . 151
24.13 Subsets or permutations: which siblings are distinct? . . . 152
24.14 Word Search: visited means "on this route" . . . 152
24.15 N-Queens: which conditions can prune a branch? . . . 153
24.16 A spare A cannot stand in for B . . . 153
24.17 The empty prefix counts as a start . . . 154
24.18 First true, not just any equal position . . . 154
24.19 LIS tails are state, not a subsequence . . . 155
24.20 A shared endpoint means two different things . . . 155
24.21 Strings that contain your delimiter . . . 156
24.22 Old grid and new grid in one cell . . . 156
24.23 A range query's three possible overlaps . . . 157
24.24 Bitset or Bloom filter? . . . 157
24.25 Enumerating submasks without cycling forever . . . 157
24.26 LRU: a read is an update . . . 158
24.27 Snapshots: latest write not after the query . . . 158
24.28 Expired, replaced, and still present . . . 159
24.29 Exact turns, no floating-point slope . . . 159

24.30Reservoir: the current item gets a fair ticket . . . 159
24.31Undo: store the state before the first write . . . 160
24.32Jump boundaries: do not count arrival twice . . . 160
24.33A collision is not an equal key . . . 161
24.34Tombstones: a deletion is not a never-used slot . . . 161
24.35A queue wraps while its logical order does not . . . 161
24.36A found word can still be a useful prefix . . . 162
24.37LFU: the minimum moves only after its bucket empties . . . 162
24.38KMP: a mismatch does not rewind the text . . . 163
24.39Rate limit: burst or exact sliding window? . . . 163
24.40Sieve: why mark at the square? . . . 164
24.41Exponentiation: carry exactly one binary rung . . . 164
24.42Compression: the scribe must not outrun the reader . . . 165

25 Transfer: make the next variant your own 166
25.1 A grid cell may have more than one state . . . 166
25.2 An event queue uses POPREXEN with a new priority . . . 167
25.3 Trie search: a prefix is unfinished work . . . 167
25.4 Sorted postings: the frontier is two positions . . . 168
25.5 Compose a TTL with a capacity limit . . . 169
25.6 The same algorithm, a changed output contract . . . 169

A Every source entry, by coding move 170

Part I

Algorithms primer

How to use the algorithms primer

The primer is a quick working tour, not a substitute for solving problems. Each section gives an ordinary input, the data kept while processing it, a short Python implementation, and a case that breaks an appealing shortcut. Read it when an algorithm's name is familiar but its operations are not yet automatic. The coding-pattern chapters that follow go a step further: they concentrate on where the condition, state update, and answer belong in new problems.

Before choosing an algorithm, identify what a single operation must do. A list gives inexpensive indexed reads and end appends, but inserting at its front moves elements. A set answers exact membership questions; a dictionary also remembers a value or count per key. A deque removes from either end without shifting a list. A heap finds the smallest pending priority; it does not keep an entire list sorted. An integer used as a bitmap is compact only when its bit positions have a clear meaning.

Question to ask	Good first representation	Check before coding
Read by position?	Dynamic array	Must order and random access be kept?
Count earlier keys?	Dictionary of counts	What is a valid, hashable key?
Explore in hop order?	Queue and visited states	When is a state claimed?
Explore every choice?	Frames and a working path	What is restored on return?
Ask many range queries?	Prefix table or segment tree	Can the input change?
Reuse subproblems?	Memo or DP table	What parameters define one future?

Imagine an inventory of numbered items. "Does item 42 exist?" suggests a set. "How many copies?" changes that set to a dictionary of counts. "What is the next available item above 42 after inserts and deletes?" needs ordered operations, such as a search tree. "How many units were stocked between two indices?" is a prefix-sum question until the values start changing, when a segment tree can maintain range sums. The data did not change much; the *question* did.

For every code example, say what its input permits: negative values, duplicate keys, empty collections, cycles, or weighted edges can change the appropriate technique. Trace one normal case and one boundary case before using the example as a template. A return value is not correct just because the loop ran to completion; it must satisfy the stated contract. Each chapter introduces its terms, imports what its runnable listings need, and shows calls with expected results. Some short excerpts deliberately show only the decision inside an algorithm; treat those as sketches, not independently runnable functions. The companion notebook keeps runnable cells next to the text and distinguishes sketches from those cells.

The tour begins with arrays and linked structures, then builds recursion, trees, hashing, sorting, graphs, bit and string operations, and dynamic programming. N-Queens, straight runs, Candy, and rainwater show how the same small decisions transfer across problem statements. If you already know one topic, skip it: the parts are designed to be read independently.

CHAPTER 1

Sequences: positions, spans, and remembered sums

An array is a row of numbered slots; its position numbers, called *indices*, start at zero. Knowing an index lets us go straight to its slot, but changing other items' positions is not necessarily cheap. A *window* tracks neighboring items with no gaps; *two pointers* are two moving indices; a *prefix sum* totals items from the start up to a chosen boundary. These small ideas lead to different programs depending on the question. Here n is the number of items: $O(n)$ means work grows at most in proportion to n, whereas $O(1)$ means an amount of work independent of n.

1.1 An array's position promise and a dynamic array's bill

A fixed-length array reserves adjacent memory slots for a known number of elements. Adjacent (or *contiguous*) slots permit indexing and replacement at position `i` in constant time: finding slot `i` does not require visiting the earlier slots. If the array is full, appending requires a different, larger array. Even with free capacity, inserting at position `i` must shift the occupants of positions `i`, `i+1`, and so on. For example, inserting 5 between 4 and 6 in `[4, 6, 8, 10]` shifts 6, 8, and 10. Deleting at the front shifts almost everything back. These edits cost $O(n)$, not because finding the position is hard, but because preserving order is work.

A Python `list` is a *dynamic* array of references (links to Python objects, rather than copies of those objects). Its *length* is the number of items; its *capacity* is the number of slots reserved, including any unused ones. `append(x)` uses a free slot when available; occasionally it must allocate more space and copy the existing references. That particular append can cost $O(n)$. When capacity grows by a fixed factor, a sequence of n appends does $O(n)$ total copying: the infrequent expensive appends are paid for by the many cheap ones. We call each append $O(1)$ *amortized* (averaged across the sequence), not $O(1)$ worst-case. In a conceptual doubling array, the fifth append copies four references after capacity four fills, but the next three appends fit without copying; the total number of copies across doubling steps is less than twice the final length. For a Python list `a`, `a[i]` is $O(1)$; `a.insert(i, x)` and `a.pop(0)` remain $O(n)$, because spare capacity cannot avoid shifting. `a.pop()` removes the last item in $O(1)$ amortized time. An array is therefore an excellent default when positions matter; repeated front removals suggest a queue instead.

Break test. A list has plenty of unused capacity. Does inserting at index zero now take constant time? No: capacity removes a possible reallocation, but every existing reference still has to move one slot.

1.2 Fixed-length windows: exchange one item

Suppose we need the largest sum of exactly `k` consecutive numbers. Re-summing all `k` numbers at each starting position repeats work. After one initial sum, moving right changes only two memberships: the old leftmost number leaves, and the new rightmost number enters. The window's *length* is fixed; its sum and its best answer are separate state. An *invariant* is a fact about that state that holds before every loop step and is restored by the step.

```python
def max_window_sum(nums, k):
    """Sequence of ints, 1 <= k <= len(nums); return greatest
    length-k sum. Do not mutate nums; reject invalid k."""
    if not 1 <= k <= len(nums):
        raise ValueError("k must be between 1 and len(nums)")
    window = sum(nums[i] for i in range(k))
    best = window
    for right in range(k, len(nums)):
        window += nums[right] - nums[right - k]
        best = max(best, window)
    return best
```

For `[2, -5, 3, 4, -1]` and `k = 3`, the first sum is 0. Admitting 4 and evicting 2 makes it 2; admitting `-1` and evicting `-5` makes it 6. The return is 6. Negative numbers do not harm this fixed-membership exchange. Initialization costs $O(k)$, and each of the remaining positions takes constant time: $O(n)$ time and $O(1)$ auxiliary space (extra working memory beyond the input and answer). Starting `best` at zero would incorrectly allow an empty window when every legal sum is negative.

> **State.** The current length-`k` sum and the best completed sum.
>
> **Invariant.** Before processing `right`, `window` sums `nums[right-k:right]`; `best` includes that window.
>
> **Update.** Add the entering value, subtract the departing value, then compare the new valid window with the best.

Break test. With `[-4, -2]` and `k = 1`, the answer is `-2`, not zero. A best value initialized to zero describes an illegal candidate.

Source notes: #12.

1.3 Variable-length windows: repair a condition

Now ask for the shortest nonempty span whose sum reaches a positive *target* (the required threshold). If every number is *nonnegative* (zero or positive), extending a span cannot lower its sum; removing its leftmost number cannot raise it. This is the monotonic, or one-direction, fact that makes a moving left boundary sound. Expand until the current span qualifies. While it qualifies, record its length *before* removing the leftmost number: removal may make it fail.

```python
def shortest_at_least(nums, target):
    """Nonnegative integer sequence, positive target; return the
    shortest qualifying length, or 0 if none. Do not mutate nums."""
    if target <= 0 or any(x < 0 for x in nums):
        raise ValueError("target > 0 and nonnegative nums required")
    left = total = 0
    best = len(nums) + 1
    for right, value in enumerate(nums):
        total += value
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == len(nums) + 1 else best
```

With target 7 and `[2, 1, 5, 2, 3, 2]`, reaching 5 first gives a length-three span with sum 8. Removing 2 makes it fail. After admitting the next 2, removing 1 leaves the valid span `[5, 2]` of length two; record that length *before* removing 5. Each pointer crosses at most `n` positions, so even the nested `while` takes $O(n)$ total time and $O(1)$ extra space. Empty input returns zero.

Do not transfer this rule to signed sums merely because they also form contiguous spans. If we removed the input guard, on `[3, -2, 5]` with target 5 the same shrink loop would record length three, remove 3, and stop shrinking at sum 3. It would miss the single value 5: removal of the negative number would have *raised* the sum. A prefix-based method, or a different specialized structure, is needed when that monotonic fact disappears.

> **State.** The inclusive span from `left` to `right`, its sum, and the shortest valid length seen.
>
> **Invariant.** After each shrink loop, the current span is below target; every just-removed left endpoint had a valid span recorded before removal.
>
> **Update.** Extend on the right; while valid, record and then advance the left endpoint.

Break test. Delete the negative-input guard and try `[3, -2, 5]` with target 5. The monotonic-window rule reports 3 instead of 1; adding a different shrink condition does not restore its input promise.

Source notes: #11, #53, #54.

This is a change of *contract*, not a change in variable names. Let `nums` be `[2, 1, 5, 2, 3, 2]`. `max_window_sum(nums, 3)` returns 10, a *sum* over exactly three positions; asking for the shortest span with sum at least 7 returns 2, a *length*. The first routine swaps exactly one item per move and accepts negative numbers. The second may shrink repeatedly and needs nonnegative input.

1.4 Two pointers need a reason to move

Two pointers are not an algorithm by themselves. In a sequence *sorted from smallest to largest*, the ends let us rule out many possible pairs at once. If the smallest remaining value plus the largest is below the target, the smallest cannot pair with *any* remaining value; advance the left pointer. If their sum is above the target, discard the largest instead. The values may be negative; it is sorted order, not positivity, that justifies these steps.

```
def two_sum_sorted(nums, target):
    """Nondecreasing integer sequence and integer target; return
    two distinct indices whose values sum to target, or None.
    Do not mutate nums; caller guarantees sorted order."""
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return left, right
        if total < target:
            left += 1
        else:
            right -= 1
    return None
```

For `[1, 3, 4, 8]` and target 7, sums 9, then 5, then 7 lead to indices `(1, 2)`. Each move permanently excludes one endpoint, so time is $O(n)$ and extra space is $O(1)$. The loop condition `left < right` forbids using one item twice. Calling this function on an unsorted array does not merely make it slower: with `[8, 1, 6]` and target 7 it returns `None`, missing the pair at original indices `(1, 2)`.

If sorting is not promised

The new contract allows any input order but still asks for original indices. Sorting a copy would change those indices; discarding an endpoint without sorted order is unjustified. Instead, use a *hash map*—a Python `dict` that looks up a stored value in expected $O(1)$ time—to remember earlier values and their indices. Ask for the complement *before* storing the current item, so one index cannot pair with itself.

```
def two_sum_unsorted(nums, target):
    """Any integer sequence and integer target; return two
    distinct original indices, or None. Do not mutate nums."""
    seen = {}
    for right, value in enumerate(nums):
        earlier = seen.get(target - value)
        if earlier is not None:
            return earlier, right
        seen[value] = right
    return None
```

The unsorted `[8, 1, 6]` now returns `(1, 2)` for target 7. The two 3s in `[3, 3]` return `(0, 1)` for target 6, not the same index twice. Unlike inward pointers on sorted input, this version uses $O(n)$ extra space; its $O(n)$ time is *expected* because hash lookups can occasionally be slower.

On *unsorted* input, pointers can also track *progress* rather than compare values. A read pointer examines each item; a write pointer marks where the next retained item belongs. Neither pointer relies on numeric order. In this example, retaining nonzeros is stable: their original relative order is unchanged.

```
def move_zeroes(nums):
    """List of integers; move zeroes to the end in place,
    preserve nonzero order, and return the same list."""
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1
    for i in range(write, len(nums)):
        nums[i] = 0
    return nums
```

Starting from `[0, 4, 0, -3, 2]`, the first pass writes 4, `-3`, and 2 into the first three slots; the second pass fills the remainder with zeros. The result is `[4, -3, 2, 0, 0]`. Before each read, slots before `write` are precisely the nonzeros already read, in order. A write never goes ahead of the read, so it cannot destroy an unexamined item. This costs $O(n)$ time and $O(1)$ additional space, but unlike the sorted pair routine it *mutates* the input.

Break test. Do not use the inward pair-sum rule on `[8, 1, 6]` with target 7; sorting would make it valid but would also change the original indices. Do not advance `write` for zeros in the second routine: its settled prefix would then contain holes.

Source notes: #7, #55.

1.5 Prefix sums: subtract away an earlier span

For many queries about the same sequence, store the sum of everything *before* each boundary. Let `prefix[0]` be zero, and let `prefix[i]` include positions before `i`. A *half-open* range `nums[left:right]`

includes the left index but excludes the right. Its sum is `prefix[right] - prefix[left]`. The shared part cancels, even if some terms are negative; no monotonic assumption is involved.

```
def build_prefix(nums):
    """Integer sequence; return prefix sums starting with zero.
    Do not mutate nums; empty input returns [0]."""
    prefix = [0]
    for value in nums:
        prefix.append(prefix[-1] + value)
    return prefix

def range_sum(prefix, left, right):
    """Prefix built above, 0 <= left <= right < len(prefix);
    return sum of original positions [left:right]."""
    if not 0 <= left <= right < len(prefix):
        raise IndexError("invalid half-open range")
    return prefix[right] - prefix[left]
```

For `[3, -2, 4, -5]`, the prefix array is `[0, 3, 1, 5, 0]`. Positions `[1:3]` sum to `prefix[3] - prefix[1]`, namely $5 - 3 = 2$. The empty range `[2:2]` sums to zero. Building costs $O(n)$ time and space; each later query is $O(1)$. Keep the extra initial zero: omitting it makes a range that starts at position zero awkward to represent.

1.6 Prefix sums plus a hash map: count unknown starts

Suppose we want to count *all nonempty* subarrays (contiguous spans) summing to `target`, not answer one known range. At current running sum `total`, an earlier prefix of value `total - target` marks one valid left boundary. Several earlier prefixes can have that same value, so remember a *count*, not just whether it appeared. Query the map *before* inserting the current prefix: a subarray must start before it ends.

```
def count_subarrays(nums, target):
    """Integer sequence (negatives allowed), integer target;
    return number of nonempty target-sum subarrays. No mutation."""
    counts = {0: 1}           # one empty prefix, before index zero
    total = matches = 0
    for value in nums:
        total += value
        matches += counts.get(total - target, 0)
        counts[total] = counts.get(total, 0) + 1
    return matches
```

With `[1, -1, 1]` and target 1, the first prefix 1 finds the empty prefix and contributes one. The next prefix is zero; there are now *two* earlier zero prefixes. The final prefix 1 pairs with both, giving three subarrays in total. Negative values are harmless because this is an equality of prefix sums, not a shrinking rule. Expected time is $O(n)$ with hash-map operations and space is $O(n)$. For `[0, 0]` and target zero, the correct count is three, not two: both singletons and the whole span qualify.

Here is another small contract change with different state and output. For `[1, -1, 1]`, one *specified* range `[0:3]` has sum 1, returned by `range_sum`; asking how many *possible* ranges sum to 1 returns 3 from `count_subarrays`. A prefix array retains boundary sums for known endpoints; a map of *frequencies* (numbers of earlier occurrences) counts all possible starting boundaries. A set that only records whether zero appeared would miss one of those three spans.

State. A running prefix, its earlier-frequency map, and a count of completed matching spans.

Invariant. Before each lookup, the map contains exactly the prefixes *before* the current end, including the empty prefix.

Update. Add the current value, count matching earlier prefixes, then store the new prefix for later ends.

Break test. Inserting the new prefix before looking it up counts a zero-length span when target is zero. Starting without the empty prefix misses every valid span beginning at position zero.

Source notes: #2.

Try it

Each definition listing above is executable on its own and needs no imports. Paste them into one Python file in chapter order, then run this *driver* (a block of calls, not another function); its assertions specify the expected results. Only the driver depends on the earlier definitions.

```
nums = [2, 1, 5, 2, 3, 2]
assert max_window_sum(nums, 3) == 10
assert shortest_at_least(nums, 7) == 2
assert two_sum_sorted([1, 3, 4, 8], 7) == (1, 2)
assert two_sum_unsorted([8, 1, 6], 7) == (1, 2)
assert move_zeroes([0, 4, 0, -3, 2]) == [4, -3, 2, 0, 0]
prefix = build_prefix([1, -1, 1])
assert range_sum(prefix, 0, 3) == 1
assert count_subarrays([1, -1, 1], 1) == 3
print("sequences: OK")  # sequences: OK
```

CHAPTER 2

Links and worklists: order without indexes

An array finds items by numbered position. A *linked list* chains *nodes*, objects holding a value and a link to another node. A stack serves the newest waiting item first; a queue serves the oldest. An *operation* is an action such as `pop()`; an *invariant* is a fact about state that survives each action. Later chapters use these small mechanisms in larger problems.

2.1 Singly linked lists: keep the next link before changing it

A singly linked node references its *successor* (next node). Reaching the eighth node takes seven links, but inserting after a *known* node changes only two links. Removing a node requires its *predecessor*, the node just before it: searching from the *head* (first node) costs $O(n)$ for n nodes, but unlinking takes $O(1)$, a fixed number of changes. An empty list has head `None`. An *acyclic* chain eventually reaches `None` instead of revisiting a node; the routines below require this.

```
class Node:
    """A value and its optional successor."""
    def __init__(self, value, next_node=None):
        self.value = value
        self.next = next_node

def insert_after(node, value):
    """Existing non-None node; insert after it and return new node."""
    node.next = Node(value, node.next)
    return node.next

def remove_first(head, value):
    """Acyclic chain or None; unlink first match, return new head."""
    dummy = Node(None, head)
    previous = dummy
    while previous.next is not None:
        if previous.next.value == value:
            previous.next = previous.next.next
            break
        previous = previous.next
    return dummy.next

def reverse(head):
    """Acyclic chain or None; reverse in place, return new head."""
    previous, current = None, head
    while current is not None:
        after = current.next
        current.next = previous
        previous, current = current, after
    return previous
```

The temporary `dummy` is a *sentinel*, a fake node before the real head. Removing the first real node therefore uses the same assignment as removing any later node. From 1, 2, 3, removing 1 returns a head at 2; removing 9 changes nothing. The $O(n)$ search, not the $O(1)$ unlink, dominates the cost.

Change "remove the *first* 2" to "remove *every* 2." On 2, 2, 3 the current routine leaves 2, 3, but the new answer must be 3. After unlinking, keep `previous` in place and inspect its new `previous.next`; advance only when nothing was removed. Otherwise adjacent matches are skipped.

For reversal, `previous` holds the reversed prefix and `current` heads the untouched suffix. On 1, 2, 3, save 2, point 1 to `None`; then point 2 to 1 and 3 to 2. Return 3 as the new head. Saving `after` *before* changing `current.next` prevents losing the suffix. This takes $O(n)$ time and $O(1)$ extra working space.

> **State.** A previous reversed prefix and a current untouched suffix.
>
> **Invariant.** Before each reversal step, every link in the prefix points backward; `current` still reaches all unprocessed nodes.
>
> **Update.** Save the next node, reverse one link, then advance both references.

Break test. Reverse an empty chain and a one-node chain. Then remove the head of a two-node chain: code that keeps only the old head cannot return the correct new head.

Source notes: #6, #76.

2.2 Two-way links and permanent boundary nodes

A *doubly linked* node also stores its predecessor. Given an *owned* node (linked into this very list), joining its neighbors removes it in $O(1)$; finding a value still costs $O(n)$. A *deque*, or double-ended queue, works at either end. Permanent head and tail sentinels make both ends ordinary neighbor cases: they point directly to each other when empty and are never returned as data.

```
class DNode:
    """A value with links in both directions."""
    def __init__(self, value=None):
        self.value = value
        self.prev = self.next = None

class LinkedDeque:
    """Doubly linked sequence with two permanent sentinels."""
    def __init__(self):
        self.head, self.tail = DNode(), DNode()
        self.head.next, self.tail.prev = self.tail, self.head

    def insert_before(self, following, value):
        """Owned node except head (tail OK); insert, return new node."""
        prior = following.prev
        node = DNode(value)
        node.prev, node.next = prior, following
        prior.next = node
        following.prev = node
        return node

    def remove(self, node):
        """Owned non-sentinel node; unlink and return its value."""
        node.prev.next = node.next
        node.next.prev = node.prev
```

```
        node.prev = node.next = None
        return node.value
```

Inserting 4 and 9 before `tail` yields head, 4, 9, tail in both directions. The invariant is that neighbors point back to one another and the sentinels remain. Check `head.next is tail` *before* removing from the front: identity asks whether these are the very same node. Otherwise remove `head.next` to get 4, then the saved node to get 9. Ownership is an input promise because searching to verify it costs $O(n)$. Extra links cost memory; a Python list is simpler for indexed scans.

Break test. On an empty list, `head.next` is the tail sentinel. Do not pass it to `remove`: that routine accepts only real nodes, and calling it could corrupt the sentinel links before failing. Check identity before removing from either end.

2.3 Fast and slow pointers: distance is state

Move one reference (`slow`) one link per step and another (`fast`) two. When fast falls off an acyclic list, slow is at the middle; here even lengths select the *second* middle. Check `fast.next` before reading `fast.next.next`. A *cycle* revisits a node instead of reaching `None`; `middle` requires no cycle, while `has_cycle` detects one.

```
def middle(head):
    """Acyclic chain or None; return second middle/None; no edits."""
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow

def has_cycle(head):
    """Next-linked chain or None; return cycle bool; no edits."""
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

On 7, 8, 9, 10, slow visits 8, then 9 as fast falls off. Change the contract to the *first* middle and the answer becomes 8: after handling an empty head, advance only if both `fast.next` and `fast.next.next` exist. In a cycle fast cannot fall off; its lead gains one link per round, so they meet. Compare nodes with `is`, not by stored value, and test *after* moving: both start at the head, even without a cycle. Both routines use $O(1)$ space and linear time over the chain or cycle reached.

Break test. Two distinct nodes both holding 7 are not a cycle; a single node pointing back to itself is. Compare identity, not values.

Source notes: #10.

2.4 Stack versus queue: which promise gets served?

A stack serves the *newest* pending item first (LIFO means last in, first out); a queue serves the *oldest* (FIFO means first in, first out). To enforce these invariants, use a Python list's `append`/last-element `pop` for a stack, or a `collections.deque`'s `append`/`popleft` for a queue.

Changing just "serve newest" to "serve oldest" changes the next output on arrivals `["Ana", "Bo", "Cy"]` from "Cy" to "Ana": the *Try it* script runs both and gets `("Cy", "Ana")`. Loading n items costs $O(n)$; either end removal is $O(1)$ amortized (averaged over many calls). For a list `a`, `a.pop(0)` instead shifts references in $O(n)$. Check emptiness before popping when it means the worklist is done; otherwise let an empty removal raise `IndexError`. The operation changes the container; the invariant explains why it removes the requested item.

2.5 A monotonic stack finishes earlier questions

A *monotonic stack* keeps its pending values ordered in one direction. For each position's nearest *strictly greater* value to its right, save indices still awaiting an answer. Their values remain non-increasing from stack bottom to top: a larger arrival resolves the top and perhaps more below it, but an equal one does not.

```
def next_strictly_greater(nums):
    """Integer sequence; return next greater/None per index; no edits."""
    answer = [None] * len(nums)
    waiting = []
    for i, value in enumerate(nums):
        while waiting and nums[waiting[-1]] < value:
            earlier = waiting.pop()
            answer[earlier] = value
        waiting.append(i)
    return answer
```

Trace `[2, 1, 3, 2, 4]`: indices 0 and 1 wait with values 2 and 1. The arriving 3 pops both, answering each with 3; the next 2 waits, then 4 answers it and the earlier 3. Only 4 stays unanswered, giving `[3, 3, 4, 4, None]`. A single arrival may pop many indices, but each enters and leaves at most once: $O(n)$ total time and $O(n)$ extra space.

Break test. For `[2, 2]`, both answers are `None`. Using "less than or equal" in the pop condition calls an equal value strictly greater.

Source notes: #23.

2.6 A queue schedules processing; it is not the answer

A *graph* has nodes connected by links called edges. The input `graph` below maps each node to an *iterable* of neighbors (something a loop can read, such as a list). A *hashable* node can be a dictionary or set key, such as an integer. *Breadth-first search* (BFS) serves discoveries from a queue: one-edge neighbors are processed before two-edge ones. Its rhythm is pop–process–expand–enqueue. Claim each node at *enqueue*, so it gets only one future turn; this small version counts reached nodes, not distances or paths.

```python
from collections import deque

def reachable_count(graph, start):
    """Hashable-node graph and start; return reachable count incl. start.
    Missing keys have no neighbors; do not mutate graph."""
    pending = deque([start])
    claimed = {start}
    count = 0
    while pending:
        node = pending.popleft()             # pop
        count += 1                           # process
        for neighbor in graph.get(node, ()):  # expand
            if neighbor not in claimed:
                claimed.add(neighbor)
                pending.append(neighbor)   # enqueue
    return count
```

With edges 0 to 1 and 2, 1 to 2, and 2 to 0, the queue runs `[0]`, `[1, 2]`, `[2]`, then empty; the count is 3. Node 2 is already *claimed* when 1 examines it, but is not yet processed. Thus the queue is pending work, not the answer. For V reached nodes and E inspected edges, work is $O(V+E)$ with $O(V)$ working space.

The invariant is that `pending` holds claimed but unprocessed nodes; `count` changes only on pop, and each node gets one turn.

Break test. On the three-edge cycle above, omitting the claimed set never empties the queue. Marking only when popping also allows two parents to queue the same not-yet-popped child.

Source notes: #39.

2.7 Adapters: paying for the order you did not get

Two stacks can implement a queue without shifting an array's front. New items enter `inbox`; only when `outbox` empties do we transfer them, reversing their order to expose the oldest. Items still in `outbox` must be served before newer arrivals.

```python
class QueueViaStacks:
    def __init__(self):
        self.inbox, self.outbox = [], []

    def enqueue(self, value):
        """Add value to back; return None."""
        self.inbox.append(value)

    def dequeue(self):
        """Remove/return oldest; raise IndexError if empty."""
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox.pop()  # raises IndexError if still empty
```

Enqueue A, B: the first dequeue transfers both and returns A. Enqueue C: the next dequeue returns B from `outbox`, not C; only then does another transfer expose C. One transfer costs $O(n)$, but each item

enters and leaves each stack at most once. Thus enqueue and dequeue take $O(1)$ *amortized* time over a sequence, with $O(n)$ storage.

One queue can also implement a stack, but with a different bill. After appending a new item, rotate older ones behind it so the newest is always at the front. Only FIFO operations `append` and `popleft` are used.

```
from collections import deque

class StackViaQueue:
    def __init__(self):
        self.line = deque()

    def push(self, value):
        """Make value the new top; return None."""
        self.line.append(value)
        for _ in range(len(self.line) - 1):
            self.line.append(self.line.popleft())

    def pop(self):
        """Remove/return newest; empty deque raises IndexError."""
        return self.line.popleft()
```

Push 1, 2, 3: the front-to-back order becomes `[1]`, then `[2, 1]`, then `[3, 2, 1]`; pop returns 3. After a push leaves n items, it has rotated the $n - 1$ older ones, costing $O(n)$, while pop takes $O(1)$. Across n consecutive pushes, roughly $1 + 2 + \cdots + n$ operations grow with n squared: there is *no* constant amortized push bound. Rotating only on pop would trade the costs: cheap push, linear-time pop.

Break test. Enqueue A, B, dequeue A, enqueue C: transferring C onto nonempty `outbox` returns C before B. In the one-queue stack, skip rotation after push and pop returns the oldest item.

Try it

Paste the complete definition listings (imports included) in order, then run this driver. It uses those functions and classes; none is an incomplete transition sketch. Assertions specify the outputs.

```
from collections import deque

head = Node(1, Node(2, Node(3)))
assert reverse(remove_first(head, 1)).value == 3
nodes = Node(7, Node(8, Node(9, Node(10))))
assert middle(nodes).value == 9 and not has_cycle(nodes)
links = LinkedDeque()
saved = links.insert_before(links.tail, 4)
assert links.remove(saved) == 4
stack, line = ["Ana", "Bo", "Cy"], deque(["Ana", "Bo", "Cy"])
assert (stack.pop(), line.popleft()) == ("Cy", "Ana")
assert next_strictly_greater([2, 1, 3]) == [3, 3, None]
assert reachable_count({0: [1], 1: [0]}, 0) == 2
queue, pile = QueueViaStacks(), StackViaQueue()
for value in ("A", "B"):
    queue.enqueue(value)
    pile.push(value)
assert (queue.dequeue(), pile.pop()) == ("A", "B")
print("links and queues: OK")  # links and queues: OK
```

CHAPTER 3

Recursion and the search tree

Recursion asks the same function for a smaller answer, then finishes the current one. A *call frame* saves local variables and where to resume; a *base case* returns without another call. Before coding, state the *contract* (allowed input and output), how calls get smaller, and the *invariant* (a fact each step keeps true). Every listing here includes a runnable example.

3.1 Make the base case part of the contract

For a nonnegative integer n, `factorial(n)` returns $n! = 1 \cdot 2 \cdots n$; the empty product is $0! = 1$. Negative inputs are invalid, and $n - 1$ is the smaller problem:

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(0), factorial(3))  # 1 6
```

For `factorial(3)`, frames for 3, 2, and 1 wait for smaller answers. Zero returns 1; on the way back they return 1, 2, then 6. Each frame needs only its own n and the next call's result.

Break test. Without the negative check, `factorial(-1)` calls `factorial(-2)`, which calls `factorial(-3)`. None reaches zero. Without the zero case, `factorial(0)` never returns.

There are $n+1$ calls: $O(n)$ steps and saved frames, where $O(n)$ means growth proportional to n, ignoring constants. Python limits nested calls, so a loop suits very large n. The input must be an integer, not a fraction.

3.2 Repeated subproblems: cache or carry two values

A *recurrence* uses smaller answers: $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. Naive recursion recomputes F_3 through both branches of F_5. *Memoization* saves answers by input; `@lru_cache(maxsize=None)` remembers each helper result. If only F_n is needed, a loop can instead carry the last two numbers:

```
from functools import lru_cache

def fib(n):
    if n < 0:
        raise ValueError("n must be nonnegative")

    @lru_cache(maxsize=None)
    def value(k):
        if k < 2:
            return k
        return value(k - 1) + value(k - 2)

    return value(n)

def fib_iter(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(fib(5), fib_iter(5), fib_iter(0))  # 5 5 0
```

In `fib(5)` the cached F_3 is reused: $O(n)$ additions but $O(n)$ cache and call stack (the cache is fresh per call). Before loop step i, `a` is F_i and `b` is F_{i+1}; the pairs for $i = 0, 1, 2, 3, 4, 5$ are `(0, 1)`, `(1, 1)`, `(1, 2)`, `(2, 3)`, `(3, 5)`, `(5, 8)`. The loop uses $O(1)$ number variables and avoids the depth limit; their bit sizes still grow.

Break test. A base case saying "return 1 whenever $n \leq 2$" quietly changes F_0 from 0 to 1. Always test index zero.

3.3 A tree frame returns one subtree answer

A binary tree has a *root* and at most two children per node. A *subtree* is one node and everything below it; *acyclic* means no child link leads back to an earlier node. For a finite tree of numbers, the sum of an empty subtree is zero. The sum of a nonempty subtree is its root value plus the two child sums. This is the same contract at every depth, including leaves:

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left, self.right = left, right

def sum_tree(root):
    if root is None:
        return 0
    return root.value + sum_tree(root.left) + sum_tree(root.right)

root = Node(4, Node(2), Node(7, Node(1)))
print(sum_tree(root), sum_tree(None))  # 14 0
```

The sample's left frame returns 2, right returns $7 + 1 = 8$, and root returns 14. `None` gives every absent child a zero answer. Each of n nodes is visited once; the stack holds $O(h)$ frames for tree height h, possibly n for a chain. A cycle never ends, while a shared child is counted twice: this contract requires an unshared, acyclic tree.

Source notes: #33, #34.

3.4 A maze has cycles: decide who owns "visited"

Now imagine a rectangular maze of strings, where `"##"` is a wall and every other character is open. The input consists of the maze, a start cell and a goal cell, each a `(row, col)` pair. A move may go up, down, left or right; an out-of-bounds or blocked start has no route. *Depth-first search* (DFS) follows one route until it ends before trying another. If the output is only `True` or `False` for reachability, permanently mark every open cell on first discovery:

```
def maze_reachable(maze, start, goal):
    height = len(maze)
    width = len(maze[0]) if height else 0
    seen = set()

    def dfs(r, c):
        if not (0 <= r < height and 0 <= c < width):
            return False
        if maze[r][c] == "#" or (r, c) in seen:
            return False
        seen.add((r, c))
        if (r, c) == goal:
            return True
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            if dfs(r + dr, c + dc):
                return True
        return False

    return dfs(*start)

maze = ["...", "..."]
print(maze_reachable(maze, (0, 0), (0, 2)))  # True
```

`seen` permanently marks first discovery; revisit cannot change reachability. Mark before exploring neighbors to avoid cycles. For R rows and C columns, time and set/stack space are $O(RC)$. If the output changes to a *shortest route*, use breadth-first search (BFS): it explores cells by step count, and a `parent` dictionary can store each cell's predecessor. DFS first discovery does not guarantee a shortest route.

Now ask "how many *simple paths*?" A simple path cannot reuse a cell, but other paths may. Return 0 for an invalid or blocked endpoint, 1 for an open start equal to the goal:

Try it: same maze, different question.

```
def count_simple_paths(maze, start, goal):
    height = len(maze)
    width = len(maze[0]) if height else 0
    on_path = set()

    def dfs(r, c):
        if not (0 <= r < height and 0 <= c < width):
            return 0
        if maze[r][c] == "#" or (r, c) in on_path:
            return 0
        if (r, c) == goal:
            return 1
        on_path.add((r, c))
        ways = 0
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            ways += dfs(r + dr, c + dc)
        on_path.remove((r, c))
        return ways

    return dfs(*start)

maze = ["...", "..."]
print(count_simple_paths(maze, (0, 0), (0, 2)))  # 4
```

The goal returns before marking; every other mark is undone before trying the next path, even if a variant exits early on success. **Contract change:** on the same two-by-three open maze, reachability returns `True`, but counting returns 4: two paths start right, two down. A permanent mark reached from above can block the lower-left cell and lose the downward paths. Counting may take exponentially many branches but uses only $O(RC)$ marks and stack frames. Deep grids may need an explicit stack.

Break test. Keep `on_path` marked across sibling branches; the two-by-three example then undercounts.

Source notes: #38, #74.

3.5 Backtracking: one shared path, different answer events

Backtracking restores shared choices after each recursive call. Given distinct `items`, a *permutation* arranges all items in order; a *combination* chooses exactly k without order; a *subset* chooses any number. The same growing `path` needs different admission rules and different moments to record:

```
def permutations(items):
    result, path = [], []
    used = [False] * len(items)

    def visit():
        if len(path) == len(items):
            result.append(path.copy())
            return
        for i in range(len(items)):
            if used[i]:
                continue
            used[i] = True
            path.append(items[i])
            visit()
```

```
            path.pop()
            used[i] = False

    visit()
    return result

def combinations(items, k):
    if not 0 <= k <= len(items):
        return []
    result, path = [], []

    def choose(start):
        if len(path) == k:
            result.append(path.copy())
            return
        needed = k - len(path)
        for i in range(start, len(items) - needed + 1):
            path.append(items[i])
            choose(i + 1)
            path.pop()

    choose(0)
    return result

def subsets(items):
    result, path = [], []

    def grow(start):
        result.append(path.copy())
        for i in range(start, len(items)):
            path.append(items[i])
            grow(i + 1)
            path.pop()

    grow(0)
    return result

items = ["A", "B"]
print(permutations(items))     # [['A', 'B'], ['B', 'A']]
print(combinations(items, 2))  # [['A', 'B']]
print(subsets(items))          # [[], ['A'], ['A', 'B'], ['B']]
```

For three items, choosing A, B, C records `["A", "B", "C"]`; undo C then B to try A, C, B. **Contract change:** on the same two items, permutations returns A–B and B–A, combinations returns only A–B, and subsets also includes empty and one-item choices. `used` allows every unused index; increasing `start` forbids reversed duplicates, while `needed` skips branches too short to reach k. Permutations and combinations record completed paths; subsets record every call. Each sibling sees the same restored `path`; save a copy, not the mutable list. Empty input (or $k = 0$) gives `[[]]`; invalid k gives `[]`. Equal input values can yield duplicate-looking permutations by position.

There are $n!$ permutations, $\binom{n}{k}$ combinations (ways to choose k of n), or 2^n subsets, each requiring up to n copied values. Working state is $O(n)$ besides the returned lists.

State. A partial `path`; either available indices or a next allowed `start` index.

Invariant. At each frame the path obeys all earlier choices, and sibling branches start with identical parent state.

Update. Choose, recurse, undo; record at leaves for permutations and fixed-length combinations, but at every frame for subsets.

Break test. Saving subsets only at leaves loses `["A"]` from `["A", "B"]`; omitting `pop()` leaks a choice into the next branch.

Source notes: #30, #31.

3.6 N-Queens: name a constraint so it is cheap to test

Queens attack by row, column, and diagonal. Place one per row so row conflicts disappear; return a column per row, such as `[1, 3, 0, 2]` for $n = 4$. A down-right diagonal keeps `row - col` constant, a down-left one `row + col`. Three sets hold occupied columns and diagonal names; record a solution only after filling all rows. For $n = 0$ there is one empty board:

```
def n_queens(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    solutions, placed = [], []
    columns, down, up = set(), set(), set()

    def put(row):
        if row == n:
            solutions.append(placed.copy())
            return
        for col in range(n):
            d, u = row - col, row + col
            if col in columns or d in down or u in up:
                continue
            columns.add(col)
            down.add(d)
            up.add(u)
            placed.append(col)
            put(row + 1)
            placed.pop()
            columns.remove(col)
            down.remove(d)
            up.remove(u)

    put(0)
    return solutions

print(n_queens(4))  # [[1, 3, 0, 2], [2, 0, 3, 1]]
```

For $n = 4$, columns 1, 3, 0, 2 in consecutive rows make one solution; undo the list append and all three marks before trying a sibling. The sets give expected $O(1)$ attack tests. At most $n \cdot n!$ candidates are checked; the sets, path, and frames use $O(n)$ space besides $O(n)$ per saved solution.

Break test. With only the column set, queens at `(0, 0)` and `(1, 1)` pass despite sharing `row - col`. With only one diagonal set, the other direction goes unchecked. Check `n_queens(2) == []`, and check that $n = 4$ has exactly two solutions; a forgotten removal can incorrectly eliminate one.

Source notes: #113.

CHAPTER 4

Trees as traversals and indexes

A tree has a top *root*; every node owns part of the data: a binary subtree (itself and descendants), a segment tree's array interval, a quadtree's square, or a trie's string prefix. The useful question is what that region lets us skip or summarize.

4.1 Binary trees: order of visits is an answer choice

Given a binary decision tree, a UI needs a parent-first outline, a child-first removal order, or rows of choices at equal depth. Nodes have up to two children; `None` means empty. Assume no cycles or shared children. Depth-first search (DFS) follows a branch before its sibling: record on entry (*preorder*), between children (*inorder*), or on exit (*postorder*). Breadth-first search (BFS) uses a first-in-first-out *queue* for levels; Python's `deque` pops its front efficiently. This one notebook cell returns all four views:

```
from collections import deque

class TreeNode:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left, self.right = left, right

def dfs_orders(root):
    preorder, inorder, postorder = [], [], []

    def visit(node):
        if node is None:
            return
        preorder.append(node.key)
        visit(node.left)
        inorder.append(node.key)
        visit(node.right)
        postorder.append(node.key)

    visit(root)
    return preorder, inorder, postorder

def bfs_levels(root):
    if root is None:
        return []
    queue = deque([root])
    result = []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.key)
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
```

```
                queue.append(node.right)
        result.append(level)
    return result

root = TreeNode(5, TreeNode(3, TreeNode(2), TreeNode(4)),
                TreeNode(8, None, TreeNode(9)))
pre, ino, post = dfs_orders(root)
print(pre)               # [5, 3, 2, 4, 8, 9]
print(ino)               # [2, 3, 4, 5, 8, 9]
print(post)              # [2, 4, 3, 9, 8, 5]
print(bfs_levels(root))  # [[5], [3, 8], [2, 4, 9]]
```

Root 5 has left subtree 3 with children 2 and 4, and right subtree 8 with child 9. Preorder starts at 5; postorder finishes at 5. The BFS queue holds `[5]`, then `[3, 8]`, then `[2, 4, 9]`: the snapshot `len(queue)` keeps new children out of this round. Inorder is sorted *only* for a search tree. Both walks cost $O(n)$ for n nodes; DFS saves $O(h)$ frames for height h, BFS up to $O(w)$ nodes for widest level w. Empty input produces empty lists.

Break test. Without the queue-length snapshot, BFS merges levels. Recording before the left call makes preorder, not inorder.

Source notes: #32, #33.

4.2 BSTs: a live catalog of available prices

A shop adds and removes *unique* whole-dollar prices. Besides exact search, it asks for the *ceiling* (least price at least x) or *floor* (greatest at most x); return `None` if none qualifies. A binary search tree (BST) keeps every left-subtree key smaller and right-subtree key larger, discarding one side per comparison. Equal insertions do nothing.

Insert 20 as root, 10 left, 30 right, 25 left of 30, 35 right. For `ceiling(21)`, discard 20 and its left subtree; save 30 as a candidate, improve it to 25, then stop at an empty link. For `floor(21)`, save 20; the keys to its right are too high. Search returns a node or `None`, edits return a new root, and ceiling/floor return a price or `None`.

Try it: live price changes. The complete notebook cell includes its own node class:

```
class BSTNode:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left, self.right = left, right

def bst_find(root, key):
    while root is not None:
        if key == root.key:
            return root
        root = root.left if key < root.key else root.right
    return None

def bst_insert(root, key):
    if root is None:
        return BSTNode(key)
    if key < root.key:
        root.left = bst_insert(root.left, key)
    elif key > root.key:
```

```
        root.right = bst_insert(root.right, key)
    return root

def bst_delete(root, key):
    if root is None:
        return None
    if key < root.key:
        root.left = bst_delete(root.left, key)
    elif key > root.key:
        root.right = bst_delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        successor = root.right
        while successor.left is not None:
            successor = successor.left
        root.key = successor.key
        root.right = bst_delete(root.right, successor.key)
    return root

def bst_ceiling(root, key):
    answer = None
    while root is not None:
        if root.key >= key:
            answer = root.key
            root = root.left
        else:
            root = root.right
    return answer

def bst_floor(root, key):
    answer = None
    while root is not None:
        if root.key <= key:
            answer = root.key
            root = root.right
        else:
            root = root.left
    return answer

prices = None
for amount in (20, 10, 30, 25, 35, 20):
    prices = bst_insert(prices, amount)
print(bst_find(prices, 25) is not None,
      bst_ceiling(prices, 21), bst_floor(prices, 21))  # True 25 20
prices = bst_delete(prices, 25)                # leaf
print(bst_ceiling(prices, 21))                 # 30
prices = bst_delete(prices, 30)                # one child: 35
print(bst_ceiling(prices, 21))                 # 35
prices = bst_delete(prices, 20)                # two children
print(bst_floor(prices, 21), bst_ceiling(prices, 21))  # 10 35

available = {10, 20, 25, 30, 35}
print(21 in available)  # False: membership is a different question
```

```
available.add(21)
print(21 in available)  # True
available.discard(21)
```

Deleting 25 removes a leaf; deleting 30 promotes its one child, 35. Deleting root 20 with two children copies its *successor*, the smallest key on the right (35), then removes the old 35. The remaining prices are 10 and 35. Always reassign the returned root, which may change or become `None`.

> **State.** The current node, target key, and for ceiling or floor the best qualifying candidate so far.
>
> **Invariant.** All left-subtree keys are smaller and all right-subtree keys larger; a candidate already qualifies.
>
> **Update.** Follow one side; for ceiling, save a qualifying node before trying its left side. An edit reattaches the returned subtree.

Break test. Deleting a lone root returns `None`; forgetting to remove a copied successor leaves a duplicate. With an empty tree, `bst_ceiling(None, 21) is None`.

Source notes: #35, #76, #77.

All BST operations cost $O(h)$ for height h, *not* guaranteed $O(\log n)$: sorted insertions make a chain of height n. Iterative queries use $O(1)$ extra variables; recursive edits use $O(h)$ frames. A balanced BST keeps $h = O(\log n)$. A key–value map would also copy the value when replacing a key with its successor.

Contract change: if the shop only asks exact membership, `21 in available` is `False` where ceiling previously returned 25. A Python hash `set` provides expected fast membership and updates, but no ordered neighbor; `dict` similarly stores values by exact key. For rare sorted reads, `sorted(available)` may suffice instead of a balanced tree.

4.3 Segment trees: sums of changing intervals

An arena scoreboard holds points in a fixed player lineup. `update(index, value)` corrects one score; `query(left, right)` sums the *half-open* positions `[left, right)` (right excluded). An array edits quickly but scans a range. *Prefix sums* make a range query a difference of running totals but require repairing later totals after a correction. A segment tree stores sums of nested intervals so both operations touch a path of ranges. Pad its leaves to a power of two; extra leaves contain zero:

```
class RangeSumTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.size = 1
        while self.size < self.n:
            self.size *= 2
        self.tree = [0] * (2 * self.size)
        self.tree[self.size:self.size + self.n] = nums
        for i in range(self.size - 1, 0, -1):
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def update(self, index, value):
        if not 0 <= index < self.n:
            raise IndexError("index out of range")
        i = self.size + index
        self.tree[i] = value
        while i > 1:
```

```
            i //= 2
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def query(self, left, right):
        if not 0 <= left <= right <= self.n:
            raise IndexError("invalid half-open range")
        left += self.size
        right += self.size
        total = 0
        while left < right:
            if left % 2:
                total += self.tree[left]
                left += 1
            if right % 2:
                right -= 1
                total += self.tree[right]
            left //= 2
            right //= 2
        return total

points = [5, 8, 6, 3, 2, 7, 2, 6]
prefix = [0]
for value in points:
    prefix.append(prefix[-1] + value)
print(prefix[7] - prefix[2])  # 20 before edits
totals = RangeSumTree(points)
print(totals.query(2, 7))     # 20
totals.update(4, 9)
points[4] = 9
print(totals.query(2, 7), prefix[7] - prefix[2])  # 27 20 (stale)
```

Every node stores its interval's exact sum; index i's children are $2i$ and $2i + 1$. Query borders enclose unaccounted leaves: take a whole interval at an odd left border or just before an odd, exclusive right border, then move both borders up. For `[2, 7)`, the disjoint pieces `[2, 4)`, `[4, 6)`, `[6, 7)` sum to 20. **Contract change:** without edits, two prefixes answer that query in $O(1)$. Assigning 9 at index 4 makes the true answer 27, while stale prefixes still say 20; a segment update repairs only the ancestors of that leaf. Rebuilding prefixes costs up to $O(n)$ per correction. An empty query returns 0; `RangeSumTree([]).query(0, 0)` is valid, but updating it is not. To include right endpoint `R`, call `query(L, R + 1)`.

Break test. `query(4, 4)` must return zero, not the value at index 4; mixing inclusive and half-open bounds loses the last item.

Source notes: #107.

Build and storage cost $O(n)$; point updates and range queries take $O(\log n)$ for nonempty arrays. A minimum tree would change both the combine rule and the identity for an empty overlap. Range-wide edits need deferred updates ("lazy propagation"), not this point-update code.

4.4 Quadtrees: stop when a square is uniform

A parking map marks each space 0 (free) or 1 (occupied). Build a quadtree from a square, power-of-two grid, then count occupied spaces in a viewport: half-open rows `[top, bottom)` and columns `[left,`

`right)`. Each node owns a square and its count of ones; the stored tree branches into four quadrants only when mixed. Uniform squares become leaves; side 1 always stops.

```
class Quad:
    def __init__(self, row, col, size, ones, children=()):
        self.row, self.col, self.size = row, col, size
        self.ones, self.children = ones, children

def build_quad(grid):
    n = len(grid)
    if n == 0 or any(len(row) != n for row in grid):
        raise ValueError("expected a nonempty square")
    side = 1
    while side < n:
        side *= 2
    if side != n:
        raise ValueError("side must be a power of two")
    if any(v not in (0, 1) for row in grid for v in row):
        raise ValueError("expected zero-one cells")

    def build(row, col, size):
        if size == 1:
            return Quad(row, col, 1, grid[row][col])
        half = size // 2
        children = (
            build(row, col, half),
            build(row, col + half, half),
            build(row + half, col, half),
            build(row + half, col + half, half),
        )
        ones = sum(child.ones for child in children)
        if ones == 0 or ones == size * size:
            return Quad(row, col, size, ones)
        return Quad(row, col, size, ones, children)

    return build(0, 0, n)

def quad_count(root, top, left, bottom, right):
    if top > bottom or left > right:
        raise ValueError("inverted query box")

    def visit(node):
        r, c, s = node.row, node.col, node.size
        if bottom <= r or r + s <= top or right <= c or c + s <= left:
            return 0
        if top <= r and r + s <= bottom and left <= c and c + s <= right:
            return node.ones
        if not node.children:
            height = min(bottom, r + s) - max(top, r)
            width = min(right, c + s) - max(left, c)
            return height * width if node.ones else 0
        return sum(visit(child) for child in node.children)

    return visit(root)

image = [[1, 1, 0, 0], [1, 1, 0, 0],
```

```
         [0, 0, 0, 0], [0, 0, 0, 0]]
quad = build_quad(image)
print(quad_count(quad, 0, 0, 2, 2))  # 4
print(quad_count(quad, 1, 1, 3, 3))  # 1
```

In the sample only the upper-left two-by-two region is full: the first box returns 4, while shifting to `(1, 1)` leaves one occupied space. A query rejects disjoint squares, uses a cached count on full cover, or descends; a partially covered uniform leaf contributes its overlap area if full. Building scans n^2 cells, but a uniform image stores one node, an isolated occupied cell $O(\log n)$ nodes, and a checkerboard $O(n^2)$. A query costs $O(v)$ for v visited nodes; a full-root query is $O(1)$, but a mixed region may require many visits. A dense, fixed image needing only rectangle sums might favor two-dimensional prefix sums.

For arbitrary *points*, instead keep a small bucket per leaf, split when full, and give children disjoint half-open squares. Stop at a minimum cell size so identical points do not split forever. Sparse point trees need not allocate empty children; this image variant instead compresses uniform areas.

Break test. Returning a full leaf's cached count for a one-cell overlap gives 4 instead of 1.

Source notes: #148.

4.5 Tries: a path is a string prefix

A grocery app inserts product names and asks either "is this exact name stocked?" or "does any name begin with what the customer typed?" A *trie* shares paths for common prefixes; a node's `end` flag marks a complete name, not just a path. This case-sensitive API returns Booleans, not suggestion lists:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.end = True

    def walk(self, text):
        node = self.root
        for char in text:
            node = node.children.get(char)
            if node is None:
                return None
        return node

    def contains(self, word):
        node = self.walk(word)
        return node is not None and node.end
```

```
    def has_prefix(self, prefix):
        return self.walk(prefix) is not None

catalog = Trie()
catalog.insert("apple pie")
print(catalog.contains("apple"), catalog.has_prefix("apple"))  # False True
catalog.insert("apple")
print(catalog.contains("apple"))  # True
```

After inserting `"apple pie"`, the `"apple"` path exists but has no end flag: exact lookup is false, prefix lookup true. Inserting `"apple"` sets that flag; repeating it has no further effect. By convention the empty prefix exists even in an empty trie; the empty *word* needs an explicit insert. Each operation follows L characters in expected $O(L)$ time; at most $O(T)$ nodes store T inserted characters, with shared prefixes saving space. A `set` answers exact membership but needs another index for prefixes; listing completions costs the subtree walk and output size.

Break test. Treating path existence as an end flag wrongly calls `"apple"` stocked when only `"apple pie"` was inserted.

Source notes: #43, #97.

Choose the invariant from the question: level order needs a queue; ordered neighbors need BST bounds; mutable ranges, sparse squares, and prefixes each need their own node summary. Every listing above runs independently in a notebook.

CHAPTER 5

Hashing: remember equality without rescanning

As values arrive one at a time, repeatedly scanning everything seen so far to answer "Have I seen this?" wastes work. A *hash* is an integer computed from a key; a hash table uses it to narrow the search, then checks equality. Different keys choosing the same place *collide*. A set answers membership (yes/no); a dict also stores a value, such as an index. Neither answers "next larger key" without an ordered index.

Python's set and dict have *expected* $O(1)$ lookup/insertion for *hashable* keys (stable hash consistent with equality). Expected assumes well-spread hashes, not a worst-case guarantee: long keys cost time to hash, and many collisions can make lookup linear. Dict insertion order is not sorted key order.

5.1 Choose the remembered fact

Problem. Given integer list `values` and a target, return `(i, j)` with $i < j$ and `values[i] + values[j] == target`; choose the smallest j, then i. Return `None` if absent and leave the list unchanged. This input/output promise is the *contract*. A dict maps each value to its earliest index; `setdefault` stores that index only if the key is missing.

```
def first_pair_sum(values, target):
    seen = {}                       # value -> earliest earlier index
    for j, value in enumerate(values):
        i = seen.get(target - value)
        if i is not None:
            return i, j
        seen.setdefault(value, j)
    return None
```

Query before insertion so an item cannot pair with itself: `[4, 4]` gives `(0, 1)`, but `[4]` gives `None`. For n integers, time is expected $O(n)$ and space $O(n)$.

> **State.** Earliest index per processed value.
>
> **Invariant.** Before j, the map covers only earlier indices; the first hit minimizes j.
>
> **Update.** Query the complement, then save the earliest index.

Break test. `if i` misses a match at index 0; inserting before querying falsely pairs the lone `[4]` with itself.

Source notes: #1, #5, #72.

Requirement change: yes/no only. The same integer inputs now need a Boolean, not indices; remember values in a set rather than indices in a dict. This complete function leaves the input untouched:

```python
def has_pair_sum(values, target):
    seen = set()
    for value in values:
        if target - value in seen:
            return True
        seen.add(value)
    return False
```

On `[4, 4]`, the answer changes from `(0, 1)` to `True`; on `[4]`, from `None` to `False`. Both query before adding the current value.

5.2 Hashable keys and canonical forms

Equal keys need equal, unchanging hashes; unequal keys *may* collide, so equality still decides lookup. Integers, strings, and tuples of hashable items work; lists and tuples containing lists do not. Mutating a custom key's hash/equality fields can lose its entry. Python also treats `True == 1`: tag with `(type(x), x)` if your domain distinguishes them.

Problem. Group lowercase words `["eat", "tea", "bat"]` into `[["eat", "tea"], ["bat"]]`, retaining group and word arrival order. Anagrams contain the same letters. Their *canonical key* is one shared signature: a tuple of 26 letter counts, not an unhashable list. This function accepts only `a`–`z` (otherwise `ValueError`) and does not mutate its input. `ord` maps letters to codes; subtract `ord("a")` for slots 0–25.

```python
def group_anagrams(words):
    groups = {}
    for word in words:
        counts = [0] * 26
        for char in word:
            slot = ord(char) - ord("a")
            if not 0 <= slot < 26:
                raise ValueError("expected lowercase a-z")
            counts[slot] += 1
        signature = tuple(counts)
        groups.setdefault(signature, []).append(word)
    return list(groups.values())
```

Counting a length-L word takes $O(L+26)$ work. Different signatures may hash alike; the dict still checks equality.

> **State.** Counts for this word and a dict of groups.
>
> **Invariant.** Processed words belong to the group for their frozen count tuple.
>
> **Update.** Count letters, make a tuple, append the original word.

Break test. Raw-word keys split `"eat"` from `"tea"`; list-of-counts keys raise `TypeError`.

Source notes: #5, #51.

Requirement change: ignore case. The first function rejects `"Eat"`. Keep each original word, but change the dict key to sorted `casefold()` characters (Unicode-aware case conversion; $O(L \log L)$ per word):

```
def group_anagrams_folded(words):
    groups = {}
    for word in words:
        signature = tuple(sorted(word.casefold()))
        groups.setdefault(signature, []).append(word)
    return list(groups.values())
```

`group_anagrams_folded(["Eat", "tea", "bat"])` returns `[["Eat", "tea"], ["bat"]]` without lowercasing the output.

5.3 Collisions, equality, and growth

Problem. A catalog stores IDs 0, 4, 8 with labels `"zero"`, `"four"`, `"eight"`; looking up 4 must return `"four"` even though the IDs initially collide. In *separate chaining*, each of B buckets holds a list of key–value pairs; hash(*key*) mod B chooses one bucket (modulo means remainder). Compare keys inside it, never return its first value blindly. Here `put` inserts or replaces and returns `None`; `get` returns a value or supplied default, and `len(table)` counts distinct keys. Keys need stable hashes and reflexive equality; values may be `None`.

```
class ChainedMap:
    def __init__(self):
        self.buckets = [[] for _ in range(4)]
        self.size = 0

    def get(self, key, default=None):
        bucket = self.buckets[hash(key) % len(self.buckets)]
        for stored_key, value in bucket:
            if stored_key == key:
                return value
        return default

    def put(self, key, value):
        bucket = self.buckets[hash(key) % len(self.buckets)]
        for i, (stored_key, _) in enumerate(bucket):
            if stored_key == key:
                bucket[i] = (stored_key, value)
                return
        bucket.append((key, value))
        self.size += 1
        if 4 * self.size > 3 * len(self.buckets):
            old = self.buckets
            self.buckets = [[] for _ in range(2 * len(old))]
            for old_bucket in old:
                for stored_key, stored_value in old_bucket:
                    slot = hash(stored_key) % len(self.buckets)
                    self.buckets[slot].append((stored_key, stored_value))

    def __len__(self):
        return self.size
```

IDs 0, 4, 8 share bucket 0 in the initial four-bucket table. ID 12 makes the *load factor* N/B (keys per bucket) exceed 3/4: double capacity and *rehash* every pair; ID 4 now belongs in bucket 4. One $O(N)$ resize spreads across many cheap insertions (amortized cost). Uniformly spread hashes keep buckets

short on average; all-colliding keys still cost $O(N)$ to find. For stored `None`, use a unique `object()` default to distinguish it from a missing key.

State. Buckets, key count N, capacity B.

Invariant. Each key occurs once in its bucket under the current capacity.

Update. Check equality before appending; redistribute every pair when growing.

Break test. Copy old buckets without rehashing on growth: ID 4 becomes unreachable in its new bucket.

Requirement change: edit a label. After `put(4, "FOUR")`, `get(4)` returns `"FOUR"` without changing `len(table)` from 3. The equal-key branch replaces rather than appends.

CPython's dict and set instead use *open addressing*: colliding entries probe other array slots. Adding deletion needs a *tombstone* ("deleted, keep looking"). If A occupied slot 2 and colliding B probed to slot 3, clearing A to "never used" would stop lookup before B. Neither scheme tolerates a key whose hash/equality changes while stored.

Source notes: #51, #101.

5.4 Consistent hashing: map to the next point

Problem. Cache machines A and B must each own some of `["u4", "u5", "u6", "u9"]`; return one owner per key in that order. Modulo a changing machine count remaps many keys. *Consistent hashing* places keys and machines on a circle and chooses the first machine clockwise, wrapping past its end. A *virtual node* is one of several positions per machine: they spread ownership, without guaranteeing perfect balance.

The code accepts unique nonempty string node names and string keys; `owner` raises `LookupError` for an empty ring. SHA-256 supplies a stable *digest* (hash of text), unlike Python's process-randomized string `hash()`. `insort` maintains sorted points; `bisect_left` finds the first one at or after a key. Tuple order resolves digest ties.

```
from bisect import bisect_left, insort
from hashlib import sha256

def ring_hash(text):
    return int.from_bytes(sha256(text.encode("utf-8")).digest(), "big")

class HashRing:
    def __init__(self, replicas=4):
        if type(replicas) is not int or replicas < 1:
            raise ValueError("replicas must be positive")
        self.replicas = replicas
        self.points = []
        self.nodes = set()

    def add(self, node):
        if not isinstance(node, str) or not node:
            raise ValueError("node must be a nonempty string")
        if node in self.nodes:
            raise ValueError("duplicate node")
        points = [(ring_hash(f"node:{node}:{i}"), node, i)
                  for i in range(self.replicas)]
```

```
        for point in points:
            insort(self.points, point)
        self.nodes.add(node)

    def owner(self, key):
        if not self.points:
            raise LookupError("empty ring")
        if not isinstance(key, str):
            raise TypeError("key must be a string")
        position = ring_hash(f"key:{key}")
        i = bisect_left(self.points, (position, "", -1))
        return self.points[i % len(self.points)][1]
```

With two virtual nodes each, owners are `["A", "A", "A", "B"]`. **Requirement change: add C.** Insert its points; owners become `["C", "C", "A", "B"]`, so `"u6"` and `"u9"` stay put. For a visible wrap, picture a toy ring 0–99 with A at 10, 60 and B at 30, 80. Key 95 wraps to A at 10. Adding C at 25, 70 claims only arcs $(10, 25]$ and $(60, 70]$: key positions 20 and 65 move from B to C; 75 stays with B. Those arcs exclude 10 and 60 but include 25 and 70. The program uses SHA-256 coordinates instead; Try it checks its real outputs.

With P virtual points, lookup is $O(\log P)$; inserting one into this list shifts up to $O(P)$ entries. The ring decides ownership; a distributed cache must still move keys on changed arcs (*rebalancing*) and agree on ring membership.

State. Sorted virtual points and key position.

Invariant. Owner is the next clockwise point, wrapping if necessary.

Update. Insert new points and migrate only their newly claimed arcs.

Break test. A key past the last point (95 here) still has an owner: returning "not found" instead of wrapping loses it.

Source notes: #139.

5.5 Bloom filters: a qualified answer in few bits

Problem. Catalog `"cat"` must answer `True` for `"cat"` and `False` for `"dog"`. A set is exact but can be too large for memory; remote negative lookups then cost time. A *Bloom filter* stores m bits and probes k hash-derived positions for each key. A zero proves absence; all ones mean only "maybe present". This version accepts strings, positive integer bit/probe counts, and insertion/membership only. `bytearray` stores mutable eight-bit bytes.

```python
from hashlib import sha256

class Bloom:
    def __init__(self, bit_count, probes):
        if type(bit_count) is not int or type(probes) is not int:
            raise ValueError("bit_count and probes must be integers")
        if bit_count < 1 or probes < 1:
            raise ValueError("bit_count and probes must be positive")
        self.width = bit_count
        self.probes = probes
        self.bits = bytearray((bit_count + 7) // 8)

    def _positions(self, item):
        if not isinstance(item, str):
            raise TypeError("item must be a string")
        raw = item.encode("utf-8")
        for i in range(self.probes):
            digest = sha256(str(i).encode("ascii") + b":" + raw).digest()
            yield int.from_bytes(digest, "big") % self.width

    def add(self, item):
        for p in self._positions(item):
            self.bits[p // 8] |= 1 << (p % 8)

    def might_contain(self, item):
        return all(self.bits[p // 8] & (1 << (p % 8))
                   for p in self._positions(item))
```

For a toy trace, A sets bits 1, 3 and B sets 3, 5. Absent C probing 1, 5 looks present: a *false positive*. An inserted key cannot return absent (a *false negative*) while its bits stay set and hash functions/parameters stay fixed. As more bits fill, false positives rise; the filter cannot store the original keys.

Requirement change: skip costly misses. Keep exact storage but add a filter before it: a definite no skips a read, a maybe requires exact verification. In Try it, 64 bits reject `"dog"`; one bit saturates and says maybe, but the exact set still returns `False`. Never use maybe alone for authorization or uniqueness. Naively clearing A's bit 3 would also lose still-present B; ordinary deletion needs an exact set or another design.

State. Bits and stable probe positions.

Invariant. Every inserted key's positions remain set; any zero proves absence.

Update. Set bits on insert; verify maybes against exact storage.

Break test. Clear shared bit 3 to delete A: B now appears absent.

Source notes: #146.

5.6 Try it: from a question to a call

Run the complete function and class definitions above in order; none is just a transition sketch. The ring and filter listings include their standard-library imports. This call-site example repeats those imports so it can be pasted after the definitions:

```
from bisect import bisect_left, insort
from hashlib import sha256

print(first_pair_sum([4, 4], 8))       # (0, 1)
print(has_pair_sum([4, 4], 8))         # True
print(group_anagrams(["eat", "tea", "bat"]))
# [['eat', 'tea'], ['bat']]
print(group_anagrams_folded(["Eat", "tea", "bat"]))
# [['Eat', 'tea'], ['bat']]
table = ChainedMap()
for key, label in ((0, "zero"), (4, "four"), (8, "eight")):
    table.put(key, label)
print(table.get(4), len(table))       # four 3
table.put(4, "FOUR")
print(table.get(4), len(table))       # FOUR 3
table.put(12, "twelve")
print(table.get(12), len(table.buckets))  # twelve 8
ring = HashRing(replicas=2)
ring.add("A")
ring.add("B")
customers = ["u4", "u5", "u6", "u9"]
print([ring.owner(key) for key in customers])
# ['A', 'A', 'A', 'B']
ring.add("C")
print([ring.owner(key) for key in customers])
# ['C', 'C', 'A', 'B']
exact = {"cat"}
screen = Bloom(64, 3)
screen.add("cat")
print("cat" in exact, screen.might_contain("cat"))  # True True
print(screen.might_contain("dog"))  # False: skip exact lookup
filter_bits = Bloom(1, 2)
filter_bits.add("cat")
print("dog" in exact, filter_bits.might_contain("dog"))  # False True
print("dog" in exact if filter_bits.might_contain("dog") else False)
# False: the exact lookup resolves the false positive
```

CHAPTER 6

Sorting: choose the guarantee you need

"Sort" does not specify a *key* (price or whole record), whether equal-key items retain arrival order, or whether the caller's list may change. A *stable* sort preserves that tie order. Let n be the number of items; assume key extraction/comparison take constant time. $O(1)$ means fixed work, $O(\log n)$ grows with the number of halvings, and $O(n^2)$ with item pairs. Extra space is scratch memory beyond input/output; "mutates" means changing the input list.

Method	Stable?	Mutates?	Extra space	Worst time
Insertion	yes	yes	$O(1)$	$O(n^2)$
Merge below	yes	no	$O(n)$	$O(n \log n)$
Quick below	no	yes	$O(\log n)$ stack	$O(n^2)$
Heap below	no	yes	$O(1)$	$O(n \log n)$
`list.sort()`	yes	yes	up to $O(n)$	$O(n \log n)$

Only insertion and heap sort here use strict $O(1)$ extra space; quicksort uses a stack, and even mutating `list.sort()` may allocate scratch. Insertion takes $O(n)$ on ordered input; quicksort is $O(n \log n)$ on balanced splits but can be quadratic. Python's stable `list.sort()` exploits already ordered runs; `sorted(values)` makes a new list instead.

6.1 Insertion sort: grow a settled prefix

Problem. Sort requests `[(3, "A"), (1, "B"), (3, "C"), (2, "D")]` by priority, keeping A before C, into the *same* list: `[(1, "B"), (2, "D"), (3, "A"), (3, "C")]`. For small/nearly sorted lists, hold each new item and shift greater ones right. The key function must be *pure* (same key without side effects); keys need a consistent *total order*.

```
def insertion_sort(values, key=lambda item: item):
    for i in range(1, len(values)):
        item = values[i]
        item_key = key(item)
        j = i
        while j > 0 and key(values[j - 1]) > item_key:
            values[j] = values[j - 1]
            j -= 1
        values[j] = item
    return values
```

Trace 3A, 1B, 3C, 2D: 1B shifts 3A; 3C does not shift its equal 3A; 2D shifts both threes. The strict `>` preserves ties. With I *inversions* (out-of-order pairs), time is $O(n + I)$, worst-case $O(n^2)$.

State. Sorted prefix, held item, hole at j.

Invariant. Before step i, the prefix is stably ordered.

Update. Shift only greater keys right; fill the hole with the held item.

Break test. Using `>=` on equal keys 2A, 2B moves 2B ahead of 2A: sorted numbers, wrong arrival order.

6.2 Merge sort: settle two ordered halves

Problem. Sort `[(1, "A"), (3, "B"), (1, "C"), (2, "D")]` by first field into a *new* list, keeping the input unchanged: `[(1, "A"), (1, "C"), (2, "D"), (3, "B")]`. Merge sorted halves to preserve ties; the function accepts a list or tuple and pure key supporting `<=`. Slices and merge output cost $O(n)$ peak scratch and $O(n \log n)$ worst-case time.

```
def merge_sort(values, key=lambda item: item):
    if len(values) <= 1:
        return list(values)
    middle = len(values) // 2
    left = merge_sort(values[:middle], key)
    right = merge_sort(values[middle:], key)
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

For halves 1A, 3B and 1C, 2D, each head is its half's smallest remaining item. On the tie at 1, take left 1A first: left-half items originally preceded right-half items. Recursion keeps ties ordered within each half.

State. Two sorted halves, head indices, merged prefix.

Invariant. The prefix holds the smallest chosen items in stable order.

Update. Take the smaller head; on ties take left, then advance that head.

Break test. Use `<` rather than `<=`: tied 1C on the right now precedes 1A on the left.

Source notes: #17.

Requirement change: preserve the input. For `[3, 1, 2]`, insertion changes the caller's list to `[1, 2, 3]`; merge returns that result while the original remains `[3, 1, 2]`. The state changes from an in-place prefix to new merged lists (both calls appear in Try it).

6.3 Quicksort: make a pivot final

Problem. Sort `[3, 1, 3, 2]` in the same list to `[1, 2, 3, 3]`; tie order does not matter. Choose the last item as *pivot*; *partition* the interval into smaller items, pivot, and items at least as large. The pivot is now final; sort only the sides. An explicit stack of pending intervals avoids Python's recursion limit. Keys must have a consistent order under `<`; the function returns the same list.

```
def quick_sort(values):
    pending = [(0, len(values) - 1)]
    while pending:
        lo, hi = pending.pop()
        while lo < hi:
            pivot = values[hi]
            cut = lo
            for scan in range(lo, hi):
                if values[scan] < pivot:
                    values[cut], values[scan] = values[scan], values[cut]
                    cut += 1
            values[cut], values[hi] = values[hi], values[cut]
            if cut - lo < hi - cut:
                pending.append((cut + 1, hi))
                hi = cut - 1
            else:
                pending.append((lo, cut - 1))
                lo = cut + 1
    return values
```

During scanning, items before `cut` are smaller than the pivot; those from `cut` to the scan position are not. For `[3, 1, 3, 2]`, partition puts 1 before 2, then the two threes. Swaps may reverse equal records. The code saves the larger side and processes the smaller: pending stack size stays $O(\log n)$. Time does not: sorted or all-equal input repeatedly scans $n-1, n-2, \ldots$ items, $O(n^2)$. Random pivots help typical inputs; three-way partitioning helps many equal keys.

> **State.** Active interval, pivot, `cut`, scan index.
>
> **Invariant.** Within the interval, entries before `cut` are smaller; entries from `cut` to `scan - 1` are not. The pivot stays at `hi`.
>
> **Update.** Swap smaller items left; place the pivot at `cut`, then process the sides.

Break test. Claiming $O(n \log n)$ worst-case time for last-pivot quicksort fails on an already sorted list.

Source notes: #18.

Requirement change: worst-case guarantee. If long sorted inputs are allowed, use `heap_sort` for $O(n \log n)$ worst-case time. Both return `[1, 2, 3, 4]` for that input, but the heap repairs its maximum instead of repeatedly scanning partitions.

6.4 Heap sort: put each maximum at the end

Problem. Sort `[4, 1, 3, 2]` in the original list to `[1, 2, 3, 4]` using $O(1)$ scratch and $O(n \log n)$ worst-case time. A *heap* is a partly ordered tree in an array: each parent outranks its children. A *max-heap* puts the greatest item at the root. Move it into the fixed *suffix* (end of the list), then *sift down*: swap the new root with its larger child until the shortened prefix is a heap again.

```
def heap_sort(values):
    def sift_down(root, end):
        while 2 * root + 1 < end:
            child = 2 * root + 1
            if child + 1 < end and values[child] < values[child + 1]:
                child += 1
            if values[root] >= values[child]:
                return
            values[root], values[child] = values[child], values[root]
            root = child

    n = len(values)
    for root in range(n // 2 - 1, -1, -1):
        sift_down(root, n)
    for end in range(n - 1, 0, -1):
        values[0], values[end] = values[end], values[0]
        sift_down(0, end)
    return values
```

Bottom-up heap building costs $O(n)$: most nodes lie near leaves. For `[4, 1, 3, 2]`, it makes `[4, 2, 3, 1]`; extract 4, repair to heap `[3, 2, 1]` with suffix `[4]`. That suffix grows ascending while dominating the unsorted prefix.

Python's `heapq` instead maintains a *min*-heap. Its `heapify` does not sort; repeated `heappop` consumes the heap and needs $O(n)$ space to retain all results. A *priority queue* supports updates between minimum queries. For stable ties, pair each priority with an increasing arrival number; heap sort itself is not stable.

State. Max-heap prefix and sorted suffix.

Invariant. Root is greatest unfixed item; suffix items are greater.

Update. Swap root to suffix, shorten heap, sift down.

Break test. `heapq.heapify` on `[3, 1, 2]` can leave `[1, 3, 2]`: a heap, not a sorted list.

Source notes: #59, #109.

Requirement change: next minimum after updates. Instead of the full sorted list, use a `heapq` min-heap. Pop 1, push 0, then pop 0; a static sorted answer would offer 2 next. Try it imports `heapq` and shows both outputs.

6.5 When can sorting be linear?

For arbitrary distinct keys, *comparison sorting* cannot beat $n \log n$ comparisons in the worst case. Counting pays for a small integer range, bucket sorting assumes a distribution, and radix sorting pays per digit. Their "linear" costs require those conditions, not just a different algorithm name.

Counting sort: pay for the integer universe

Problem. Sort tagged scores `[(2, "A"), (-1, "B"), (2, "C"), (0, "D")]` stably into `[(-1, "B"), (0, "D"), (2, "A"), (2, "C")]` without changing the input. Keys are plain integers in `-1` through `2`. A count array spans the inclusive range of width $R = maximum - minimum + 1$; subtract the minimum to index even negative keys. Accumulated counts give each key's *end position*, just past its output block. The function rejects keys outside its stated bounds.

```
def counting_sort(items, minimum, maximum, key=lambda item: item):
    if (type(minimum) is not int or type(maximum) is not int
            or minimum > maximum):
        raise ValueError("expected ordered integer bounds")
    counts = [0] * (maximum - minimum + 1)
    for item in items:
        value = key(item)
        if type(value) is not int or not minimum <= value <= maximum:
            raise ValueError("key outside integer range")
        counts[value - minimum] += 1
    for i in range(1, len(counts)):
        counts[i] += counts[i - 1]
    result = [None] * len(items)
    for item in reversed(items):
        slot = key(item) - minimum
        counts[slot] -= 1
        result[counts[slot]] = item
    return result
```

Keys `-1, 0, 1, 2` give frequencies `[1, 1, 0, 2]` and cumulative ends `[1, 2, 2, 4]`. Place items backward: 2C takes the last key-2 slot, then 2A the preceding one, preserving their arrival order. Time and extra space are $O(n + R)$.

> **State.** Offset counts, cumulative ends, output list.
>
> **Invariant.** Each end points just beyond its key's block.
>
> **Update.** Decrement an end before placing an item, walking input backward.

Break test. Direct indexing with `-1` hits Python's last slot; forward placement from end positions reverses ties.

Source notes: #108.

Requirement change: unbounded scores. If a score may be `1000000000`, a count array wastes roughly a billion slots. Use `merge_sort`: input `[-2, 1000000000, 1]` returns `[-2, 1, 1000000000]` without that array.

Bucket sort: pay for a distribution assumption

Problem. Sort readings `[0.42, 0.05, 0.38, 0.91, 0.44]` into `[0.05, 0.38, 0.42, 0.44, 0.91]`, preserving input. Values are finite, roughly uniform and independent on $[0, 1)$ (zero included, one excluded). Divide this range into n *buckets* (lists for subintervals); sort each, then concatenate. Out-of-range input raises `ValueError`. The `min` in code guards against floating-point rounding at the final bucket.

```python
def bucket_sort_unit(values):
    if not all(0 <= x < 1 for x in values):
        raise ValueError("values must be in [0, 1)")
    n = len(values)
    if n == 0:
        return []
    buckets = [[] for _ in range(n)]
    for x in values:
        slot = min(n - 1, int(n * x))
        buckets[slot].append(x)
    for bucket in buckets:
        bucket.sort()
    return [x for bucket in buckets for x in bucket]
```

These readings land in buckets 2, 0, 1, 4, 2. Every earlier bucket precedes every later one; only local order remains. Uniform independent input keeps buckets small on average: expected $O(n)$ work. If all readings cluster together, Python's inner comparison sort costs $O(n \log n)$ worst-case.

State. Ordered buckets of readings.

Invariant. Earlier buckets contain smaller values.

Update. Sort each bucket, concatenate in order.

Break test. A cluster near zero fills one bucket; the distribution assumption, not correctness, fails.

Source notes: #61.

Requirement change: negatives allowed. The bucket slot formula rejects `[-0.1, 0.42]`; without a new distribution, `sorted` returns `[-0.1, 0.42]` using comparisons instead.

Radix sort: preserve lower-digit decisions

Problem. Sort nonnegative integer IDs `[31, 12, 11, 22]` into `[11, 12, 22, 31]` without changing input. The *radix* is ten here. Sort ones, tens, and so on using the *stable* `counting_sort` above: a tens tie must preserve the ones order. This function rejects negatives and nonintegers.

```python
def radix_sort_nonnegative(values):
    if any(type(x) is not int or x < 0 for x in values):
        raise ValueError("nonnegative integers only")
    result = list(values)
    if not result:
        return result
    place = 1
    maximum = max(result)
    while place <= maximum:
        result = counting_sort(result, 0, 9,
                               key=lambda x: (x // place) % 10)
        place *= 10
    return result
```

The ones pass gives `[31, 11, 12, 22]`; tens yields `[11, 12, 22, 31]`. After d passes, the lowest d digits are ordered; stable ties retain earlier decisions. With constant-cost digit extraction, time is $O(d(n + 10))$, space $O(n + 10)$. Huge Python integers make digit arithmetic more costly.

State. Copied list, decimal place, stable digit pass.

Invariant. Processed low digits remain ordered.

Update. Sort by the next digit, then multiply the place by ten.

Break test. Unstable tens sorting can undo the ones-pass order `[11, 12]` of `[12, 11]`.

Requirement change: signed IDs. Radix rejects `[-2, 12]`: digit passes have no minus-sign policy. `merge_sort` returns `[-2, 12]` instead, or a separate signed-digit policy is needed.

Without those special domain/distribution promises, prefer Python's stable `sorted` (new list) or `list.sort()` (mutating) to a custom sorting implementation.

6.6 Try it: changing the contract changes the tool

Run the complete function definitions above in chapter order; none is just a transition sketch. Only `radix_sort_nonnegative` depends on an earlier listing, `counting_sort`. No sorting function here needs an import; the changing-priority example imports the standard-library `heapq` module explicitly.

```
import heapq

values = [3, 1, 2]
print(insertion_sort(values), values)   # [1, 2, 3] [1, 2, 3]
requests = [(3, "A"), (1, "B"), (3, "C"), (2, "D")]
print(insertion_sort(requests, key=lambda row: row[0]))
# [(1, 'B'), (2, 'D'), (3, 'A'), (3, 'C')]
original = [3, 1, 2]
print(merge_sort(original), original)   # [1, 2, 3] [3, 1, 2]
print(merge_sort([(1, "A"), (3, "B"), (1, "C"), (2, "D")],
                 key=lambda row: row[0]))
# [(1, 'A'), (1, 'C'), (2, 'D'), (3, 'B')]
print(quick_sort([3, 1, 3, 2]))         # [1, 2, 3, 3]
ordered = [1, 2, 3, 4]
print(quick_sort(ordered[:]), heap_sort(ordered[:]))
# [1, 2, 3, 4] [1, 2, 3, 4]

data = [4, 1, 3, 2]
print(heap_sort(data[:]))               # [1, 2, 3, 4]
queue = data[:]
heapq.heapify(queue)
print(heapq.heappop(queue))             # 1
heapq.heappush(queue, 0)
print(heapq.heappop(queue))             # 0

scores = [(2, "A"), (-1, "B"), (2, "C"), (0, "D")]
print(counting_sort(scores, -1, 2, key=lambda row: row[0]))
# [(-1, 'B'), (0, 'D'), (2, 'A'), (2, 'C')]
print(merge_sort([-2, 1000000000, 1]))  # [-2, 1, 1000000000]
readings = [0.42, 0.05, 0.38, 0.91, 0.44]
print(bucket_sort_unit(readings))     # [0.05, 0.38, 0.42, 0.44, 0.91]
print(sorted([-0.1, 0.42]))           # [-0.1, 0.42]
print(radix_sort_nonnegative([31, 12, 11, 22]))
# [11, 12, 22, 31]
print(merge_sort([-2, 12]))           # [-2, 12]
```

CHAPTER 7

Graphs: decide what a route must accomplish

Imagine stations (nodes) joined by roads (edges): that picture is a *graph*. "Find a route" could mean the fewest roads, the least travel time, the cheapest network connecting *all* stations, or a walk that uses *every* road. Those are different questions, even when the picture is identical. First decide what an edge means and what the answer must contain. Then choose the state that makes the next move visible.

7.1 Nodes, edges, and neighbors

Number nodes from 0 to `n - 1`. An adjacency list stores each node's neighbors: nodes reachable by one edge. A *directed* edge is one-way, so an edge from 0 to 1 appears only in `graph[0]`; an *undirected* road can be used both ways and appears in both lists, although it is still *one* road. If a road has a cost, store a `(neighbor, weight)` pair. These three representations are not interchangeable: choose one before writing a traversal.

```
def adjacency(n, edges, directed=False):
    graph = [[] for _ in range(n)]
    for u, v in edges:              # endpoints are in 0..n-1
        graph[u].append(v)
        if not directed:
            graph[v].append(u)
    return graph

def grid_neighbors(row, col, rows, cols, diagonal=False):
    moves = ((1, 0), (-1, 0), (0, 1), (0, -1))
    if diagonal:
        moves += ((1, 1), (1, -1), (-1, 1), (-1, -1))
    for dr, dc in moves:
        nr, nc = row + dr, col + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc
```

The first problem is small: given a one-way road 0 to 1, return each station's outgoing neighbors and decide whether 1 can reach 0. The directed adjacency is `[[1], []]`, so it cannot. **Change one requirement:** make that road two-way. Append the reverse neighbor too; adjacency becomes `[[1], [0]]`, and 1 can now reach 0. An isolated node still owns an empty list. A grid usually needs no stored adjacency: `grid_neighbors` generates four *geometrically* valid cells. The caller subsequently checks for a wall and for prior visits. Moving diagonally is a different neighbor contract. Building adjacency takes $O(V + E)$ time and space for V nodes and E edges: the work grows with nodes plus roads. Generating one grid cell's neighbors takes $O(1)$ extra space.

Break test. If the only road is directed `0 -> 1`, a search starting at 1 must not reach 0. Adding both adjacency entries has changed the problem, not just optimized its representation.

7.2 Depth-first versus breadth-first search

Depth-first search (DFS) uses a stack: `pop()` takes the most recently added node, following a branch. Breadth-first search (BFS) uses a queue: `popleft()` takes the oldest, exploring all one-hop arrivals before two-hop arrivals. Pending nodes form the *frontier*. Both searches need a rule for claiming a node reached by several routes. For ordinary reachability, claim it *when adding it*, not when it eventually comes out. The functions below assume valid node IDs and unweighted edges.

```
def reachable_dfs(graph, start):
    seen = {start}
    stack = [start]
    while stack:
        u = stack.pop()
        for v in graph[u]:
            if v not in seen:
                seen.add(v)
                stack.append(v)
    return seen

from collections import deque

def bfs_distances(graph, start):
    distance = {start: 0}          # also the discovered set
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in distance:
                distance[v] = distance[u] + 1
                queue.append(v)
    return distance
```

Suppose the input is the undirected diamond with edges 0–1, 0–2, 1–3, 2–3. First ask which nodes 0 can reach: DFS returns all four. **Change the output to the fewest roads from 0 to each station:** replace the stack and `seen` with a FIFO queue and a distance map; BFS says 3 is two hops away. BFS starts with queue `[0]`. Popping 0 claims 1 and 2 at distance 1, leaving `[1, 2]`. Popping 1 claims 3 at distance 2, leaving `[2, 3]`; popping 2 does *not* enqueue 3 again. The returned distances are `0:0`, `1:1`, `2:1`, `3:2`. DFS may pop 2 before 1: its `seen` describes reachability, *not* minimum distance or a promised visiting order. To record DFS nodes on entry (*preorder*) or after their neighbors (*postorder*), keep a frame recording which neighbor to try next, instead of pushing all neighbors at once (#37).

An *invariant* is a promise about the state before and after each step:

> **State.** Pending nodes and their first-claim marks; BFS additionally stores the hop distance at first claim.
>
> **Invariant.** Every pending node has one claim. BFS's queue holds arrivals in nondecreasing hop distance.
>
> **Update.** Pop one, examine its neighbors, and claim each new one *before* adding it to the frontier.

Break test. Mark 3 only when it leaves the queue. Both 1 and 2 can enqueue it in the diamond. DFS might repeat work; BFS can no longer rely on one queue entry per node.

Each node is claimed once and each adjacency entry examined once: $O(V + E)$ time, $O(V)$ extra space. BFS finds fewest *edges* only when edges have equal cost. A Python recursive DFS can hit the recursion

limit on a long chain; this stack does not. #38 and #39 use these same first-claim decisions in grids and paths.

7.3 Components and islands: restart outside the loop

Now take roads 0–1 and 2–3 with isolated station 4. The desired output is the three groups of mutually reachable stations. In an undirected graph, each such whole group is a *connected component*. Start DFS whenever the outer scan finds an unclaimed node; keep the *outer* `seen` across starts. The function calls `reachable_dfs` from above.

```
def components(graph):
    seen, groups = set(), []
    for start in range(len(graph)):
        if start not in seen:
            group = reachable_dfs(graph, start)
            seen.update(group)
            groups.append(group)
    return groups                    # sets; internal order is irrelevant
```

The output groups are 0 with 1, 2 with 3, and 4 alone. The helper's `seen` belongs to one walk; the outer one prevents another start in a finished group. **Change the roads to directed:** now this loop does *not* find strongly connected components (groups whose nodes can all reach one another). With just 0 to 1 it groups 0 and 1, but the correct strong components are separate: 1 cannot get back to 0. This task needs mutual reachability, not a claim on first forward visit.

For a rectangular grid of `"1"` land and `"0"` water, return the size of every island when only four-way contact counts. An island is a component whose grid edges are generated only when needed. The function uses `grid_neighbors` above.

```
def island_sizes(grid, diagonal=False):
    if not grid:
        return []
    rows, cols = len(grid), len(grid[0])
    seen, sizes = set(), []
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            stack = [(row, col)]
            seen.add((row, col))    # claim the new island's seed
            size = 0
            while stack:
                r, c = stack.pop()
                size += 1
                for nr, nc in grid_neighbors(r, c, rows, cols, diagonal):
                    if grid[nr][nc] == "1" and (nr, nc) not in seen:
                        seen.add((nr, nc))
                        stack.append((nr, nc))
            sizes.append(size)
    return sizes
```

Rows `"1100"`, `"0101"`, `"0011"` yield sizes `[3, 3]`. The first island is the upper-left L; the second occupies the lower-right corner. Cells touching only diagonally do not join. **Change one rule:** on rows `"10"`, `"01"`, the four-way answer is `[1, 1]`; if diagonal contact also counts, the answer is `[2]`. The `diagonal`

option adds four neighbor offsets; the traversal and its land-before-claim guard stay the same. The outer scan and walks together cost $O(RC)$ time and at most $O(RC)$ extra space for R rows and C columns.

7.4 Dijkstra: first *popped* minimum, not first offer

Suppose roads have travel times. A queue cannot compare a one-hop road of cost 9 with a two-hop route costing 3. From a starting node called `source`, Dijkstra keeps the best cost *so far* for each node in `dist`. A min-heap is a queue that removes the smallest-cost proposal first. Lowering a recorded cost is called *relaxing* an edge. All weights must be nonnegative. A better offer may arrive before an old one is popped, so old heap entries are skipped rather than removed in place.

```
from heapq import heappop, heappush

def dijkstra(weighted, source):
    dist = {source: 0}
    heap = [(0, source)]
    while heap:
        cost, u = heappop(heap)
        if cost != dist[u]:         # a better offer superseded this one
            continue
        for v, weight in weighted[u]:
            if weight < 0:
                raise ValueError("Dijkstra needs nonnegative weights")
            candidate = cost + weight
            if candidate < dist.get(v, float("inf")):
                dist[v] = candidate
                heappush(heap, (candidate, v))
    return dist                     # absent nodes are unreachable
```

Consider directed roads 0 to 1, 0 to 2, 2 to 1, and 1 to 3. **Contract switch:** If the answer is *fewest roads* from 0 to 1, BFS returns 1 for the direct edge. If the same roads instead have costs 9, 2, 1, and 4 respectively, the cheapest *travel cost* to 1 is 3, via 0, 2, 1. The queue and `distance[u] + 1` must become a heap and `cost + weight`. After popping 0, the heap offers `(2, 2)` and `(9, 1)`; popping 2 improves node 1 to 3. Popping 1 at 3 then gives node 3 cost 7. The old `(9, 1)` is stale. Node 1's shortest cost becomes final at its *valid pop*, not its first push.

> **State.** Best-known costs and a heap of possibly outdated proposals.
>
> **Invariant.** A matching minimum-cost pop settles that node's shortest distance when all weights are nonnegative.
>
> **Update.** Skip proposals that differ from the stored cost; relax an edge only if it strictly improves its destination, then push the new proposal.

Each successful relaxation inserts at most one heap entry per edge. The lazy heap uses $O(V + E)$ space and $O((V + E) \log(V + E))$ time. Checking weights in the loop detects a negative edge if it is reached; the *input contract* must rule out negative weights everywhere. #42 develops this relaxation.

7.5 A minimum spanning tree is not a shortest route

Given undirected roads 0–1 of cost 9, 0–2 of cost 10, 1–2 of cost 2, and 2–3 of cost 1, return the cheapest set of roads connecting *all* four stations, including its total cost. A minimum spanning tree (MST) is

that connected, cycle-free set of $V - 1$ roads in an undirected graph (a cycle is a route back to its start). Prim grows one tree: its heap holds candidate edges crossing from the tree to the outside. Do *not* mark a neighbor when pushing it; a cheaper crossing edge may be offered later. Unlike Dijkstra, the heap priority is the edge's own weight, not the cost of a path from the start.

```
from heapq import heappop, heappush

def prim_mst(graph):                # weighted, undirected adjacency
    n = len(graph)
    if n == 0:
        return 0, []
    used, chosen, total = set(), [], 0
    heap = [(0, -1, 0)]             # weight, parent, node
    while heap and len(used) < n:
        weight, parent, u = heappop(heap)
        if u in used:
            continue
        used.add(u)                 # accept the crossing edge now
        if parent != -1:
            chosen.append((parent, u, weight))
            total += weight
        for v, edge_weight in graph[u]:
            if v not in used:
                heappush(heap, (edge_weight, u, v))
    return (total, chosen) if len(used) == n else None
```

Kruskal starts somewhere else: sort the *physical* edges once and merge components when the next edge connects two different ones. Union-find records each node's group leader (its *root*) and merges two groups by linking their roots. Give Kruskal each undirected edge *once*, not once from each adjacency list. For each edge in sorted order, `root` follows parent links to its group leader; skip the edge if both leaders match. Otherwise, link the smaller group under the larger and keep the edge. Stop after $V - 1$ edges; fewer means disconnected input.

```
def kruskal_mst(n, edges):      # each undirected road appears once
    parent, size = list(range(n)), [1] * n

    def root(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    chosen, total = [], 0
    for u, v, weight in sorted(edges, key=lambda e: e[2]):
        a, b = root(u), root(v)
        if a == b:
            continue            # would create a cycle
        if size[a] < size[b]:
            a, b = b, a
        parent[b] = a
        size[a] += size[b]
        chosen.append((u, v, weight))
        total += weight
        if len(chosen) == n - 1:
            break
    return (total, chosen) if n == 0 or len(chosen) == n - 1 else None
```

Prim starting at 0 must first take 0–1 at 9. It can then take 1–2 at 2 and 2–3 at 1. Kruskal takes 2–3 at 1, 1–2 at 2, and 0–1 at 9: a forest may be disconnected *during* its construction. Both finish at total 12. Yet the cheapest *route* from 0 to 3 uses 0–2 and 2–3 for cost 11; the tree's 0-to-3 route costs 12. They return `None` if the graph is disconnected, and both permit negative edge weights. Kruskal's invariant is that accepted edges link different groups; Prim's is that its next accepted edge leaves the growing tree.

Lazy Prim takes $O(V + E \log(E+1))$ time and $O(V+E)$ space. Kruskal takes $O(V + E \log(E+1))$ time including sorting and $O(V+E)$ space including the sorted copy; union-find operations are nearly constant amortized time, meaning averaged over a sequence of operations (#118, #119). Neither algorithm defines an MST of a directed graph without a different problem specification.

7.6 Topological order and two kinds of cycle check

A *topological order* places every prerequisite before the task that depends on it. A directed edge points *from* prerequisite *to* task. Given edges 0 to 2, 1 to 2, and 2 to 3, return any order with every prerequisite first, or `None` if none exists. The indegree of a task is its number of incoming edges; Kahn's algorithm tracks how many still block it. Only zero-indegree tasks enter its queue. Removing a task discharges its outgoing edges, perhaps freeing another task.

```
from collections import deque

def topological_order(graph):      # directed graph on 0..n-1
    indegree = [0] * len(graph)
    for neighbors in graph:
        for v in neighbors:
            indegree[v] += 1
    queue = deque(i for i, d in enumerate(indegree) if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return order if len(order) == len(graph) else None
```

If 0 and 1 both lead to 2, and 2 leads to 3, start with `[0, 1]`. Removing 0 leaves indegree 1 at 2; removing 1 reduces it to zero, admitting 2, then 3. One valid order is `[0, 1, 2, 3]`. If 3 also leads to 1, nodes 1, 2, and 3 never become free: returning `None` reports a directed cycle. The order need not be unique.

If the output changes from a course order to just "cycle?", add edge 3 to 1 to the example: Kahn returns `None`, while a color-based DFS returns `True`. A white node (0) is new, gray (1) is on the current call path, and black (2) has exited. An edge to gray closes a loop; an edge to black only reaches finished work. For a *simple undirected* graph (no duplicate roads or loops) instead, the edge back to the parent (the node we came from) is expected; another visited neighbor reveals a cycle.

```
def directed_cycle(graph):
    color = [0] * len(graph)

    def visit(u):
        color[u] = 1
        for v in graph[u]:
            if color[v] == 1:
                return True
            if color[v] == 0 and visit(v):
                return True
        color[u] = 2
        return False

    return any(color[u] == 0 and visit(u)
               for u in range(len(graph)))
```

In a directed diamond, both arms may reach the same black node without forming a cycle; a single permanent `seen` flag cannot distinguish that from a back edge. The undirected parent-*vertex* rule skips `v == parent` before testing whether `v` was seen; any other seen neighbor closes a cycle. Restart from every unseen node. This simple-graph rule assumes no parallel edges (two distinct roads with the same ends): the second road to the parent forms a cycle. A *multigraph* permits such roads; there, skip only the incoming *edge ID*, as in the next section, not every road to its parent. Kahn and both kinds of cycle checks take $O(V + E)$ time. The recursive directed check uses $O(V)$ stack space and may exceed Python's recursion limit; Kahn's queue avoids that risk (#40, #81).

7.7 Eulerian trails: spend each edge exactly once

Given undirected roads 0–1, 1–2, 1–3, 3–4, 4–1, return a sequence of vertices traversing each road exactly once, or `None` if impossible. An Eulerian trail may revisit a *node*, but uses each *edge* exactly once. A node's degree counts its incident edge ends (a loop back to itself contributes two). In an undirected graph, all vertices that touch edges must be in one connected component. Zero odd-degree vertices gives a circuit that returns to its start; exactly two gives an open trail between those odd vertices. Any other odd count makes a trail impossible. Isolated vertices do not prevent a trail over the actual edges.

Here `edges` contains one pair per physical undirected road. Two identical pairs are two different roads, so each gets its own ID: its position in the input. A loop contributes two adjacency entries but is marked *used* only once. Hierholzer's method walks over unused edges, then records vertices on the way back to splice any leftover cycles into the finished trail.

```
def euler_trail(n, edges):
    if not edges:
        return [0] if n else []
    adj = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        adj[u].append((v, edge_id))
        adj[v].append((u, edge_id))
    odd = [u for u in range(n) if len(adj[u]) % 2]
    if len(odd) not in (0, 2):
        return None
    start = odd[0] if odd else next(u for u in range(n) if adj[u])

    seen, pending = {start}, [start]
    while pending:                  # check non-isolated connectivity
```

```
        u = pending.pop()
        for v, _ in adj[u]:
            if v not in seen:
                seen.add(v)
                pending.append(v)
    if any(adj[u] and u not in seen for u in range(n)):
        return None

    used = [False] * len(edges)
    cursor = [0] * n
    stack, trail = [start], []
    while stack:
        u = stack[-1]
        while cursor[u] < len(adj[u]) and used[adj[u][cursor[u]][1]]:
            cursor[u] += 1
        if cursor[u] == len(adj[u]):
            trail.append(stack.pop())     # no unused exit: backtrack
        else:
            v, edge_id = adj[u][cursor[u]]
            cursor[u] += 1
            used[edge_id] = True
            stack.append(v)
    trail.reverse()
    return trail if len(trail) == len(edges) + 1 else None
```

Only 0 and 2 have odd degree. The first walk goes 0, 1, 2 and gets stuck at 2, but three unused edges form a cycle through 1. Backtracking there takes 1, 3, 4, 1; reversing the finished stack output splices the cycle in and yields `[0, 1, 3, 4, 1, 2]`. A greedy "stop when stuck" would lose those roads. **Contract switch:** If two 0–1 roads are distinct physical roads, the output is `[0, 1, 0]`; `used` must track *edge IDs*. If the contract instead counts each unordered endpoint pair once, deduplicate first and the output is `[0, 1]`. Marking just `(u, v)` without changing the contract loses an edge.

> **State.** A stack of the current walk, unused edge IDs, and a reverse list of vertices whose outgoing edges are exhausted.
>
> **Invariant.** Each physical edge is used at most once; finished vertices can be spliced into the final walk in reverse finish order.
>
> **Update.** Take any unused incident edge, or backtrack and record a vertex when it has none left.

Break test. Two separate triangles have even degree at every vertex, but cannot form one trail. Parity without connectivity is not sufficient.

For a *directed* trail, parity is replaced by indegree/outdegree: either they are equal at every used vertex (circuit), or one start has one extra outgoing edge and one finish has one extra incoming edge. The non-isolated vertices must still be connected when directions are ignored; the actual walk must follow directions. The implementation above is *undirected*. Its connectivity scan, edge marks, and per-node cursors each do $O(V + E)$ total work and use $O(V + E)$ space.

Try it: ask different questions of the same roads

Run the complete function definitions above in order (including their imports). The component and island functions call earlier helpers; none of those listings is a transition sketch. This block supplies imports, inputs, calls, and expected outputs:

```
from collections import deque
from heapq import heappop, heappush

one_way = adjacency(2, [(0, 1)], directed=True)
print(one_way, sorted(reachable_dfs(one_way, 1)))   # [[1], []] [1]
print(sorted(reachable_dfs(adjacency(2, [(0, 1)]), 1)))  # [0, 1]
diamond = adjacency(4, [(0, 1), (0, 2), (1, 3), (2, 3)])
print(bfs_distances(diamond, 0)[3])                 # 2 hops
separate = adjacency(5, [(0, 1), (2, 3)])
print([sorted(g) for g in components(separate)])  # [[0, 1], [2, 3], [4]]
print(island_sizes(["1100", "0101", "0011"]))       # [3, 3]
print(island_sizes(["10", "01"]))                   # [1, 1]
print(island_sizes(["10", "01"], diagonal=True))    # [2]

roads = [(0, 1), (0, 2), (2, 1), (1, 3)]
graph = adjacency(4, roads, directed=True)
weighted = [[(1, 9), (2, 2)], [(3, 4)], [(1, 1)], []]
print(bfs_distances(graph, 0)[1], dijkstra(weighted, 0)[1])  # 1 3

links = [(0, 1, 9), (0, 2, 10), (1, 2, 2), (2, 3, 1)]
network = [[] for _ in range(4)]
for u, v, w in links:
    network[u].append((v, w))
    network[v].append((u, w))
print(prim_mst(network)[0], kruskal_mst(4, links)[0])  # 12 12
print(dijkstra(network, 0)[3])                      # 11, not 12

courses = [[2], [2], [3], []]
print(topological_order(courses))                   # [0, 1, 2, 3]
courses[3].append(1)
print(topological_order(courses), directed_cycle(courses))  # None True
trail_edges = [(0, 1), (1, 2), (1, 3), (3, 4), (4, 1)]
print(euler_trail(5, trail_edges))                    # [0, 1, 3, 4, 1, 2]
physical = [(0, 1), (1, 0)]                           # two roads
print(euler_trail(2, physical))                       # [0, 1, 0]
unique = sorted({tuple(sorted(edge)) for edge in physical})
print(euler_trail(2, unique))                         # [0, 1]
```

The question chooses the state: counting hops, minimizing travel cost, connecting a network, respecting prerequisites, and using every road exactly once need different promises about the edges.

CHAPTER 8

Bits and strings: decide what one position means

A position in an integer's binary representation can mean "item included." A position in a Python string means a Unicode *code point* (one numbered character), not necessarily one displayed letter or one encoded byte. Both topics reward the same first question: *what is the unit, and what must be remembered about it?* A *mask* is an integer whose one bits mark selected positions. It records choices; a frequency map records how many copies of each item exist; a prefix (the start of a string) or a stack records unfinished string work.

8.1 Bits as switches, with an explicit width

Python's `0b` prefix writes a number in binary. Number bit positions from the right, starting at zero: in `0b1010`, bits 1 and 3 are set. Shifting a one left by `k` places it at position `k`. AND (`&`) keeps a one only if *both* inputs have one; OR (`|`) needs *either*; XOR (`^`) needs *exactly one*. Thus AND tests or clears, OR sets, and XOR toggles one switch. The complement `~` flips bits; Python needs an explicit width when flipping all of them. Here is a defined task: given nonnegative `x = 0b1010`, width 4, and bit index 3, report whether it is set and the values after setting, clearing, and toggling *that bit*. Return all four answers without mutating `x`.

```
def edit_bit(x, k, width):
    if (width < 0 or x < 0 or k < 0 or k >= width
            or x >= (1 << width)):
        raise ValueError("x must fit; k must be in range")
    bit = 1 << k
    is_set = bool(x & bit)
    with_bit = x | bit
    without_bit = x & ~bit
    flipped = x ^ bit
    return is_set, with_bit, without_bit, flipped
```

The result is `(True, 0b1010, 0b0010, 0b0010)`. "Set" is not "toggle": setting an already set bit leaves it alone. Python integers do not overflow at a machine word boundary. **Change one requirement:** if the caller wants *all* four bits flipped rather than just bit 3, use `(~x) & ((1 << width) - 1)`. That returns `0b0101` (5), not `0b0010` (2). Plain `~x` is negative because Python treats leading sign bits as unbounded; mask left shifts too if fixed-width wraparound is required.

Break test. On `0b1010`, replace the set operation with XOR at position 3. It turns the existing one *off*; OR would preserve it. A negative `x` also does not have a finite leading-bit representation unless a width is specified.

8.2 Counting set bits and enumerating chosen items

The *population count* (popcount) is the number of one bits. For nonnegative integers, Python's `x.bit_count()` computes it directly. The loop below shows why another approach works: subtracting one changes the lowest one to zero and fills lower positions with ones; AND with the old value removes exactly that lowest one.

```
def popcount(x):
    if x < 0:
        raise ValueError("expected a nonnegative integer")
    count = 0
    while x:
        x &= x - 1
        count += 1
    return count
```

The value `0b10110` becomes `0b10100`, then `0b10000`, then zero: three iterations, three set bits. Zero starts and ends with count zero. This loop performs one iteration per one bit. Because Python integers can grow beyond a machine word, each bit operation also has a cost that grows with the integer's width. Prefer the built-in for ordinary Python code (#136).

Suppose the input is `["A", "B", "C", "D"]` and mask `0b1010`; return selected items `["B", "D"]`. Bit `i` selects item `i`. A *submask* turns off some selected bits without turning on any new bit. If the output changes to *all allowed masks*, including the empty choice, enumerate submasks instead of constructing one list of items.

```
def selected_items(items, mask):
    if mask < 0 or mask >= (1 << len(items)):
        raise ValueError("mask must fit the item count")
    return [item for i, item in enumerate(items)
            if mask & (1 << i)]

def submasks(mask):
    if mask < 0:
        raise ValueError("expected a nonnegative mask")
    sub = mask
    while True:
        yield sub
        if sub == 0:
            return
        sub = (sub - 1) & mask
```

For mask `0b1010`, `submasks` yields `0b1010`, `0b1000`, `0b0010`, and `0b0000`. Subtract, then AND with the original mask: positions not allowed by the original never appear. If the original has `k` set bits, there are exactly 2^k results, including zero. Checking for zero *before* subtracting is essential: otherwise Python's `-1 & mask` becomes `mask`, silently restarting the loop. To enumerate subsets of all `n` items instead, iterate masks from zero through `(1 << n) - 1`; materializing each chosen list costs $O(n2^n)$ total time. `yield` produces one result at a time, so `submasks` need not store all 2^k outputs; it keeps just the original and current masks (#137).

8.3 An exact bitmap or a set?

Suppose IDs are integers from 0 to `U - 1` and only membership matters. One Python integer can represent their *exact* membership: test with AND, add with OR, remove with AND and a complemented one-bit mask. Two bitmaps combine with OR for union or AND for intersection. The highest possible ID determines the worst-case storage width: setting even one bit near `U - 1` uses roughly $O(U)$ bits. A Python `set` instead stores the keys, needs no agreed upper bound, and gives expected constant-time membership for keys such as integers and strings; it costs much more space per stored small integer. Use the bitmap for a small, bounded universe or many set operations; use a set for sparse, large, unknown,

or noninteger keys. **Contract switch:** For IDs `[2, 2]`, "was 2 ever seen?" needs one bit or set entry and returns `True`; "how many times?" needs a count map and returns 2. This is an *exact* bitmap, not a Bloom filter (a compact test that may report "present" for an absent item).

Break test. With IDs 2 and one billion, a bitset stretches to a billion positions while a set stores two keys. A filter with false matches cannot replace an exact membership test.

8.4 Text units, anagrams, and character counts

Python `str` iteration visits Unicode code points. UTF-8 encodes them into bytes, possibly several bytes per point. **Contract switch:** Counting code points in `"café"` returns 4; counting its UTF-8 bytes returns 5 after `encode("utf-8")`. Changing the unit changes both the representation and the output. An accented letter may be a base letter plus a combining mark (an extra point modifying the previous one). If equivalent spellings must compare equal, normalize them first (convert them to the same standard Unicode form). The routines below count the original code points exactly: case and accents matter.

Anagrams are rearrangements with the same number of copies of each character. **Contract switch:** If the question is "same distinct letters?", `"aab"` and `"abb"` return `True` using sets. If it is "anagrams?", they return `False`: the counts differ. Replace the set with a count map. Each character of the second word must pay down one positive need.

```
from collections import Counter

def are_anagrams(a, b):
    if len(a) != len(b):
        return False
    need = Counter(a)
    for ch in b:
        if need[ch] == 0:          # absent or already exhausted
            return False
        need[ch] -= 1
    return True
```

For `"aab"` versus `"aba"`, the initial needs are two a's and one b. Consuming a, b, a reaches zero for each. Against `"abb"`, the second b finds no remaining b and fails *before* the decrement. Matching lengths and spending every required character means no leftover need is possible. This takes expected $O(n)$ time and $O(d)$ space for n code points and d distinct code points. To group many anagrams, an immutable signature such as sorted `Counter` items can be used as a dictionary key (#5).

An *invariant* is a fact kept true as the loop consumes each character:

> **State.** A map from code point to copies still needed.
>
> **Invariant.** All consumed characters have paid for distinct copies in the first string; no count has gone below zero.
>
> **Update.** Reject an absent or exhausted character before subtracting its count.

8.5 Knuth–Morris–Pratt (KMP): reuse a matched start

Given text `"ababab"` and pattern `"abab"`, return *all* match starts: `[0, 2]`. A mismatch after a long partial match should not force the text index backward. For the piece `pattern[:i+1]`, `pi[i]` records the length of its longest *proper* prefix (a beginning shorter than that piece) that is also its suffix (an

ending of that piece). That is the part already known to match after an earlier candidate fails. Follow shorter prefixes repeatedly until the new character can extend one, or no prefix remains.

```
def prefix_lengths(pattern):
    pi = [0] * len(pattern)
    matched = 0
    for i in range(1, len(pattern)):
        while matched and pattern[i] != pattern[matched]:
            matched = pi[matched - 1]
        if pattern[i] == pattern[matched]:
            matched += 1
        pi[i] = matched
    return pi

def kmp_starts(text, pattern):
    if not pattern:
        return []                   # this API excludes empty matches
    pi = prefix_lengths(pattern)
    starts, matched = [], 0
    for i, ch in enumerate(text):
        while matched and ch != pattern[matched]:
            matched = pi[matched - 1]
        if ch == pattern[matched]:
            matched += 1
        if matched == len(pattern):
            starts.append(i - matched + 1)
            matched = pi[matched - 1]  # retain the overlap
    return starts
```

For this pattern, the table is `[0, 0, 1, 2]`. The first match starts at 0; afterward two characters still form a matching prefix. The last `"ab"` completes an overlapping match at 2. Resetting `matched` to zero would miss it. **Change the output to just the first hit:** `text.find(pattern)` returns 0, and we may stop there instead of retaining overlap and collecting starts. For all hits, a `while`, not a single `if`, is needed on mismatch: after `"ababa"` matched against `"abababaca"`, the next `"x"` drives lengths 5 to 3 to 1 to 0 without rereading any text. KMP takes $O(n + m)$ time and $O(m)$ space for text length n and nonempty pattern length m; Python's built-in search may be more convenient when this guarantee is not needed (#86).

> **State.** The prefix table and length of the pattern prefix matching the suffix of text already read.
>
> **Invariant.** Everything before the current text position has been accounted for, including overlapping hits.
>
> **Update.** On mismatch fall back through prefix lengths; after a complete hit, record its start and retain the longest proper suffix-prefix for the next character.

Break test. Search for `"abab"` in `"abababab"`. Stopping or resetting to zero after the first hit loses the hit at index 2.

8.6 Tries share starts; stacks close obligations

If many words share their beginning, a trie gives each shared prefix one path of child nodes. After inserting `"art"` and `"artist"`, ask whether `"arti"` is a prefix and whether it is a complete word. Walk one child map per character, then check the end marker: the outputs are `(True, False)`. This small complete version uses `None` as an end marker, distinct from character keys:

```
def trie_add(root, word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node[None] = True

def trie_status(root, word):
    node = root
    for ch in word:
        if ch not in node:
            return False, False
        node = node[ch]
    return True, None in node     # prefix exists, whole word exists
```

Change the requirement to whole-word lookup only: a `set` of the two words is simpler. It reports `False` for `"arti"` because that prefix is not a complete word; it cannot answer the original prefix question. The fuller trie in chapter 4 shares the same word-end rule. A trie lookup costs $O(L)$ expected time for query length L, but child maps have overhead (#43).

Now input the bracket string `"([)]"`; return whether each closer matches the most recent unclosed opener. The answer is `False`. A stack retains those openers, and its last element must match the next closer. The function accepts only the three bracket pairs and rejects other characters explicitly.

```
def balanced(brackets):
    partner = {")": "(", "]": "[", "}": "{"}
    openers = set(partner.values())
    stack = []
    for ch in brackets:
        if ch in openers:
            stack.append(ch)
        elif ch in partner:
            if not stack or stack[-1] != partner[ch]:
                return False
            stack.pop()
        else:
            raise ValueError("expected only brackets")
    return not stack
```

For `"([])"`, pushing two openers and popping each on its matching closer leaves the stack empty, so it returns `True`. In `"([)]"`, the first `")"` expects `"("` but sees `"["`. **Change the requirement to counting openers and closers regardless of order:** counts now agree, giving `True`; that weaker question needs counts rather than a nesting stack. The stack approach takes $O(n)$ time and $O(n)$ worst-case space (#22, #25).

> **State.** Opening brackets not yet closed, in nesting order.
>
> **Invariant.** The stack is precisely the unclosed prefix; only its top can match the next closer.
>
> **Update.** Push an opener; on a closer, check the top *before* popping; finally require an empty stack.

8.7 Kadane's scan: a best ending and a best ever

Input `[4, -6, 3, 2, -1, 4]`; return the largest sum of a *nonempty contiguous* span and its bounds: `(8, (2, 6))`. Let `current` be the largest sum of a span ending at the current position, and `best` the largest

span anywhere in the prefix read so far. An old ending sum below zero hurts a new start; otherwise extending it is at least as good as starting fresh. A separate best preserves an answer after the current ending sum later falls. Return the bounds as a half-open interval `(lo, hi)`: include `lo`, exclude `hi`.

```
def max_subarray(values):
    if not values:
        raise ValueError("expected a nonempty sequence")
    current = best = values[0]
    begin = best_lo = 0
    best_hi = 1                  # half-open answer interval
    for i in range(1, len(values)):
        if current < 0:
            current, begin = values[i], i
        else:
            current += values[i]
        if current > best:
            best = current
            best_lo, best_hi = begin, i + 1
    return best, (best_lo, best_hi)
```

The ending sums are 4, `-2`, 3 (restart), 5, 4, 8. The answer is `(8, (2, 6))`: values from index 2 up to but not including 6. On `[-5, -2]`, the answer is `-2`, not zero; initializing the best to zero would wrongly allow an empty span. The strict comparison retains the earlier winner on a tie. This is $O(n)$ time and $O(1)$ auxiliary space when the values are already available (#28).

Change the input to a scored string: assign a numeric score per code point, run the same scan on those scores, and use its bounds to return the best *contiguous substring*. For `"abca"` with a scored at 2, b at `-5`, and c at 3, the scores are `[2, -5, 3, 2]` and the best output is score 5 and substring `"ca"`. If scoring is performed by building a list first, that list adds $O(n)$ space. A request for the longest substring with at most two distinct characters instead needs a moving range with character counts, not a forced numeric score.

> **State.** Best nonempty span ending here, its starting position, and the best completed span with bounds.
>
> **Invariant.** The ending span must include the current item; the saved answer is optimal over all items processed.
>
> **Update.** Restart after a negative ending sum, otherwise extend; then compare the new ending candidate with the saved best.

Try it: change the unit or the question

Run this chapter's complete function definitions (including their imports) in one Python session, then this block. It includes its own import, inputs, calls, and expected outputs:

```
from collections import Counter

x, width = 0b1010, 4
print(edit_bit(x, 3, width))                  # (True, 10, 2, 2)
print((~x) & ((1 << width) - 1))              # 5: flip all four
print(popcount(0b10110))                      # 3
print(selected_items(["A", "B", "C", "D"], x)) # ['B', 'D']
print(list(submasks(x)))                      # [10, 8, 2, 0]
seen = 0
for item in [2, 2]:
    seen |= 1 << item
print(bool(seen & (1 << 2)), 2 in set([2, 2]))  # True True
print(Counter([2, 2])[2])                       # 2 copies

word = "café"
print(len(word), len(word.encode("utf-8")))  # 4 5
a, b = "aab", "abb"
print(set(a) == set(b), are_anagrams(a, b))  # True False
print(are_anagrams("aab", "aba"))            # True
text, pattern = "ababab", "abab"
print(text.find(pattern), kmp_starts(text, pattern))  # 0 [0, 2]

root = {}
for stored in ("art", "artist"):
    trie_add(root, stored)
print(trie_status(root, "arti"))             # (True, False)
print("arti" in {"art", "artist"})           # False: whole word
s = "([)]"
same_counts = s.count("(") == s.count(")") and s.count("[") == s.count("]")
print(balanced(s), same_counts)              # False True
print(balanced("([])"))                      # True

print(max_subarray([4, -6, 3, 2, -1, 4]))   # (8, (2, 6))
scores = {"a": 2, "b": -5, "c": 3}
text = "abca"
best, (lo, hi) = max_subarray([scores[ch] for ch in text])
print(best, text[lo:hi])                     # 5 ca
```

CHAPTER 9

Dynamic programming and common problem moves

Dynamic programming is useful when different routes reach the same smaller question. Instead of solving that question again, give it a name and save its answer. But a table is not the starting point. A *state* is that named question with its parameters; a *base case* answers a smallest question without recursion; a *transition* combines already solved states. First write the question each entry answers, then its base case, its dependencies, and the order that makes them ready. The return line should name a finished entry.

Some problems below are *not* DP. They are here because their implementation also depends on an honest state sentence: which answer is settled, and which new item can change it? Choosing between a DP table, a greedy boundary, and two pointers is itself a coding skill. Run the following standard-library import first; the remaining function definitions and the calls at the end of the chapter form one runnable Python file in reading order.

```
from collections import Counter
```

9.1 One-dimensional DP: choose the last item or skip it

Suppose adjacent houses cannot both be selected, and the goal is the largest sum. For the processed prefix, keep two answers: `skip`, the best sum if its final house was skipped, and `take`, the best sum if its final house was taken. Before reading the next house, those values describe *old* prefixes. The new skipped case can come from either old case; the new taken case can come only from an old skipped case.

```
def max_nonadjacent(values):
    skip = take = 0
    for value in values:
        new_skip = max(skip, take)
        new_take = skip + value
        skip, take = new_skip, new_take
    return max(skip, take)
```

For `[2, 7, 9, 3, 1]`, the pairs `(skip, take)` after each item are `(0, 2)`, `(2, 7)`, `(7, 11)`, `(11, 10)`, `(11, 12)`. The answer is 12. The empty input gives zero; negative values are harmless when choosing no house is allowed. The two variables are a compressed one-dimensional table: each new entry needs only the previous step's two possibilities.

> **State.** Two best sums for the processed prefix, distinguished by its final choice.
>
> **Invariant.** Neither state includes two adjacent chosen items, and each is optimal under its final-choice constraint.
>
> **Update.** Compute *both* new values from the old pair, then assign them together.

Break test. Update `skip` first, then compute `take = skip + value` using that new `skip`. On `[2, 7]`, this can combine both houses and report 9 instead of 7.

Source notes: #28, #90.

Small change: the houses form a circle. Now the first and last houses are adjacent, so the previous answer for `[2, 3, 2]` would incorrectly allow the two end houses, total 4. Either omit the first house or omit the last; run the original linear helper on both cases. The single-house case needs its own choice.

```
def max_nonadjacent_circle(values):
    if not values:
        return 0
    if len(values) == 1:
        return max(0, values[0])
    return max(max_nonadjacent(values[1:]),
               max_nonadjacent(values[:-1]))
```

The circular answer for `[2, 3, 2]` is 3. Slicing copies the two candidate subarrays; an index-range helper avoids those copies when extra space matters.

9.2 Two dimensions: give each prefix pair a meaning

For longest common *subsequence*, characters must stay in their original relative order but need not be adjacent. Let `dp[i][j]` be the length of the best subsequence shared by the *first* `i` characters of one string and the *first* `j` of another. A zero-length prefix shares nothing, so row zero and column zero start at zero. At `(i, j)`, equal last characters can extend the diagonal answer. Otherwise one of those last characters is not used, so take the better answer from above or left.

```
def lcs_length(a, b):
    rows, cols = len(a), len(b)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for i in range(1, rows + 1):
        for j in range(1, cols + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[rows][cols]
```

For `"abcde"` and `"ace"`, the last entry is 3. Notice the offset: `dp[1][1]` speaks about `a[0]` and `b[0]`. Creating the rows with a comprehension gives each one an independent list. The shorthand `[[0] * m] * n` aliases every row; changing one entry would change several "different" subproblems. This table takes $O(|a||b|)$ time and space. If only the length is needed, two rows suffice; reconstructing the subsequence needs additional information about the choices.

Small change: require adjacent characters. Now the request is a longest common *substring*, not a subsequence. For `"abxc"` and `"abc"`, the common subsequence `"abc"` has length 3, but the longest contiguous common substring `"ab"` has length 2. On a mismatch the matching run at `dp[i][j]` becomes zero instead of inheriting an answer from above or left. Record the largest cell *anywhere*, not merely the bottom-right cell. One changed word in the output contract changes both the recurrence and where the answer is collected.

A grid can use the same dependency picture. If moves are only right and down, ways to reach an open cell are the ways from above plus the ways from the left. Obstacles contribute zero. One row of memory works because `ways[col]` still holds the old row's "above" value until it is replaced, while `ways[col - 1]` already holds the current row's "left" value.

```python
def grid_paths(grid):
    if not grid or not grid[0]:
        return 0
    cols = len(grid[0])              # rectangular 0/1 grid
    ways = [0] * cols                # 1 means blocked
    for row, cells in enumerate(grid):
        for col, blocked in enumerate(cells):
            if blocked:
                ways[col] = 0
            elif row == 0 and col == 0:
                ways[col] = 1
            elif col > 0:
                ways[col] += ways[col - 1]
    return ways[-1]
```

The open three-by-three grid with its center blocked has two such routes. A blocked start leaves zero ways in every reachable cell; there is no invented base-case path to add later. This compressed table works because every dependency is above or left, not somewhere in a future row.

Break test. At an obstacle, leave the old `ways[col]` unchanged. A route from the row above then passes through a blocked cell as if no obstacle existed.

9.3 Zero-one knapsack: one item, one decision

Each item has a positive weight and a value; choose it at most once, with total weight at most a capacity. Let the i-th row of a conceptual table answer "best value using only the first i items with room c." Skip item i, or take its value and use the *previous* row at $c - \text{weight}_i$. The previous row is the proof that this item cannot be chosen a second time.

Only the preceding row is needed, so use one list and scan capacities *downward*. The smaller entry read during an update still describes earlier items. Each entry means "capacity *at most* this much," so zero is the correct base when no item fits.

```python
def knapsack01(items, capacity):
    if capacity < 0 or any(weight <= 0 for weight, _ in items):
        raise ValueError("capacity must be nonnegative and weights positive")
    best = [0] * (capacity + 1)
    for weight, value in items:
        for room in range(capacity, weight - 1, -1):
            best[room] = max(best[room], best[room - weight] + value)
    return best[capacity]
```

Take `items = [(2, 4), (3, 5)]` and `capacity = 4`. The correct answer is 5: the two items do not fit together. If capacity were 2, the answer would be 4. At capacity 4, an *ascending* inner loop could reuse the first weight-two item: it would set `best[2] = 4` and then wrongly set `best[4] = 8` in the same item pass. Descending iteration prevents that. For *unbounded* knapsack, ascending is the intended policy; the loop direction encodes whether reuse is legal. Time is $O(nC)$ and memory $O(C)$; this is pseudo-polynomial in the numeric capacity C.

> **State.** One best value for every available capacity, representing items already processed.
>
> **Invariant.** Before each new item's capacity loop, no entry uses that item; on a descending pass, smaller-capacity entries still refer to earlier items.

Update. At each capacity, choose the better of skipping and taking the current item exactly once.

Break test. Use an ascending pass for the two items above and capacity 4. Reporting 8 silently converts zero-one knapsack into a different, reusable-item problem.

9.4 A straight run: start only where a run starts

"Straight" needs a contract. Here it means the longest run of consecutive integer *values*, in any input order; duplicate values do not extend the run. A poker straight with special ace or wildcard rules is a different question. Store values in a set. A number whose predecessor is present cannot begin a new run, so count upward *only* from numbers without a predecessor.

```
def longest_straight(nums):
    values = set(nums)
    best = 0
    for start in values:
        if start - 1 in values:
            continue
        end = start
        while end in values:
            end += 1
        best = max(best, end - start)
    return best
```

On `[100, 4, 200, 1, 3, 2, 2]`, only 1 starts the four-value run `1, 2, 3, 4`. Every value belongs to at most one counted run, so this uses expected $O(n)$ time and $O(n)$ space. Sorting also works, but costs $O(n \log n)$ and needs a separate duplicate rule.

Break test. Begin a walk from *every* value on a run of n consecutive integers. The repeated counts cost $1 + 2 + \cdots + n$, even though a set made each membership test cheap.

Source notes: #4.

Small change: use every card exactly once. In the Hand of Straights problem, can a hand be partitioned into groups of a given positive width, each containing consecutive ranks? Duplicates now matter: `[1, 1, 2, 3]` cannot form two width-two groups because only one rank-2 card exists. A set has forgotten that shortage. Instead count copies with `Counter`, then begin groups at the lowest rank still owed and remove *all* its copies from each rank in the group.

```
def can_partition_straights(hand, width):
    if width <= 0:
        raise ValueError("width must be positive")
    if len(hand) % width:
        return False
    remaining = Counter(hand)
    for first in sorted(remaining):
        copies = remaining[first]
        if copies:
            for rank in range(first, first + width):
                if remaining[rank] < copies:
                    return False
                remaining[rank] -= copies
    return True
```

For `[1, 2, 3, 6, 2, 3, 4, 7, 8]` with width 3, the groups can be `[1, 2, 3]`, `[2, 3, 4]`, and `[6, 7, 8]`. An empty hand forms zero groups and returns true. If there are u distinct ranks, sorting costs $O(u \log u)$ and the straightforward group scan costs $O(u \cdot \text{width})$; the counter stores $O(u)$ entries. One word in the question—*all* cards—changed a membership set into a count map and made ordered starts necessary.

9.5 Candy: satisfy constraints from both directions

Give each child at least one candy. A child rated higher than an adjacent child must get more candy, and the total should be smallest. A left-to-right pass can enforce the rule for the left neighbor, but knows nothing about a higher rating than the *right* neighbor. A right-to-left pass repairs that second rule without undoing the first: take the maximum of the old obligation and the newly required count.

```
def minimum_candies(ratings):
    candies = [1] * len(ratings)
    for i in range(1, len(ratings)):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1
    for i in range(len(ratings) - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)
    return sum(candies)
```

For ratings `[1, 0, 2]`, the first pass gives `[1, 1, 2]`; the second gives `[2, 1, 2]`, total 5. Equal ratings impose no inequality, so `[1, 2, 2]` takes `[1, 2, 1]`. Each pass certifies one direction of a local constraint. Replacing rather than taking `max` on the way back can break the requirement already established on the way forward.

Break test. On `[1, 3, 4, 5, 2]`, overwrite the rating-5 child's four candies with only one more than the rating-2 child. The right rule holds, but the left rule fails.

Source notes: #3, #111.

9.6 Trapped rainwater: settle the side with a known bound

Water above position i is $\min(\text{highest left wall}, \text{highest right wall}) - h_i$. Computing both maxima for each position repeats work. Two pointers carry the best wall seen from each end. When the left wall is no higher than the right wall, the right already supplies enough height: settle the left cell, then advance left. The symmetric case settles right.

```
def trapped_water(heights):
    left, right = 0, len(heights) - 1
    left_wall = right_wall = 0
    water = 0
    while left <= right:
        if left_wall <= right_wall:
            left_wall = max(left_wall, heights[left])
            water += left_wall - heights[left]
            left += 1
        else:
            right_wall = max(right_wall, heights[right])
            water += right_wall - heights[right]
            right -= 1
```

```
    return water
```

For nonnegative heights `[3, 0, 2, 0, 4]`, the interior cells hold `3, 1, 3` units, so the answer is 7. Both walls begin at zero; an empty input returns zero. Each pointer moves once per cell, giving $O(n)$ time and $O(1)$ extra space. The reason to move a pointer is not "that side looks smaller now": its *limiting wall* is already known.

Break test. At height 2 in that example, use only the nearest left neighbor of height 0. It misses the earlier height-3 wall and claims no water.

Source notes: #8, #9.

9.7 A reachability boundary instead of a DP table

For Jump Game, each nonnegative number tells how far one may jump from that index. A full DP table can mark reachable indices, but a running `farthest` boundary already summarizes all routes through the scanned prefix. If the next index is beyond it, no earlier position could have reached that index; scanning farther cannot repair the gap.

```
def can_reach_end(jumps):
    if not jumps:                  # no starting position
        return False
    farthest = 0
    for index, reach in enumerate(jumps):
        if index > farthest:
            return False
        farthest = max(farthest, index + reach)
        if farthest >= len(jumps) - 1:
            return True
    return False
```

For `[2, 3, 1, 1, 4]`, index 1 extends the boundary beyond the end. For `[3, 2, 1, 0, 4]`, the boundary stops at 3 and index 4 cannot be read as a reachable launcher. This is a useful contrast with knapsack: only one boundary survives, because the question is reachability rather than which collection of choices achieves the best value.

Break test. Update `farthest` from index 4 without checking `index > farthest`. The unreachable final 4 then creates a fictional route to itself.

Source notes: #67, #68.

Small change: ask for the fewest jumps. A single reachable/not-reachable boundary no longer answers the question. Treat each interval of reachable indices as a BFS layer: `current_end` marks the last index in this jump count, and `farthest` collects the next layer's right edge. Advancing past a layer boundary spends exactly one jump.

```
def minimum_jumps(jumps):
    if not jumps:
        return -1
    if len(jumps) == 1:
        return 0
    count = current_end = farthest = 0
    for index in range(len(jumps) - 1):
        if index > farthest:
            return -1
        farthest = max(farthest, index + jumps[index])
        if index == current_end:
            if farthest == current_end:
                return -1
            count += 1
            current_end = farthest
            if current_end >= len(jumps) - 1:
                return count
    return -1
```

The same input `[2, 3, 1, 1, 4]` now answers 2 rather than `True`; `[3, 2, 1, 0, 4]` returns `-1`. Checking for a stuck frontier before increasing `count` prevents a fictitious final jump.

9.8 Try the definitions

Copy this chapter's function listings into one Python file in reading order, then run the following calls. Every value after `##` is the expected printed result; the `Counter` import above is the only setup required.

```
print(max_nonadjacent([2, 7, 9, 3, 1]))              # 12
print(max_nonadjacent_circle([2, 3, 2]))             # 3
print(lcs_length("abcde", "ace"))                     # 3
print(grid_paths([[0, 0, 0], [0, 1, 0], [0, 0, 0]]))  # 2
print(knapsack01([(2, 4), (3, 5)], 4))               # 5
print(longest_straight([100, 4, 200, 1, 3, 2, 2]))  # 4
print(can_partition_straights([1, 2, 3, 6, 2, 3, 4, 7, 8], 3))  # True
print(minimum_candies([1, 0, 2]))                    # 5
print(trapped_water([3, 0, 2, 0, 4]))                # 7
print(can_reach_end([3, 2, 1, 0, 4]))                # False
print(minimum_jumps([2, 3, 1, 1, 4]))               # 2
```

Part II

Coding patterns

CHAPTER 10

State, invariant, update

Knowing that a problem is BFS does not decide when to mark a node, where to append a result, or what the queue contains at a particular instant. Those are the decisions that turn an idea into code. Make them explicit *before* you write the loop. A useful translation has four layers:

1. A **contract**: input, output, boundary cases, and whether mutation is allowed.
2. A **state**: the minimum information needed to resume the work.
3. An **invariant**: a sentence true before each iteration and restored after it.
4. A **transition**: read state, do one unit of work, update state, and either continue or stop.

An algorithm describes why the transitions reach the answer. A coding pattern specifies the transitions themselves. The latter is reusable even when the problem statement changes.

10.1 Begin with a contract, not a loop

Suppose a graph problem asks for a shortest path between two vertices. "Run BFS" leaves important details unresolved. Does it ask for a distance, a list of vertices, or a yes/no answer? Are edges directed? Is an unreachable target an empty list or `-1`? Is the source allowed to equal the target? Write down an example of each possible return shape before choosing state. For a path, the state needs a parent for each discovered node; for a distance only, it may need just distances; for reachability, a visited set suffices.

For each function, state the *output contract* and the *ownership contract*. A function returning a new array must not accidentally alter its input. An in-place matrix operation must say so. A class with `get()` and `put()` must define which operations count as a use. Return types should not change between ordinary and empty cases: a path finder that returns a list normally should return `[]` when no route exists, unless its API explicitly requires something else.

> **State.** Input graph, source, target; queue, discovered nodes, parents, and possibly distances.
>
> **Invariant.** Every parent entry describes the first edge that discovered that node; the queue holds discovered but unprocessed nodes.
>
> **Update.** When a new neighbor is found, assign its parent and mark it before enqueuing.

Most failures attributed to a complex algorithm start here: the correct loop computes the wrong *shape* of answer. In #39, adding parents is not a variant of BFS's search order; it is a consequence of requesting a path.

Break test. Source equals target. A correct path answer is a one-vertex list; returning no path because the traversal found no edges violates the contract.

10.2 The state–invariant–update sentence

Write three lines on scratch paper, not in code:

State: What values tell me where I am and what I still owe?
Invariant: What do those values mean *now*?
Update: Which action preserves that meaning?

"`left` is an integer" is a type, not an invariant. "All indices before `left` are proven too small" is an invariant. "`seen` is a set" is a type. "Every node in `seen` has been claimed for exactly one queue entry" is the invariant that prevents duplicate work. An invariant should let you predict the result of one iteration without rereading the whole history.

For lower-bound search (#14), the state is `lo`, `hi`, and a sorted array. Before each iteration, all positions below `lo` fail the predicate, all at or beyond `hi` pass it, and only `[lo, hi)` is unknown. Checking `mid` proves one side; moving exactly one boundary restores the sentence. The loop ends when there is no unknown position. That is a more powerful description than memorizing the two assignments.

For a one-pass running maximum, say "`best` is the largest candidate among the first `i` items". Processing item `i` creates one more candidate, so compare it with `best` *before* incrementing `i`. This language exposes off-by-one errors: if you claim to have processed `i` items, your state cannot include item `i` yet under zero-based indexing.

Break test. Use the input `[1, 3, 3, 5]` with target 3 to check a lower-bound loop. If `hi = mid - 1` after a passing `mid`, the first 3 can be discarded; the alleged invariant no longer holds.

Source notes: #14, #28, #39.

10.3 The blank-page S/I/U worksheet

When you know an algorithm but cannot start coding, write the following six answers next to the problem. They are deliberately about *the program's present moment*, not about the algorithm's history:

1. **Unit of work.** Is one step an item, index, graph node, candidate path, range, or public method call?
2. **State.** Which named variables survive that step? Which ones are local to it, and which persist between calls?
3. **Invariant.** What precise fact holds just before the step? Which part of the input is already settled?
4. **Update.** Which incoming item invalidates or extends the fact? What must be written, and in what order?
5. **Answer event.** When is a candidate complete enough to record or return?
6. **Progress.** Which boundary advances or which pending item disappears? Why must the work end?

If an answer is only a data type, sharpen it. "`pending` is a stack" is not useful; "each index in `pending` has not yet met a warmer day, and their temperatures decrease from bottom to top" almost writes the monotonic-stack loop by itself. "`parent` is a dict" is not enough; "a parent is assigned once at first discovery" tells you where the assignment belongs.

For Top-K (#19), let one step be the next input value. The state is a min-heap of at most `k` selected values. The invariant is that it contains the `k` largest values of the processed prefix (or all of them if fewer than `k` exist). The update is to push the new value and pop the smallest when size exceeds `k`. The answer event is exhaustion of the input; only then is the heap's root the kth largest of the *whole* array. Progress is one more value consumed. These statements justify each of the four lines of the implementation before any `heapq` call is written.

For backtracking (#30), one step is one call frame. The state is `path` plus `start`; the invariant is that `path` contains only choices with increasing indices and siblings begin from the same parent path. The update is append, recurse with `i + 1`, then pop; the answer event is *every* frame for subsets but only a complete leaf for permutations. Progress is the increasing start index, which bounds recursion depth. If you cannot say why the pop restores the invariant, the helper's state ownership is not yet defined.

For BFS (#39), one step is one pop. The queue holds discovered, unprocessed nodes; `seen` records nodes already claimed for one queue slot. Expansion claims new neighbors *before* enqueueing them. The answer event depends on the requested result: process a node for traversal order, discover it for shortest distance, or complete a whole round for level order. Progress is that each node can be claimed at most once. POPREXEN is this S/I/U worksheet turned into four spoken moves.

Break test. Use the worksheet on a sliding window. If "`left` always moves right" is the only invariant you can write, you still cannot tell whether a window is valid *before* or *after* shrinking it.

Source notes: #13, #19, #30, #39.

10.4 POPREXEN: pop, process, expand, enqueue

For a traversal with a worklist, use the spoken rhythm **pop – process – expand – enqueue**. The queue is a promise to do work later, not a list of completed work. *Pop* takes one promise. *Process* handles the item itself, such as recording it in the output. *Expand* generates possible successors. *Enqueue* accepts the successors that pass the discovery rule. Enqueue belongs inside expand; the four words are a way to locate responsibilities, not four separate passes.

```
from collections import deque

def bfs_order(graph, start):
    queue = deque([start])
    seen = {start}                # the start is already claimed
    order = []
    while queue:
        node = queue.popleft()    # POP
        order.append(node)        # PROCESS
        for nxt in graph[node]:   # EXPAND
            if nxt not in seen:
                seen.add(nxt)     # claim before anyone else can
                queue.append(nxt) # ENQUEUE
    return order
```

Trace the queue in four columns on a small diamond-shaped graph:

Pop	Processed	New claims	Queue after
–	–	A	[A]
A	[A]	B, C	[B, C]
B	[A, B]	D	[C, D]
C	[A, B, C]	none (D claimed)	[D]
D	[A, B, C, D]	none	[]

Edges are A–B, A–C, B–D, C–D; neighbors appear in that order. The queue is *pending*, not visited and not output. After B is popped, D has been discovered, but D has not been processed. That distinction matters if a problem stops on *discovery* or on *processing*.

Tree level order (#32) adds a snapshot `size = len(queue)` at the start of a round and processes exactly that many pops before saving the level. Shortest path (#39) assigns distance and parent at enqueue. Kahn's topological traversal (#40) expands by decrementing indegrees; a successor is eligible only when its count reaches zero. Multi-source BFS (#78) seeds every source before the first pop. Same four motions, different state and admission condition.

Break test. Move `seen.add(nxt)` from the enqueue branch to the pop site. In the diamond, D is queued from both B and C; it can be processed twice.

10.5 Distinguish frontier, answer, and evidence

A *frontier* is work waiting to be explored; an *answer* is what the caller will receive; *evidence* is the information needed to reconstruct or prove that answer. These containers need not be the same. In BFS, `queue` is frontier, `order` is an answer, `parent` is evidence, and `seen` is an admission rule. In Dijkstra's algorithm, the heap is a frontier of *candidate* routes and `dist` holds the best known evidence; an old candidate can be stale without corrupting the answer.

Ask when an answer becomes final. In a window that must contain required characters, record a window *before* removing its leftmost character; after removal it may be invalid. In a tree diameter computation, a path through a node is final when both child results are known; update the answer on the way back up. In topological sort, append on pop, because the node's prerequisites have all been discharged. If you cannot name the event that makes a candidate final, you have not yet chosen the right update site.

State. Frontier, answer, and any evidence needed to build it.

Invariant. Every element in the answer has passed its finality test; pending elements have not.

Update. At the finality event, record the answer once; keep the frontier and its bookkeeping separate.

Break test. Run Dijkstra with a route to X of length 9, then discover one of length 4. Returning when X is *pushed* finalizes the 9 instead of the 4.

Source notes: #13, #33, #40, #42.

10.6 Draft the spine before filling in helpers

Every implementation has a spine: initialization, loop or recursion, one transition, and return. Leave specialized operations as named holes while you check the control flow. For a graph traversal: `make_graph`, `seed_frontier`, `pop`, `is_new`, `record`, `push`, `return`. For backtracking: `choose`, `recurse`, `undo`. For binary search: `bounds`, `mid`, `predicate`, `move_boundary`, `return`.

Then give each helper a *contract*, not an adjective. "Process a node" is vague; "return its best downward arm to the parent and update the best complete path through it" says exactly what a tree helper must do. "Check feasibility at speed s without changing shared state" is a usable binary-search helper contract. Only after that should code be placed in the holes.

This ordering shortens debugging. If the skeleton cannot explain which item is popped next or when a result is recorded, a correct `neighbors()` helper will not rescue it. If it can, a broken neighbor calculation has a small, isolated surface. Keep the main function readable as the traversal story and move problem-specific adjacency or validity work into helpers when that makes the story clearer.

Break test. Say the invariant aloud at the `return` statement. If the loop only guarantees "some candidates are processed," returning a "best" value needs an explicit argument that all relevant candidates were covered.

CHAPTER 11

Where a condition belongs

A correct Boolean expression at the wrong time is still a bug. Check a neighbor *before* claiming it; discard a stale heap entry *after* popping it; record a covering window *before* shrinking it; decide that a graph has a cycle *after* a traversal runs out of eligible nodes. When translating pseudocode, annotate each test with its event.

Test site	Question	Typical move
Before initialization	Is the input usable under the contract?	Empty grid, missing root
At discovery	May this candidate enter the frontier?	Bounds, wall, `seen`
At pop	Is this queued item still current?	Stale heap key, cancelled job
Before mutation	Is the current state a valid answer?	Save best window
After mutation	Did a threshold just change?	Count became zero
After exhaustion	What did we fail to reach?	Unreachable node, directed cycle
On return/unwind	What must be restored or combined?	Unmark path, tree fold

11.1 Guard before the first unsafe read

Take a grid traversal. `len(grid[0])` is unsafe when `grid` is empty; `grid[r][c]` is unsafe until `r` and `c` have been checked. A guard is not an apology after the crash: it protects the first operation that needs its precondition. Write it immediately before that operation or make it a documented precondition of a helper.

```
def count_islands(grid):
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    # Now indexing grid[r][c] is safe after coordinate bounds checks.
```

The two early returns above express the problem's empty-board contract. Do not replace invalid inputs silently with convenient defaults if the API does not define them. For example, a cache with negative capacity or a weighted graph with a negative edge deserves an explicit precondition or error if its algorithm cannot support the input; pretending it is an empty cache or running Dijkstra anyway manufactures a success-shaped wrong answer.

For recursion, order base cases so the earliest one protects later reads. A null tree node returns before accessing `node.left`; a word-search helper checks "all characters matched" before trying to index the next character. Describe which values are legal before choosing any sentinel.

Break test. Call a grid function with `[]`, then with `[[]]`. One guard covers the first input but not the second; reading `grid[0][0]` must still wait for a real cell.

Source notes: #38, #74, #78.

11.2 Admit at discovery, not at processing

In an unweighted BFS, `seen` means *already has a queue entry*. The first discovery gives a shortest distance, so mark, assign distance and parent, and enqueue in one branch. Splitting those operations across the loop allows another predecessor to claim the same neighbor before its first entry is popped.

```
for nxt in graph[node]:
    if nxt not in distance:
        distance[nxt] = distance[node] + 1
        parent[nxt] = node
        queue.append(nxt)
```

This is a single transition: "undiscovered" becomes "discovered at distance `d + 1`". In a grid whose cells may be changed, writing a discovered cell's marker before enqueueing plays the role of `seen`. In Kahn's algorithm, the condition is different: enqueue a course only *when* its indegree reaches zero, not whenever a prerequisite edge is removed. Otherwise it enters too soon or more than once.

Distinguish *claiming* from *processing*. Claiming says nobody else should schedule the same work; processing does whatever the output requires. If the goal is enough to terminate, BFS may stop when the target is discovered. If the output requires completing its whole layer, stop at the layer boundary instead.

Break test. Return to the diamond in section 10.4. If both B and C can schedule D, the discovery condition did not establish exclusive ownership.

Source notes: #32, #39, #40, #78.

11.3 Reject stale work at pop time

A priority queue cannot cheaply remove an arbitrary old candidate. Often the best move is to leave it there and check it when it surfaces. Dijkstra may push `(9, x)` and later `(4, x)`. When `(9, x)` is popped, `dist[x]` is already 4, so that item has lost its right to expand. The same pattern appears in stock-price trackers, job schedulers, and TTL caches: add a new record, let the old one expire logically, discard it physically when it reaches the top.

```
while heap:
    cost, node = heappop(heap)
    if cost != best[node]:
        continue
    for nxt, weight in graph[node]:
        candidate = cost + weight
        if candidate < best[nxt]:
            best[nxt] = candidate
            heappush(heap, (candidate, nxt))
```

The test belongs at pop because only then is the queued item's current relevance known. For a scheduler, store a unique sequence or generation per job; checking just the name would mistake a cancelled old job for a newly scheduled job with the same name. Heap comparison tuples also need an unambiguous tie-breaker if the remaining fields contain unorderable objects.

Break test. Schedule "send" at time 5, cancel it, then schedule "send" at time 10. A check by name alone wrongly runs the old event at time 5.

Source notes: #42, #94, #140, #149.

11.4 Record while a condition still holds

In a minimum covering window, an expanded window becomes eligible when `missing == 0`. The inner loop should first compare the current window against the best answer, then remove the leftmost character. Removal may make it invalid. A window can be *valid* without being *minimal*: the repeated shrink tests find the narrowest valid window ending at the current right index.

```
for right, ch in enumerate(text):
    add(ch)                       # extend the right boundary
    while missing == 0:           # state is valid here
        if right - left + 1 < best_length:
            best_start = left
            best_length = right - left + 1
        remove(text[left])        # may invalidate coverage
        left += 1
```

The `add` and `remove` operations are conceptual here; they must update the same counters as `missing`. A different problem, longest valid window, usually shrinks *while invalid* and records *after* the inner loop restores validity. Switching between these two loops without moving the answer update is a common source of wrong but plausible outputs.

Break test. On `t = "AB"`, a covering window `"AAB"` is valid. Drop the left `A`: `"AB"` is valid and better. Drop again: `"B"` is invalid. Recording only after the second drop loses both answers.

Source notes: #11, #13, #53, #54.

11.5 Test termination with the state it proves

Some facts become knowable only when a frontier is exhausted. Kahn's queue empties when no vertex has indegree zero; if fewer than `n` vertices were emitted, the remainder participates in or depends on a directed cycle. A BFS queue empties with an unreached target when there is no route. An interval sweep ends with the last open interval still waiting to be appended. A monotonic stack may end with unresolved indices unless you flush it or define their answer as a default.

```
while ready:
    course = ready.popleft()
    order.append(course)
    for follower in outgoing[course]:
        indegree[follower] -= 1
        if indegree[follower] == 0:
            ready.append(follower)
if len(order) != number_of_courses:
    return []                     # cycle prevents a complete order
return order
```

This final test is not another per-node admission check. It compares the whole output with the original problem contract: "a complete order contains exactly `n` vertices". An early return can be safe, but only if the information available at that moment already proves the answer. The trace should identify the first instant of certainty.

Break test. A graph has two disconnected components, one acyclic and one cyclic. Returning as soon as the queue empties after examining only the first component confuses incomplete initialization with a true cycle.

Source notes: #40, #62, #79, #83.

11.6 Restore on unwind, even after success

Backtracking owns temporary state: a path, a used-cell mark, a running sum. One call may borrow that state while it explores, but it must return it in the same condition in which it received it unless the API explicitly allows mutation. A successful early return is not an excuse to leave the board marked; another start or a second call may need it. Save the old value, make the mark, examine choices, restore, then return the result.

```
old = board[row][col]
board[row][col] = "#"            # only this path owns the cell
found = False
for next_row, next_col in neighbors(row, col):
    if search(next_row, next_col, index + 1):
        found = True
        break
board[row][col] = old            # happens after success or failure
return found
```

For a complicated body with several returns or possible exceptions, Python's `try ... finally` can guarantee restoration. For a small search, one exit site after restoration often makes the contract clearest. A global visited set in a graph has a different meaning: its marks are permanent for the traversal, so do not undo them. The right timing follows from what "visited" means, not from whether the traversal happens to use recursion.

Break test. Run a failed search and then a successful search on the same board. If the first call left marks behind, the second can be wrong even though either call on a fresh board passes.

Source notes: #30, #31, #74, #113.

CHAPTER 12

Who owns the state?

"Keep a visited set" is insufficient advice until you know its owner and lifetime. A local set lives for one call. A recursive path mark lives for one branch. An object field survives calls. The output list may be reused, but the path snapshot must not alias the mutable working list. Place state where its meaning remains true for exactly as long as it is needed.

12.1 Local, call-frame, shared, or member state

A helper can read parameters, return a value, and optionally update shared state. Choose the smallest lifetime that matches the invariant:

Lifetime	Good fit	Common leak
Loop local	Current candidate, two-pointer indices	Accidentally reused next round
Call frame	Tree bounds, path index, subtree return	Confused with sibling state
One invocation	`seen`, output, global best for this run	Reused across independent calls
Object field	Cache contents, clock, browser history	Not updated by every API operation

Start with local variables. Move to an instance field only if the next public call must see the value. A `Trie` must retain children and word-end marks after `insert`; a one-shot BFS must not retain its `seen` set between two different graphs. A mutable class field declared at class scope can unexpectedly be shared by instances; initialize per-instance containers in `__init__`.

> **State.** The values one invocation needs; persistent fields only for cross-call behavior.
>
> **Invariant.** At the beginning of each public call, fields reflect all earlier calls and locals reflect only this call.
>
> **Update.** Write each field at every event that changes its meaning, including misses and removals when the contract says they count.

Break test. Create two cache instances; insert a key in one and read the other. If both see it, state has the wrong owner.

Source notes: #21, #43, #44, #91.

12.2 Visited is a state machine, not always a set

Give "visited" a precise definition before choosing a container. In BFS it is *claimed for a queue entry*, and the mark happens at discovery. In graph DFS it can mean *entered at least once*, so mark on entry and never undo. In directed cycle detection you need *three* states: not entered, on the current path, and finished. In grid backtracking the mark means *on this candidate path*, so undo it before exploring a sibling. In Dijkstra a tentative `dist` does not mean final; compare queued costs to current best instead of treating the first heap push as a permanent visited mark.

Sometimes the structure already encodes the states. A distance array initialized to `-1` means undiscovered until the first assignment. A grid cell changed from `"1"` to `"0"` means counted in an island. A topological

indegree reaching zero means eligible for the queue. Using a second set when one of these facts already serves the same invariant adds a second source of truth and a new opportunity for disagreement.

```
# Permanent for a graph traversal:
seen.add(node)
# Temporary for a single candidate path:
path.add(cell)
search(next_cell)
path.remove(cell)
# A third state for directed graphs:
color[node] = GRAY
visit_children(node)
color[node] = BLACK
```

Break test. The diamond 0->1->3 and 0->2->3 is not a cycle. A plain "already visited" test at 2->3 falsely reports one; only "currently gray" would be evidence of a back edge.

Source notes: #37, #38, #39, #74, #81.

12.3 A helper needs a return contract

Before writing a recursive helper, say what it returns and what shared state, if any, it updates. The return value should serve its caller, not automatically be the final answer to the whole problem. A height helper returns one downward arm because its parent can extend only one. It updates a separate best value for a completed path that joins both arms. An LCA helper can return a found target from one child, or the LCA if both sides have found one. An index-based deserializer can return a pair `(node, next_index)` so sibling calls know which tokens remain.

```
def height(node, answer):
    if node is None:
        return 0
    left = height(node.left, answer)
    right = height(node.right, answer)
    answer[0] = max(answer[0], left + right)
    return 1 + max(left, right)   # one arm for the parent
```

In plain Python, a one-item list such as `answer = [0]` makes a small shared update explicit; alternatively a nested helper can use `nonlocal`, or return both the arm and the best completed path. Choose one method per helper and keep it consistent. An explicit result pair avoids hidden writes; a small shared accumulator can be simpler when many calls contribute to one answer. A member field for a one-off DFS is usually unnecessary and risks a stale answer on the next invocation.

Break test. A tree root has two long child arms. Returning `1 + left + right` to its parent creates a forked "arm" that a parent can no longer extend as a single path.

Source notes: #33, #34, #35, #36, #75.

12.4 Mutability and identity: snapshot the result

Backtracking usually has one working list `path`; recursion appends on descent and pops on ascent. That list is shared by all calls. `answer.append(path)` stores a reference to the same object, not its current contents. Use `answer.append(path.copy())` when a candidate is complete. This is an ownership transfer: the output owns a snapshot, while the recursion still owns the working path.

```
answer = []
path = []
def walk(start):
    answer.append(path.copy())
    for index in range(start, len(nums)):
        path.append(nums[index])
        walk(index + 1)
        path.pop()
```

The same issue appears in interval merging. If the output must be independent of the input, appending `interval` and later changing `merged[-1][1]` modifies the caller's list. Append a new two-element list instead. A copied outer list of lists is not a deep copy of its inner lists; copy at the level you intend to mutate. On the other hand, a design class that stores live node objects *wants* shared identity: the hash map must point to the exact node threaded into the linked list.

Break test. Generate subsets of `[1, 2]`. If the output is four empty lists, the entries all alias the path that was emptied during unwind. Then mutate one output subset and check whether the others change.

Source notes: #26, #30, #31, #44.

12.5 Carry, derive, or cache?

For each value in the state, ask whether it is essential or merely recomputable. Sliding-window character counts are essential to test validity without rescanning the whole window; a second copy of the full window string is usually redundant. A BFS level can be carried as `dist[node]` or derived by a snapshot of `len(queue)` each round; use distances if individual nodes need them, levels if output is grouped. An LRU cache needs both a map (fast lookup) and a linked list (fast recency changes) because neither can derive the other's answer in constant time.

Caching a derived value is sensible when its update rule is cheap and unambiguous. `fresh` in multi-source grid BFS starts as the count of fresh cells and drops exactly when a cell becomes rotten; checking `fresh == 0` then avoids rescanning the board. But every state variable adds a synchronization obligation. Write down the event that changes it, and consider whether every branch handles that event once. If not, recompute it or merge it into an existing variable.

> **State.** Only what future transitions cannot cheaply derive, plus any deliberately cached summaries.
>
> **Invariant.** Each cached summary equals the fact it summarizes.
>
> **Update.** At the same event that changes the underlying fact, update the summary once; never repair it later "just in case".

Break test. Count a grid's fresh cells; rot one cell from two sources in the same layer. Without claiming the cell at enqueue, `fresh` falls twice and a premature zero hides another fresh cell.

Source notes: #21, #44, #78, #140.

CHAPTER 13

One pass, with a memory of the past

A scan can do much more than read one item at a time. The key question is: what summary of the prefix is sufficient to decide something about the new item? Often it is a map, a running extreme, a count, or one candidate. The order of *read old summary, make a decision, write new summary* is the coding move. Making that order explicit prevents a surprising number of self-pairs, double counts, and wrong answers on the first element.

13.1 Look up before storing the current item

In Two Sum (#1), `seen` maps values in the already processed prefix to their indices. At index `i`, the complement of `nums[i]` must come from an *earlier* index. Ask the old map first; store the new value afterward. The word "earlier" is the invariant, not a commentary on speed.

```
seen = {}
for index, value in enumerate(nums):
    need = target - value
    if need in seen:
        return [seen[need], index]
    seen[value] = index
return []
```

For `[3, 3]` and target 6, the first 3 is stored, the second sees it, and the indices differ. If storage happens first, the first 3 could pair with itself. The same timing applies to "does the prefix already contain this signature?" problems. Choose the map's values from the output contract: index for a pair, count for a number of pairs, first position for a longest span, list of items for a group.

State. Map of facts about items strictly before the current index.

Invariant. Every stored index is less than the index being processed.

Update. Query the old map for the complementary fact, then insert or update the current fact.

Break test. Target 6 with only one input item `[3]`. A version that inserts before lookup incorrectly returns the same index twice.

Source notes: #1, #5, #92, #150.

13.2 Prefix summaries and the deliberately empty prefix

A subarray `nums[left:right]` has sum `prefix[right] - prefix[left]`, with `right` exclusive. If the current running sum is `s`, an earlier prefix `s - k` produces a subarray of sum `k`. Count its occurrences before recording the current `s`. `counts = {0: 1}` represents the empty prefix before any input. Without it, a valid subarray starting at index zero is invisible.

```
counts = {0: 1}                # one empty prefix
total = 0
answer = 0
for value in nums:
    total += value
    answer += counts.get(total - target, 0)
    counts[total] = counts.get(total, 0) + 1
```

For `nums = [1, -1, 1]` and `target = 1`, the prefixes seen after each item are 1, 0, 1. The first 1 pairs with the empty prefix; the final 1 pairs with both earlier zero prefixes (empty and after the second item). The answer is 3. The map is of *counts*, not just a set, because two different starting positions produce two different subarrays.

Some prefix problems need a different value: to find the *longest* subarray with a given sum, map each prefix to its earliest index and do not overwrite it. To find whether any matching subarray exists, a set may be enough. Be exact about whether the current prefix may pair with itself; a nonempty subarray requires an earlier prefix, hence query-before-insert. Unlike a nonnegative-sum window, the prefix method works with negative values because it never assumes that shrinking will reduce the sum.

Break test. With `nums = [0]` and target 0, there is one nonempty subarray, not two. Updating the zero count before querying it counts the empty range at the current position as well.

Source notes: #2, #47, #92.

13.3 Work only at the start of a run

Sometimes the correct optimization is not a faster inner loop but fewer *starts*. In a set of integers, count a consecutive run only from a value whose predecessor is absent. Every internal value has a predecessor and cannot start new work. This makes the total number of increments linear, even when one run is very long.

```
values = set(nums)
longest = 0
for value in values:
    if value - 1 in values:
        continue                # not the left boundary of a run
    length = 1
    while value + length in values:
        length += 1
    longest = max(longest, length)
```

This "start-only" pattern appears in strings, intervals, and graph components. A flood fill starts only at an unclaimed land cell; the fill claims the whole component so the outer scan will never start it again. In interval merging, sorting ensures that only the latest open interval can be extended; a new start after a gap begins a new merged run. Ask: which test certifies that this item is the unique start of the unit of work?

> **State.** Set of candidates, a local run length, and global best.
>
> **Invariant.** Only items without predecessors open scans; a scanned item belongs to one such run.
>
> **Update.** Skip non-starts, scan each run once, compare its result to the global best.

Break test. Iterating the original array rather than the deduplicated set can rescan a run once for every duplicate start. Deduplicate first.

Source notes: #4, #26, #38, #84.

13.4 Fold a prefix into "current" and "best"

Many optimizations need two values: the best candidate that must end *here*, and the best candidate seen *anywhere so far*. They answer different questions. In Kadane's algorithm, a subarray ending at the new item either starts at that item or extends the previous ending subarray. The global best then compares itself with that new ending value.

```
ending = nums[0]
best = nums[0]
for value in nums[1:]:
    ending = max(value, ending + value)
    best = max(best, ending)
```

Initializing `best = 0` would falsely claim an empty subarray is allowed and return zero on all-negative input. By contrast, a stock profit with the choice not to trade can legitimately start at zero. This is not a magical initialization trick: it follows from which candidates the contract permits.

The same fold holds a running minimum price and the best profit so far (#29); a variable `low` can hold the minimum of earlier prices, while `best` records the best sell-at-today candidate. Some problems require a current *state* rather than an output candidate: an XOR accumulator (#135) or a running GCD (#128) is still a fold, with the invariant "it summarizes exactly the processed prefix".

> **State.** Summary of the processed prefix and a best answer, if needed.
>
> **Invariant.** Before reading an item, the summary describes exactly the previous items.
>
> **Update.** Build the new candidate from the old summary and the item; then update the summary and global answer in their proper order.

Break test. Try `[-5, -2]`. Maximum nonempty subarray is `-2`. Any initialization that treats "take nothing" as a candidate returns 0.

Source notes: #28, #29, #102, #128, #135.

13.5 Two passes when the future is unavailable

At index `i`, a left-to-right scan knows nothing about values to its right. If the answer depends independently on the left and right, record one side in an output array and bring the other side in on a second pass. For product except self, `output[i]` first receives the product of items strictly left of `i`; the reverse pass multiplies by the product strictly right of it. No division, and zeros need no special branch.

```
out = [1] * len(nums)
left_product = 1
for i in range(len(nums)):
    out[i] = left_product
    left_product *= nums[i]
right_product = 1
for i in range(len(nums) - 1, -1, -1):
    out[i] *= right_product
    right_product *= nums[i]
```

Both products are *exclusive*: at each assignment the current element has not yet joined that side's accumulator. Changing the multiplication order to include it defeats "except self". The two-pass idea also builds left/right maximum walls for rainwater, or prefix and suffix validity to answer questions about removing one element. A two-pass algorithm is often easier to audit than a compressed one-pass variant; optimize space only after both pass invariants are clear.

Candy distribution (#111) combines two directional lower bounds instead of multiplying them: a left-to-right pass satisfies each increasing pair seen from the left, and a right-to-left pass raises values with `max` to satisfy decreasing pairs from the other direction without breaking the first constraint.

Break test. Use `[2, 0, 4]`. Correct output is `[0, 8, 0]`. A division-based solution cannot handle the zero; an inclusive prefix produces a wrong value even without zeros.

Source notes: #3, #9, #72.

13.6 A candidate is not automatically an answer

Boyer–Moore's vote algorithm maintains a candidate and a balance. When the balance reaches zero, the next item becomes the new candidate; equal items add a vote, different items cancel one. The invariant says a majority of the entire input, *if one exists*, survives every pairwise cancellation. It does not assert that the final candidate is always a majority. If the problem does not promise existence, verify in another pass.

```
candidate = None
balance = 0
for value in nums:
    if balance == 0:
        candidate = value
    if value == candidate:
        balance += 1
    else:
        balance -= 1
if candidate is None or nums.count(candidate) <= len(nums) // 2:
    return None
return candidate
```

The difference between a provisional state and a final answer recurs in binary search on a feasibility predicate, streaming approximations, and randomized algorithms. Some have a second-pass verification available; some have a probabilistic answer contract. Never promote a candidate to a guarantee without checking what the invariant actually proves.

Break test. On `[1, 2, 3]`, cancellation still leaves a candidate, but no value appears more than half the time. A promise of a majority would make the verification optional; no such promise makes it essential.

Source notes: #102, #131, #146, #147.

CHAPTER 14

Move the edges of a sequence

Two pointers, a sliding window, a sorted merge, and a linked-list rewrite all manage a boundary between settled and unsettled data. The coding question is not "what are the two indices called?" It is "which region does each index delimit, and why is it safe to move this one now?" Draw the regions first; the conditions then have a place to live.

14.1 Converging pointers: justify the discarded side

When the input is sorted, left and right values bound the remaining search. For a target sum, a sum too small can be increased only by moving `left`; a sum too large can be decreased only by moving `right`. The pointer move is a proof that the discarded candidates cannot win, not simply a habit from the Two Sum template.

```
while left < right:
    total = nums[left] + nums[right]
    if total < target:
        left += 1
    elif total > target:
        right -= 1
    else:
        record(nums[left], nums[right])
        left += 1
        right -= 1
```

The code above is only a spine: a problem asking for all *unique* triplets also skips equal values after a hit and skips duplicate fixed elements in the outer loop (#7). Container With Most Water (#8) uses the same pair of indices for a different reason: move the shorter wall because keeping it while narrowing the width cannot improve the area. Trapping Rain Water (#9) uses two running walls and settles the side whose bound is lower. Do not transfer a pointer rule merely because the diagrams look alike; transfer the *proof of discard*.

> **State.** Two edges of an unexplored interval; optional sorted order or running bounds.
>
> **Invariant.** Every candidate outside the interval is already decided or safely discarded.
>
> **Update.** Evaluate the present pair and move only the side whose candidates are proven unable to improve the answer.

Break test. On heights `[1, 100, 100]`, moving the taller wall at the first comparison can discard the central pair of area 100.

Source notes: #7, #8, #9, #52, #66.

14.2 Expand right, repair left

A variable window is a contiguous region `[left, right]`. The outer loop introduces each new `right` exactly once. If a constraint is broken, the inner loop removes items at `left` until the window returns

to its promised state. The left edge never moves backward, so even a nested `while` loop can be linear: each item enters and leaves once.

For "at most k distinct", maintain character counts and a distinct counter; a count crossing from zero to one adds a distinct character, and from one to zero removes one. For "no repeated characters", a map of last indices lets left jump beyond the duplicate. In that version, guard the jump with `last[ch] >= left`: an old occurrence outside the window must not drag the left edge backward.

```
left = 0
last = {}
best = 0
for right, ch in enumerate(text):
    if ch in last and last[ch] >= left:
        left = last[ch] + 1
    last[ch] = right
    best = max(best, right - left + 1)
```

Not every condition supports shrink-to-repair. A window sum of nonnegative values falls when left advances; with negative values it may rise instead. This is why minimum-size subarray sum (#54) assumes positive values while prefix-sum counting (#2) handles negatives. Before choosing a window, say which monotonic effect adding and removing an item have on the condition.

Break test. In `"abba"`, the final `a` was last seen before the current `left`. An unconditional jump backward reports a false length-three window.

Source notes: #11, #53, #54.

14.3 Use one deficit with a clear sign

For a covering window, a map `need[ch]` can mean the number of copies still missing. Positive means a shortage, zero means exact coverage, negative means surplus. Maintain `missing` as the *total number of still-missing characters*, with multiplicity. This one number tells the inner loop when all requirements are met.

```
# need initially counts the characters in the required text
missing = len(required)
for right, ch in enumerate(text):
    if need.get(ch, 0) > 0:
        missing -= 1
    need[ch] = need.get(ch, 0) - 1
    while missing == 0:
        consider(left, right)     # cover holds before removal
        gone = text[left]
        need[gone] += 1
        if need[gone] > 0:
            missing += 1
        left += 1
```

Why check the old deficit before decrementing? The new character only fills a real gap if its old deficit was positive. Why check after incrementing on removal? A requirement becomes newly missing only if the new deficit is positive. For `text = "AAB"` and `required = "AB"`, the second A is surplus; it does not substitute for B. This sign convention is a small state machine that saves recounting every window.

State. Left and right edges, per-character signed deficits, and `missing`.

Invariant. `missing` equals the total positive shortfall of required characters.

Update. On entry decrement one deficit and possibly `missing`; on exit increment it and possibly `missing`.

Break test. Decrementing `missing` on every character in the target alphabet lets `"AA"` pass as a covering window for `"AB"`.

Source notes: #12, #13, #53.

14.4 Fixed width: add, evict, then inspect

A fixed-size window has no validity-driven inner loop. After adding the item at `right`, evict the item at `right - width` if that index exists; once `right + 1 >= width`, the state describes exactly `width` items. A standard alternative evicts after inspecting the completed window. Both work if you state which indices the state describes after each line; mixing them produces off-by-one errors.

```
window_total = 0
for right, value in enumerate(nums):
    window_total += value
    if right >= width:
        window_total -= nums[right - width]
    if right + 1 >= width:
        record(right - width + 1, window_total)
```

For anagram detection (#12), the summary is not a sum but a count vector; add one character and remove one character at each step. For sliding maximum (#64), a monotone deque holds candidates, and expiration tests the oldest index against the left edge. Separate *width management* from *summary management*: the window specifies which items are eligible, the summary makes queries cheap.

Break test. Width 3 and input `[4, 5, 6]`. The first output appears only after index 2 is added. If it appears at index 1, the loop confused the current index with the number of processed items.

Source notes: #12, #64, #138.

14.5 Sort, then keep only the active frontier

Sorting can turn an all-pairs question into one local comparison. In merge intervals, once starts are ordered, a new interval can only overlap the latest merged interval; if it starts after that interval ends, no later interval can fill the gap backwards. The output list is both a result and a compact representation of the settled prefix.

```
merged = []
for start, end in sorted(intervals):
    if merged and start <= merged[-1][1]:  # closed intervals
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])
```

The comparison changes with interval semantics. Closed intervals `[1, 4]` and `[4, 5]` share an endpoint and merge. Meetings usually use half-open intervals `[start, end)`: one meeting ending at 4 frees its room for another starting at 4, so an end time *less than or equal to* a start time is reusable. Range Module (#142) and calendars (#98) also demand a stated endpoint contract. Write "closed" or "half-open" by the input before coding the comparison.

If intervals are already sorted and disjoint and only one new interval is inserted (#65), do not sort again: copy the prefix that ends before it, absorb the overlapping run, then append the untouched suffix. The same active-frontier invariant now costs a single pass.

> **State.** Sorted intervals and one active merged interval or resource frontier.
>
> **Invariant.** All earlier output segments are final; only the active one can overlap the next input.
>
> **Update.** Extend its end with `max`, or close it and begin a new segment.

Break test. Input `[[8, 12], [9, 10]]`. Setting the merged end to 10 rather than `max(12, 10)` incorrectly shrinks a result.

Source notes: #26, #27, #65, #66, #98, #142.

14.6 Rewire a link only after saving the future

In a linked list, `node.next` is your only way to find what comes after the node. Before changing it, save its old value. Reversal makes this dependency visible: `next_node` owns the unprocessed suffix; `previous` owns the already reversed prefix.

```
previous = None
current = head
while current is not None:
    next_node = current.next
    current.next = previous
    previous = current
    current = next_node
return previous
```

For reorder list (#104), first find the middle, reverse the second half, and weave the halves together. Each phase has its own ownership boundary: split the halves before reversal so the original chain cannot form a cycle. A dummy head or permanent sentinel is useful when the first real node may be inserted or removed; it reduces special cases at the boundary rather than changing the algorithm.

Break test. Reverse `1 -> 2 -> 3`. If `current.next` is rewritten before saving it, the unprocessed suffix becomes unreachable after the first iteration.

Source notes: #6, #10, #44, #76, #77, #104.

CHAPTER 15

Search a boundary, not a lucky position

Binary search is often taught as "compare with the middle and halve the range." In code, the durable pattern is stronger: define a boundary between a proven false region and a proven true region, then shrink the unknown region without discarding the boundary. Sorted arrays, minimum feasible answers, partitions of two arrays, and time-indexed histories all reuse that contract.

15.1 Half-open search for the first true index

For a sorted array, `nums[i] >= target` becomes true once and remains true. Let `lo` be the first unknown index and `hi` the position just after the unknown region. Every index before `lo` fails; every index at or after `hi` passes, treating the end position as a sentinel that always passes. When the interval is empty, `lo` is the answer, including `len(nums)` if no element passes.

```
def lower_bound(nums, target):
    lo, hi = 0, len(nums)          # unknown interval [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < target:
            lo = mid + 1           # mid is false
        else:
            hi = mid               # mid might be first true
    return lo
```

The word "first" is why a passing `mid` remains in play. Change the predicate to `nums[mid] <= target` and the same loop finds the first value strictly greater than the target. Exact search uses the lower bound, then checks both `index < len(nums)` and equality. Do not mix this half-open version with a closed `[lo, hi]` search: `hi = mid - 1` would throw away a passing boundary here.

State. The first and past-the-last unknown positions.

Invariant. Everything left of `lo` is false; everything at or right of `hi` is true.

Update. Probe one unknown index; exclude it only if false, otherwise keep it as the possible first true.

Break test. Search for 3 in `[1, 3, 3]`. The answer is 1, not an arbitrary equal index. Then search for 4; the answer is 3, not an out-of-bounds read.

Source notes: #14, #47, #58, #92.

15.2 Move the predicate into a helper

Binary search on the answer trades a direct formula for a yes/no question: "Can I finish with capacity `x`?" First prove that feasibility is monotone. If a speed of 5 is enough to finish all work by the deadline, a speed of 6 is also enough. Search for the first feasible speed, with bounds that actually contain it.

```
def feasible(speed, piles, hours):
    needed = 0
    for pile in piles:
        needed += (pile + speed - 1) // speed
    return needed <= hours

lo, hi = 1, max(piles)            # under the promised constraints
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid, piles, hours):
        hi = mid
    else:
        lo = mid + 1
answer = lo
```

The helper should compute an answer at a hypothetical `x` without mutating shared state; the outer loop relies on repeatable evaluations. Check the extremal boundary: `hi` must be feasible under the problem's promises. If no solution is guaranteed, first handle that case explicitly rather than silently returning a non-answer. For a maximum feasible value, reverse the invariant and use a midpoint biased upward to ensure progress.

Break test. Set `lo = 0` for a speed whose feasibility divides by `speed`; the very first probe can attempt division by zero. The search bounds are part of the function's correctness, not setup decoration.

Source notes: #15, #57, #105, #106.

15.3 Partition search: certify both sides

Some searches move a cut rather than find an array element. In median of two sorted arrays (#55), a cut taking `i` items from A forces `j = half - i` items from B to join the left half. A valid cut has *both* crossing comparisons true: A's left edge is no greater than B's right edge, and B's left edge is no greater than A's right edge. A missing edge at an array boundary acts like negative or positive infinity, never like an ordinary zero.

Use the shorter array for the searched cut. For each `i`, compute `j`, the four values next to the cuts, and then decide: too much of A on the left means move `i` left; too little means move it right; both comparisons true means the cut is certified. This is an *inclusive* search over valid cut positions `0..m`; it is not the half-open lower-bound loop pasted verbatim.

> **State.** One cut `i`, implied cut `j`, and the four edge values.
>
> **Invariant.** The combined left part has the required number of items; the only unknown is whether it is ordered against the right part.
>
> **Update.** Check the two crossings, then discard cuts that put too many or too few A items on the left.

Break test. If A is empty, the valid cut uses `i = 0`. Reading `A[i-1]` as Python's `A[-1]` is not a sentinel and may crash or silently use the wrong element.

Source notes: #55, #105, #106.

15.4 Sorted halves, but a moving origin

In a rotated sorted array with distinct values, the whole current range is not sorted, but at least one half is. Compare `nums[lo]` with `nums[mid]` to identify that half. If the target lies within its endpoints, keep it; otherwise discard it. The order of questions is: "is mid the target?", "which half is sorted?", "does the target lie in that half?". A copied ordinary binary-search comparison against `nums[mid]` alone misses the rotation.

```
while lo <= hi:
    mid = (lo + hi) // 2
    if nums[mid] == target:
        return mid
    if nums[lo] <= nums[mid]:      # left half is ordered
        if nums[lo] <= target < nums[mid]:
            hi = mid - 1
        else:
            lo = mid + 1
    else:                          # right half is ordered
        if nums[mid] < target <= nums[hi]:
            lo = mid + 1
        else:
            hi = mid - 1
```

This version uses a *closed* unknown interval `[lo, hi]`, which is why it loops with `<=` and removes `mid` on each non-match. Duplicates weaken the "one half identifiable" test: equal endpoints can hide the pivot and may require shrinking an ambiguous boundary, losing the logarithmic worst-case guarantee. Write down the distinctness promise before writing the loop.

Break test. Try `[4, 5, 1, 2, 3]`, target 1. A comparison using only target versus mid has no proof that it discards a sorted region.

Source notes: #16, #56, #57.

15.5 Partition in place with region invariants

Quickselect and Dutch National Flag do not keep a sorted prefix. They maintain regions with specific promises: less than a pivot, unknown, and greater than a pivot. In the three-color partition, `nums[:low]` is 0, `nums[low:scan]` is 1, `nums[scan:high + 1]` is unclassified, and after `high` everything is 2. Only `scan` examines an unknown item.

```
low, scan, high = 0, 0, len(nums) - 1
while scan <= high:
    if nums[scan] == 0:
        nums[low], nums[scan] = nums[scan], nums[low]
        low += 1
        scan += 1
    elif nums[scan] == 2:
        nums[scan], nums[high] = nums[high], nums[scan]
        high -= 1               # re-examine what was swapped in
    else:
        scan += 1
```

The unfamiliar move is *not* incrementing `scan` after a swap with `high`; the arriving value was in the unknown region, so nobody has classified it. Quickselect uses a partition boundary to keep only the half

containing rank `k`; it does not need to sort both sides. Randomize the pivot when worst-case adversarial order matters, and state whether `k` is zero-based before comparing it with the partition index.

> **State.** Boundaries delimiting classified and unclassified regions.
>
> **Invariant.** Each classified region contains only its promised category; every item in the middle may still be anything.
>
> **Update.** Classify exactly the item at the scan pointer, advancing it only when the new item there is known.

Break test. Try `[2, 0, 1]`. If `scan` advances after swapping 2 with the far end, the replacement 1 (or 0 in a different input) can escape classification.

Source notes: #18, #52, #60, #103.

15.6 Choose a representation for ties and order

When the algorithm says "take the smallest", the code must define *smallest by what?* Python sorts tuples lexicographically, so a heap item `(time, sequence, id)` chooses earliest time and then earliest insertion sequence. This is helpful for merge-k streams, schedulers, and event sweeps. It is dangerous when equal primary keys cause Python to compare incompatible payload objects: add a unique numeric tie-breaker ahead of the payload.

Sorting is stable in Python: equal keys retain their old order. Exploit that only when the old order is meaningful. If a task specifies that ends precede starts at equal time, encode it, e.g. events of `(time, 0)` for ends and `(time, 1)` for starts. Otherwise a tied meeting boundary can count an extra room. A sort key, comparison rule, and tie policy belong in the contract beside the interval semantics, not as an afterthought.

Break test. A meeting ends at time 5 and another starts at time 5. If starts sort before ends, a sweep briefly counts two rooms even though a half-open schedule can reuse one.

Source notes: #17, #20, #27, #46, #70, #149.

CHAPTER 16

Stacks, deques, and heaps: keep only useful work

These structures are not interchangeable containers for "things I have seen." A stack waits for the most recent unfinished item. A deque can discard items from either end while preserving an order. A heap offers the smallest current priority, not a sorted list. When you get stuck writing one of these problems, name the promise made by the contents, then ask what an incoming item invalidates. The removal rule usually reveals the whole loop.

16.1 A stack of unfinished obligations

In bracket matching, each opener creates an obligation that the next corresponding closer must discharge. Nesting makes the most recent obligation the first to finish, hence a stack. When a closer arrives, it must match the current top; checking "the partner exists somewhere in the stack" ignores nesting.

```
pairs = {")": "(", "]": "[", "}": "{"}
stack = []
for char in text:
    if char in "([{":
        stack.append(char)
    elif char in pairs:
        if not stack or stack.pop() != pairs[char]:
            return False
return not stack
```

Start by writing what a stack frame represents, not by choosing an API call. In decode-string (#25), a frame is `(text_before_bracket, repeat_count)`; on "[" you push the unfinished outer expression and begin a new inner one, and on "]" you combine the inner result with that saved outer expression. Expression parsing (#63) saves pending operators or earlier evaluation contexts in the same way. In each case the stack holds *work suspended by a nested dependency.*

A Min Stack (#24) uses a different frame: each push stores `(value, smallest_so_far)`. The new minimum is the smaller of the value and the previous top's minimum. A pop discards both at once, revealing the exact minimum of the restored prefix; a single global minimum variable would not know the previous minimum after its smallest element is removed.

> **State.** Stack of obligations not yet closed.
>
> **Invariant.** The stack spells the unmatched nesting from oldest to newest; the top is the next required match.
>
> **Update.** An opener pushes an obligation; a closer must discharge the top one; finish only if none remain.

Break test. `"([)]"` has both kinds of opener and closer, but the first closer `")"` cannot discharge the top `"["`. A count of bracket types is not enough.

Source notes: #22, #24, #25, #63.

16.2 Monotonic stack: each pop completes an answer

In Daily Temperatures (#23), each index waits for the first *strictly warmer* future day. Start with this question on a blank page: "What can the new temperature answer for earlier indices?" It answers all waiting days whose temperatures are lower. Put their *indices*, not just temperatures, on a stack so you can compute distance and write into the correct output slot. The stack's temperatures are nonincreasing from bottom to top: if a warmer temperature arrives, it pops every cooler top before joining them.

```
answer = [0] * len(temps)
waiting = []                        # indices; temps fall up the stack
for today, temperature in enumerate(temps):
    while waiting and temps[waiting[-1]] < temperature:
        earlier = waiting.pop()
        answer[earlier] = today - earlier
    waiting.append(today)
return answer
```

Trace `[73, 74, 71, 75]`:

Today	Incoming	Pops (answer written)	Waiting after
0	73	–	[0:73]
1	74	0 (1)	[1:74]
2	71	–	[1:74, 2:71]
3	75	2 (1), 1 (2)	[3:75]

Index 3 still waits at the end, so its initialized answer stays zero. Every index is pushed once and popped at most once; the inner `while` does not make the algorithm quadratic. Change the comparison to `<=` only if an equal value counts as a resolution. "Next greater" and "next greater or equal" are different contracts.

> **State.** Stack of unresolved indices and an answer slot for each.
>
> **Invariant.** Waiting values are monotone; none has yet seen a qualifying item to its right.
>
> **Update.** On a new item, pop and finalize every top it qualifies for; then push its index as a new pending question.

Break test. On `[70, 70]`, `<=` claims the second day is *warmer* than the first. Strictness in the problem statement determines strictness in the pop condition.

Source notes: #23, #62, #103, #140.

16.3 A sentinel forces the remaining work to finish

Some stack items never meet a natural resolving event. In Largest Rectangle in Histogram (#62), a rising tail contains bars whose right boundary is still unknown. Append a virtual height zero to trigger all remaining pops. When popping a bar at index `j`, its right boundary is the current scan index; its left boundary is just after the *new* top, not necessarily after `j - 1`. The pop changes which older bar blocks it.

```
bars = heights + [0]
waiting = []
best = 0
for right, height in enumerate(bars):
    while waiting and bars[waiting[-1]] > height:
        h = bars[waiting.pop()]
        left = waiting[-1] + 1 if waiting else 0
        best = max(best, h * (right - left))
    waiting.append(right)
```

Two policies must be explicit: whether equal heights pop or coexist, and how the remaining stack is drained. A sentinel trades an end-of-loop cleanup block for one ordinary iteration. It is valid only if its value is guaranteed to resolve every relevant pending item; for positive heights, zero does that. An explicit cleanup loop can be clearer when the input may contain zeros or negatives or when a sentinel would change the input contract.

Break test. A strictly increasing histogram `[1, 2, 3]` never triggers a pop by itself. Returning before a flush returns zero; the best area is 4.

Source notes: #23, #62, #63.

16.4 Monotonic deque: age and quality are separate

For maximum in each window of width `k` (#64), old items can become invalid for two independent reasons. An item falls *outside* the window when its index is too small; another item is *dominated* when a newer item is at least as large. A deque stores indices of surviving candidates in decreasing value order. Its front is the current maximum. On each new index, expire the front by age, remove dominated values from the back, append the new index, then read the front after a full window exists.

```
from collections import deque
candidates = deque()
for right, value in enumerate(nums):
    left = right - width + 1
    while candidates and candidates[0] < left:
        candidates.popleft()          # expired by age
    while candidates and nums[candidates[-1]] <= value:
        candidates.pop()              # dominated by newer value
    candidates.append(right)
    if left >= 0:
        result.append(nums[candidates[0]])
```

Why is dropping an older, no-larger value safe? Every future window that still contains that older index also contains the newer one; the newer item is at least as good and survives longer. Draw two arrows on your scratch paper: time expires from the front and quality discards from the back. Trying to implement both with one end of a queue makes the invariant difficult to maintain.

> **State.** Deque of candidate indices and current window edges.
>
> **Invariant.** Indices rise from front to back; their values fall; every index is within the current window.
>
> **Update.** Expire old front indices, remove dominated back indices, append the new item, then read the front.

Break test. With `[3, 1, 2]` and width 2, the first maximum is 3 and the second is 2. Failing to expire index 0 returns 3 for the second window.

Source notes: #64, #110, #138.

16.5 Heap: the top is an appointment, not the whole answer

For Top-K (#19), a min-heap of size `k` keeps the `k` largest items seen so far; its smallest element is the one most easily displaced. This feels backward until you state the invariant: *everything discarded is no larger than a value still represented in the heap.* Push each item, pop the smallest if size exceeds `k`. The root after the scan is the kth largest when the contract guarantees at least `k` items.

```
import heapq
chosen = []
for value in nums:
    heapq.heappush(chosen, value)
    if len(chosen) > k:
        heapq.heappop(chosen)
return chosen[0]                # k > 0 and len(nums) >= k
```

The heap is *not* a sorted list. Only `chosen[0]` is guaranteed to be the smallest. A heap is a good frontier if you repeatedly need one extreme while new candidates arrive: next job time, cheapest unsettled route, earliest interval end, or next item in a merged feed. If all items are present at the start, `heapq.heapify(items)` builds the heap in linear time; repeated pushes cost more. Python's `heapq` is a min-heap, so a max-heap of numbers stores negated values (or uses a suitable priority tuple).

> **State.** Heap of active candidates and an explicit priority key.
>
> **Invariant.** The top is the minimum priority among candidates currently admitted; no promise is made about the remaining list's order.
>
> **Update.** Push a new candidate, evict or expand the top under the problem's policy.

Break test. Call `list(heap)` a sorted answer after several pushes. Only its first element is positioned by the heap invariant.

Source notes: #19, #27, #59, #60, #109.

16.6 K-way merging: a heap of frontiers

Each of `k` sorted input lists has just one possible next item: its current front. Seed the heap with one item from each nonempty list. Pop the smallest, record it, and push only its successor from the same list. The heap never needs to hold the entire input.

```
heap = []
for list_id, items in enumerate(lists):
    if items:
        heapq.heappush(heap, (items[0], list_id, 0))
while heap:
    value, list_id, pos = heapq.heappop(heap)
    result.append(value)
    next_pos = pos + 1
    if next_pos < len(lists[list_id]):
        nxt = lists[list_id][next_pos]
        heapq.heappush(heap, (nxt, list_id, next_pos))
```

The triple includes provenance: without `list_id` and `pos`, you would know the smallest value but not where to get its successor. Merge K Sorted Arrays (#20) uses this literally. A news feed (#46) stores each followee's posts in time order and pops the newest front instead. An inverted index (#150) can merge sorted posting lists the same way. Ask "what is the next item for each source?" before asking "which one is globally next?"

Break test. One of the `k` lists is empty. Seed only nonempty sources; indexing its first element before checking emptiness fails even though the merge itself could handle it.

Source notes: #20, #46, #105, #150.

16.7 Two heaps: encode a dividing line

For a streaming median (#21), split seen values into a low half and a high half. Store low as a max-heap (negated numbers in Python) and high as a min-heap. Require both *order* and *size*: every low value is no greater than every high value; low holds the same number as high or one extra. The median is then either low's top or the average of both tops.

```
heapq.heappush(low, -value)
heapq.heappush(high, -heapq.heappop(low))
if len(high) > len(low):
    heapq.heappush(low, -heapq.heappop(high))
# low has one more value, or both halves have the same size
```

The first move ensures the order invariant (everything removed from low joins high); the second repairs the size invariant. Do not try to reason only about heap sizes while neglecting which side the value belongs on. For a window median, departing items add lazy deletion: each half also needs a way to remember values scheduled for removal until they reach the top. That is a different contract from an append-only stream.

Break test. Feed 10, then 1, then 5. Inserting alternately by size without repairing the cross-heap order can leave 10 on the low side and report the wrong middle value.

Source notes: #21, #55, #93.

16.8 Lazy deletion: one authority, many copies

A heap entry is a snapshot of a priority at insertion time. If priorities later change, keep an authoritative map of the current record and treat heap entries as claims to be checked. At pop, compare against the

authority; stale entries are skipped. Distances in Dijkstra (#42), timestamps in a stock tracker (#140), and generations in a job scheduler (#149) all use this idea.

Lazy deletion trades cheap updates for temporary extra space. Its cost must be stated honestly: if a long-lived object cancels thousands of jobs due far in the future, the dead heap entries can accumulate. A real scheduler may rebuild the heap periodically from live records. In an interview solution, document the possible stale-entry growth rather than claiming that heap space always equals the number of currently live jobs.

> **State.** Priority queue of past proposals plus a map of current truth.
>
> **Invariant.** A heap item is actionable only when its version agrees with the map.
>
> **Update.** Push a replacement without removing the old entry; reject the old one when it reaches the top.

Break test. Update a stock timestamp's price from 10 to 3. The old max-heap record for 10 remains physically present but must never be returned as the current maximum.

Source notes: #42, #94, #140, #149.

CHAPTER 17

Recursive contracts and backtracking

Recursion does not make state disappear. It puts part of the state in each call frame. A good recursive helper answers one precise question, using smaller answers supplied by its children. Backtracking adds a second responsibility: restore any borrowed state before the call returns. At the blank page, draw one frame with its inputs, return value, and allowed shared changes. Then write base case, child calls, combine step, and restoration in that order.

17.1 Decide whether information travels down or up

Tree validation (#35) carries an allowed range *down*: the left child receives an upper bound equal to its parent; the right child receives a lower bound. Tree diameter (#33) passes information *up*: each child returns a height so its parent can compute a path through itself. Lowest common ancestor (#34) passes a found node upward. Ask which direction the information naturally flows before choosing helper parameters.

```
def valid(node, low, high):
    if node is None:
        return True
    if not low < node.val < high:
        return False
    return (valid(node.left, low, node.val)
            and valid(node.right, node.val, high))
```

This helper answers "does *this entire subtree* respect the bounds it inherited?" Checking only the immediate parent and child misses violations farther below. Use explicit bounds instead of overloaded numeric sentinels if the value domain might itself contain extreme values; Python's `float("-inf")` and `float("inf")` work for ordinary numeric keys.

> **State.** A call frame containing its subtree root and any constraints inherited from ancestors.
>
> **Invariant.** The frame's parameters describe precisely the legal answers below this node.
>
> **Update.** Check this node, derive each child's parameters, combine both child results under the helper's return contract.

Break test. Root 10, left child 5, that child's right child 12. Both child–parent comparisons pass, but 12 violates the root's left-subtree upper bound.

Source notes: #33, #34, #35, #75.

17.2 Postorder: return one arm, record the whole path

Imagine a tree path as a coat hanger with two arms joined at a node. A path completed at this node may use both left and right arms; a path sent to its parent may use only one. Write *two* quantities on the scratch paper: the extendable return and the completed candidate. This is the difference between a correct diameter or max-path-sum implementation and one that accidentally branches a path upward.

```
def height(node, best):
    if node is None:
        return 0
    left = height(node.left, best)
    right = height(node.right, best)
    best[0] = max(best[0], left + right)
    return 1 + max(left, right)
```

The code above counts diameter in *edges*: a missing child returns height zero, so `left + right` is the number of edges on a path bending at `node`. For maximum path *sum*, each child offers a gain, clip a negative gain to zero, include the node value in the complete candidate, and initialize the best to a real node value to handle all-negative trees. These are different answers inside the same return/update scaffold.

State. Child return values and a shared best completed answer.

Invariant. After both children return, their contributions are final for this node.

Update. Record the result that ends or bends here; return only a value the parent may legally extend.

Break test. At a node with two nonempty child branches, return `1 + left + right`. The parent tries to attach a third branch, which is no longer a simple path.

Source notes: #33, #35, #75, #148.

17.3 Backpropagation: wait, combine, propagate

Backtracking and backpropagation both revisit earlier frames, but do different work. Backtracking *undoes* a choice so another branch can borrow the same state. Backpropagation *combines* finished contributions into an earlier answer. A parent cannot use a child's subtree size, height, or low-link value until that child has returned. Ask what each child must send back and whether several contributions must be added, minimized, or kept separate.

Sometimes an answer for *every* node also needs information outside its subtree. On an undirected tree, suppose we want the sum of distances from each node to all others. In a first, bottom-up pass, `size[u]` counts nodes in `u`'s subtree, and `inside[u]` sums distances from `u` to those nodes. A finished child `v` adds `size[v]` nodes and `inside[v] + size[v]` distance: each of its nodes is one edge farther from `u`. In a second, top-down pass, shifting the root from `u` to child `v` makes its `size[v]` nodes one step closer and the other `n - size[v]` nodes one step farther. Therefore `total[v] = total[u] + n - 2 * size[v]`.

```
def distance_sums(tree):
    if not tree:
        return []
    n = len(tree)                    # connected undirected tree
    parent = [-1] * n
    parent[0] = 0
    order = []
    pending = [0]
    while pending:
        u = pending.pop()
        order.append(u)              # parent before child
        for v in tree[u]:
            if v != parent[u]:
                parent[v] = u
```

```
                pending.append(v)

    size = [1] * n
    inside = [0] * n
    for u in reversed(order):       # child before parent
        if u != 0:
            p = parent[u]
            size[p] += size[u]
            inside[p] += inside[u] + size[u]

    total = [0] * n
    total[0] = inside[0]
    for u in order[1:]:             # send global context downward
        total[u] = total[parent[u]] + n - 2 * size[u]
    return total
```

On the chain `0-1-2-3`, the upward pass gives subtree sizes `[4, 3, 2, 1]` and `inside[0] = 6`. The downward pass gives totals `[6, 4, 4, 6]`. The stored `order` replaces a recursive call stack: reverse it to collect child results, then traverse it forward to distribute context. This avoids Python's recursion-depth limit; it relies on the stated *tree* precondition, where every non-root node has exactly one parent.

> **State.** A parent-first order, each node's subtree contribution, and the final answer for each possible root.
>
> **Invariant.** On the reverse pass every child contribution is final before its parent is updated; on the forward pass the parent's global answer is final before the child's.
>
> **Update.** Add each finished child to its parent exactly once, then transform the parent's complete answer across each edge.

Break test. Compute `total[v]` using `size[u]` rather than `size[v]`. On `0-1-2-3`, the first move predicts 2 instead of the actual distance sum of 4 at node 1.

Source notes: #3, #33, #116.

When backpropagation means gradients

The same scheduling rule powers reverse-mode differentiation on a directed *acyclic* computation graph. Save the forward values and the local derivative of each operation with respect to each input. Set the output's gradient to one; visit operations in reverse topological order. A node may feed several later operations, so contributions to its gradient must be *added*, not assigned.

```
# inputs[node] holds (argument, evaluated local derivative) pairs.
gradient = [0.0] * len(inputs)
gradient[output] = 1.0
for node in reversed(topo_order):
    for argument, slope in inputs[node]:
        gradient[argument] += gradient[node] * slope
```

For `p = x * y`, `f = p + x`, and `x = 2`, `y = 3`, the backward visit to `f` sends one unit to `p` and one to `x`. The visit to `p` sends another three to `x` and two to `y`: the final gradients are 4 and 2. Sending a contribution as soon as an operation is computed in the forward pass, or overwriting `gradient[x]`, loses one of the paths. Unlike backtracking, nothing is undone; this pass accumulates evidence from all users before propagating it further. This extension beyond the 150 source algorithms has a worked check in section 24.12.

State. Forward values, a topological order, and one gradient accumulator per node.

Invariant. Before a node is processed in reverse order, all of its uses have contributed to its gradient.

Update. For each argument, add the node's complete gradient times the local derivative evaluated on the forward pass.

Break test. Set `gradient[x]` to the contribution from `p = x * y` instead of adding it. The direct `+ x` path is lost, giving 3 instead of 4.

17.4 Backtracking: choose, descend, undo

For subsets (#30), every node of the search tree is an answer; for permutations (#31), only the leaves are. That one sentence tells you where to record output. A path is a shared list representing the current choices. Each loop iteration borrows it: append one choice, recurse, then pop so the next sibling starts from the same parent state. The recursive call must narrow the future choices (next start index for combinations; `used` flags for permutations), or the recursion cannot make progress.

```
def subsets_from(start, path, answer):
    answer.append(path.copy())          # every node is a subset
    for index in range(start, len(nums)):
        path.append(nums[index])        # choose
        subsets_from(index + 1, path, answer)
        path.pop()                      # undo before next sibling
```

Start the solution by choosing the meaning of a frame: "I have chosen exactly `path`; the next choice must be at or after `start`." The invariant explains why `[1,2]` is generated but `[2,1]` is not. With duplicate input values, sort them and skip equal siblings *at the same depth*; skipping an equal value unconditionally would also remove valid subsets that include it twice.

State. Current path, next-choice rule, and independent output snapshots.

Invariant. At entry, the path equals the choices along this branch; siblings see their parent's original path.

Update. Try a legal choice, descend, undo; record at nodes or leaves according to which candidates count as answers.

Break test. On `[1, 2]`, remove `path.pop()`. After exploring `[1, 2]`, the sibling meant to start from `[2]` instead inherits `[1, 2]` and produces invalid results.

Source notes: #30, #31, #73, #113.

17.5 Path marks are loans, not global facts

Word Search (#74) temporarily claims a grid cell so it cannot reappear later in the *same* word path. That claim must be undone when the path retreats; a different candidate path may need the cell. Saving the old letter and restoring it after the neighbor loop makes the loan explicit. A single `seen` set can also work if it is updated on entry and exit. Contrast an island count (#38), where marking land as processed is permanent because the whole component is finished.

```
def search(row, col, index):
    if index == len(word):
        return True
    if not inside(row, col) or board[row][col] != word[index]:
        return False
    letter = board[row][col]
    board[row][col] = "#"            # path-local claim
    found = False
    for nr, nc in neighbors(row, col):
        if search(nr, nc, index + 1):
            found = True
            break
    board[row][col] = letter         # restore even on success
    return found
```

This sketch assumes `"##"` cannot be a letter in the searched word. If the input alphabet can contain it, use a separate path set instead of conflating "claimed" with a valid input character. The condition `index == len(word)` precedes board access because finishing the word succeeds even when the next coordinate is outside the board. The blank-page question is: "What state belongs to one branch, and what state describes work finished forever?"

Break test. Search one failed word, then another on the same board. If old path marks remain, the second search depends on call order even though both queries should be independent.

Source notes: #38, #74, #85, #113.

17.6 Memoization: the key must identify the entire future

If recursion revisits an identical subproblem, save its answer. But "identical" means the same relevant state, not merely the same input index. A recursion over array position and remaining capacity usually needs both in the key; caching by position alone merges states with different futures. A grid path that cannot reuse cells would need a key including the visited configuration, often too large to be worthwhile. Memoization is not automatically appropriate just because a search is recursive.

```
def ways(index, remaining, memo):
    key = (index, remaining)
    if key in memo:
        return memo[key]
    if remaining == 0:
        return 1
    if index == len(choices) or remaining < 0:
        return 0
    answer = (ways(index + 1, remaining, memo)
              + ways(index, remaining - choices[index], memo))
    memo[key] = answer
    return answer
```

The example assumes positive choices, so a negative remaining sum cannot recover. Notice how the two branches have different `index` updates: skipping a choice advances it; reusing a choice does not. That is a policy in the subproblem definition. Fibonacci (#90) is the simplest case: `n` alone identifies the future, and if only the two previous values matter, an iterative two-variable fold needs even less state than a memo.

State. Parameters that completely determine the answer from this point onward, plus a map from such tuples to finished results.

Invariant. A cached value is a final answer for exactly its key, independent of the path that reached it.

Update. Check the key before recursion; store the computed result only after all child answers are combined.

Break test. Cache `ways(index, remaining)` by `index` alone. The same item position with remaining 1 and remaining 5 would be forced to return the same count.

Source notes: #73, #90, #113, #137.

17.7 Pick the search tree before writing the helper

The hardest part of backtracking is often not the recursion; it is deciding what *one level* means. Draw a node as "choices made so far" and write the legal children. Then identify when the node is an answer and what prevents generating the same answer twice.

Problem	One next choice	State that forbids illegal repetition
Subsets	An item after `start`	Next index `i + 1`; record every node
Permutations	Any unused item	`used[i]`; record at path length `n`
Combination sum	A candidate from `start` onward	Same index may recur if reuse is allowed
Word path	An adjacent matching cell	Current-path marks; undo after branch
N-Queens	A column in the next row	Used columns and both diagonal families

For permutations of distinct elements (#31), a frame means "`path` is the partial arrangement; exactly the true entries of `used` are already in it". After choosing index `i`, set `used[i] = True`, recurse, then clear it. The choice loop starts at zero each time because order matters. Subsets use `start` instead because order *must not* matter. Combination Sum (#73) recurses with `i` when the chosen number may be reused, or `i + 1` when it may not. A single index change expresses a different problem contract.

```
def permute(path, used, answer):
    if len(path) == len(nums):
        answer.append(path.copy())
        return
    for i in range(len(nums)):
        if used[i]:
            continue
        used[i] = True
        path.append(nums[i])
        permute(path, used, answer)
        path.pop()
        used[i] = False
```

If the input has duplicates and the output must be unique, sort first, then skip an equal value only when the previous equal value is *not* used in the current branch: `i > 0 and nums[i] == nums[i - 1] and not used[i - 1]`. That rejects duplicate sibling choices but still permits two equal values to appear in one permutation. The analogous subset/combinations skip is `i > start`: equality matters among siblings of a given frame, not across all depths.

Break test. Permute `[1, 1, 2]` with a permanent "value seen" set. It rejects both 1s from the same valid answer. The duplicate rule must discriminate equal *sibling* branches, not ban a value globally.

Source notes: #30, #31, #73, #74, #113.

17.8 Prune before choosing; undo only what was chosen

Pruning is an early proof that an entire branch cannot work. If all combination candidates are positive and sorted, a candidate larger than the remaining target makes all later candidates too large; break the loop. If negative numbers are allowed, that reasoning fails. In N-Queens, place one queen per row and keep sets of used columns, `row - col` diagonals, and `row + col` diagonals. A position touching any of these is illegal *before* you mutate the board. Only legal positions are marked, recurred into, and unmarked.

```
for col in range(n):
    down = row - col
    up = row + col
    if col in columns or down in diag_down or up in diag_up:
        continue                    # reject without changing state
    columns.add(col)
    diag_down.add(down)
    diag_up.add(up)
    place(row + 1)
    diag_up.remove(up)
    diag_down.remove(down)
    columns.remove(col)
```

Look at the order: *guard* → *choose* → *recurse* → *undo*. The same guard can also include a bound on the best possible result, as in branch-and-bound; it must be a genuine upper or lower bound, not a plausible guess. Word Search checks bounds and the next character before marking a cell. After any successful or unsuccessful recursive path, each mark this frame created is removed. That pairs every mutation with exactly one restoration.

> **State.** Partial candidate and compact sets or counters describing its conflicts.
>
> **Invariant.** Every item on the current path is legal relative to its ancestors; siblings inherit the original parent state.
>
> **Update.** Reject a provably impossible choice before mutation; otherwise choose, recurse, and undo the identical changes in reverse order.

Break test. In N-Queens, forget to clear a diagonal on return. Later columns in the same row are rejected because of a queen that no longer exists.

Source notes: #73, #74, #85, #113.

17.9 Python recursion has a real depth limit

Recursive DFS and tree postorder can be elegant on a small tree, but a path of a thousand or more nodes risks Python's default recursion limit. Use an explicit stack when depth may scale with input size. An iterative traversal needs its own frame data: `(node, phase)` if postorder work must run after children, or just `node` if processing on entry is enough. Do not increase the recursion limit by guesswork without considering the actual depth and platform stack.

```
stack = [start]
seen = {start}
while stack:
    node = stack.pop()
    process(node)
    for nxt in graph[node]:
        if nxt not in seen:
            seen.add(nxt)
            stack.append(nxt)
```

This short stack-and-`seen` version answers *reachability*, but need not reproduce a recursive DFS's order. Pushing neighbors in reverse changes which sibling is popped first, yet marking all siblings at push time can block a deeper branch from entering one of them. For exact entry and exit order, use a stack of suspended `(node, next_neighbor)` frames as in section 18.2. An explicit stack makes both the pending work and the return event visible, which is useful when debugging recursion.

Break test. Build a chain of 2,000 vertices and run a recursive graph DFS on it. The mathematics is linear but the Python call stack may stop the program before the traversal finishes.

Source notes: #37, #38, #81, #116.

CHAPTER 18

Graph code is frontier code

Graphs make implementation mistakes visible because the same node can be approached by many routes. Before choosing an algorithm, specify what an edge means, what a "new" node means, and what event makes its answer final. A queue settles uniform-cost hops; a heap offers minimum known cost; a stack follows one path; union-find maintains groups under merges. The choice of container is downstream of the question, not the starting point of the code.

18.1 First write the node and neighbor contract

An adjacency list for `n` integer nodes can start as `graph = [[] for _ in range(n)]`. A directed edge `u -> v` adds `v` to `graph[u]`; an undirected edge also adds `u` to `graph[v]`. A weighted edge stores `(neighbor, weight)` instead of just the neighbor. A grid need not be materialized as a graph: its implicit neighbors are the in-bounds cells allowed by the problem's movement rule.

Write the validity tests in this order: coordinate inside bounds, cell traversable, cell not already claimed. Only then compute a new distance or put the cell in a frontier. For a graph that uses arbitrary keys, a dictionary of lists can replace indexed lists; define whether a missing key means no outgoing neighbors or invalid input. Do not assume a sink node appears as a map key.

```
def neighbors(row, col, rows, cols):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = row + dr, col + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc
```

This helper handles geometry only; whether a neighbor is a wall or has been visited belongs to the traversal's admission policy. Keeping those responsibilities separate lets the same geometry support flood fill, word search, and multi-source BFS without confusing their very different meanings of "visited".

> **State.** A representation of nodes, a single edge direction policy, and a neighbor function.
>
> **Invariant.** Every yielded neighbor is a valid node, not necessarily an admissible next step.
>
> **Update.** Build or generate edges, then let the traversal decide whether each proposed neighbor joins the frontier.

Break test. Course pair `[course, prerequisite]` means an edge from prerequisite to course. Reverse it in Kahn's algorithm and the resulting order may list a course before its prerequisite.

Source notes: #37, #38, #39, #40, #80.

18.2 DFS: enter, resume, exit

DFS is not merely "use a stack instead of a queue." One frame remembers which neighbor to try *next*. On entry, mark the node and record preorder if needed. Descend into an unentered neighbor, leaving the frame suspended. Only after all its neighbors are considered can the node exit; that is the moment

for postorder work such as combining subtree answers. A graph with several components needs an outer loop that starts a new walk at each still-unentered node.

```
def dfs_orders(graph, start):
    seen = {start}
    preorder = [start]
    postorder = []
    stack = [(start, 0)]             # node, next neighbor index
    while stack:
        node, index = stack[-1]
        if index == len(graph[node]):
            postorder.append(node)   # EXIT: children finished
            stack.pop()
            continue
        nxt = graph[node][index]
        stack[-1] = (node, index + 1) # RESUME here after child
        if nxt not in seen:
            seen.add(nxt)            # ENTER: permanent claim
            preorder.append(nxt)
            stack.append((nxt, 0))
    return preorder, postorder
```

For edges `0-1, 0-4, 0-2, 1-3, 3-2` in that neighbor order, starting at 0 yields preorder `[0, 1, 3, 2, 4]` and postorder `[2, 3, 1, 4, 0]`. Node 0 does *not* reserve node 2 when it first sees its neighbor list: the branch through 1 and 3 enters 2 before 0 resumes. A simpler stack that pushes all of 0's neighbors *in reverse order* and marks them immediately can instead visit `[0, 1, 3, 4, 2]`. That is fine for reachability, but wrong when the result specifies recursive neighbor order. Mark-on-pop with a stale-entry check can preserve *preorder*; suspended frames also preserve the correct *exit* event.

For an undirected graph, skip the edge to the parent if an edge-specific calculation needs it; a plain `seen` check already prevents infinite recursion. For directed cycle detection, `seen` is not enough: distinguish frames still on the stack from those already exited, as in section 18.8. For large graphs, frames avoid Python's recursion limit without changing DFS semantics.

> **State.** One frame per active node, its next-neighbor cursor, and permanent entry marks.
>
> **Invariant.** The stack is exactly the unfinished path; a node exits only after every neighbor has been considered.
>
> **Update.** Mark at entry, advance the suspended cursor, descend when new, and process at exit when done.

Break test. On the graph above, mark 2 when 0 pushes it. Then 3 cannot descend into 2; reversing the push order alone does not restore the requested DFS visit order.

Source notes: #37, #38, #81, #82.

18.3 BFS: an arrival gets a permanent layer

BFS behaves like a waiting room with numbered tickets: items already waiting at distance `d` leave before the distance `d + 1` items they admit. Mark a node at the moment it enters the queue and assign it `distance[parent] + 1`; a later route cannot shorten that number when every edge has the same cost. A `parent` map remembers the edge that first found it.

```
from collections import deque
queue = deque([source])
distance = {source: 0}
parent = {source: None}
while queue:
    node = queue.popleft()
    if node == target:
        break                       # its shortest distance is known
    for nxt in graph[node]:
        if nxt not in distance:
            distance[nxt] = distance[node] + 1
            parent[nxt] = node
            queue.append(nxt)
```

For edges `0-1, 0-5, 1-2, 5-4, 2-4`, the first two layers are `[0]` and `[1, 5]`; from 5, node 4 is claimed at distance 2 before the longer route through 1 and 2 gets there. Follow `parent[4] = 5` and `parent[5] = 0` backward, then reverse to get `[0, 5, 4]`. Reconstruct only after checking that the target appears in `distance`, or following an absent parent will fail.

For bipartite checking (#114), replace distance with one of two colors. On discovery, give a neighbor the opposite color; an edge to an already colored node of the *same* color proves failure. Start a new BFS for each still-uncolored component. Again, the color is assigned at enqueue, because two predecessors must not make conflicting first claims.

> **State.** FIFO queue of discovered but unprocessed nodes; distance and optionally parent for each discovered node.
>
> **Invariant.** Discovery is the first, shortest arrival; queue entries have nondecreasing distance.
>
> **Update.** Pop an item, handle it, admit each undiscovered neighbor with distance one greater, then enqueue it.

Break test. Mark a node when it leaves the queue instead of when it enters. Two nodes in the same layer can both enqueue it; the queue no longer has one ticket per node.

Source notes: #32, #39, #79, #114.

18.4 BFS levels: freeze the current wave

When the output is *grouped by distance*, a distance on every queue entry is unnecessary. At the start of a round, `width = len(queue)` counts exactly the current layer. Pop precisely that many nodes. Their newly admitted neighbors wait for the next round; `len(levels)` before the round is their parents' distance from the start in edges.

```
from collections import deque

def bfs_levels(graph, start):
    queue = deque([start])
    seen = {start}
    levels = []
    while queue:
        width = len(queue)           # freeze this layer
        level = []
        for _ in range(width):
            node = queue.popleft()
            level.append(node)
            for nxt in graph[node]:
                if nxt not in seen:
                    seen.add(nxt)
                    queue.append(nxt) # next layer, not this one
        levels.append(level)
    return levels
```

On the diamond `A-B, A-C, B-D, C-D` from section 10.4, the snapshots are 1, 2, 1, producing `[[A], [B, C], [D]]`. While B is processed, the live queue holds C *and* D; it is the saved width of 2, not the changing queue length, that keeps D out of the current output level. If each node needs its own distance or parent, use the maps in section 18.3 instead; if the request is only layer order or elapsed rounds, the snapshot is the simpler clock.

> **State.** FIFO queue of next work, a frozen width, and completed levels.
>
> **Invariant.** Each round pops exactly the nodes at one distance; neighbors discovered during that round are not processed until the next.
>
> **Update.** Save width, pop that many times, claim new neighbors at enqueue, then append the completed level.

Break test. On the diamond, use `while queue` inside the layer loop. D is enqueued while processing B and then consumed in the same round, so the distance-two layer disappears.

Source notes: #32, #39, #78.

18.5 BFS with more than one shortest parent

"Mark on enqueue" prevents duplicate *processing*; it need not discard all later information. If the answer is a *count* of shortest paths, a second parent at the same distance contributes another count even though the neighbor must not be queued again. Assign the first distance and count on discovery; add another contribution only when the newly proposed distance equals it.

```
distance = {source: 0}
ways = {source: 1}
queue = deque([source])
while queue:
    node = queue.popleft()
    for nxt in graph[node]:
        candidate = distance[node] + 1
        if nxt not in distance:
            distance[nxt] = candidate
            ways[nxt] = ways[node]
            queue.append(nxt)
        elif distance[nxt] == candidate:
            ways[nxt] += ways[node]   # evidence, not another visit
```

In the diamond, B first gives D one path at distance 2; C then gives it a second, also at distance 2. A `seen` check that simply skips C's edge returns 1 instead of 2. A longer arrival adds nothing. For one shortest path, keep a single first parent instead of `ways`; for *all* shortest paths, save all equal-distance parents and reconstruct after the BFS. Do not stop on first *discovery* if the answer needs contributions from the rest of that layer. If parallel edges are not distinct paths in the problem contract, deduplicate adjacency first.

> **State.** Shortest distance and number of shortest routes to each discovered node.
>
> **Invariant.** All contributions from previous layers are counted before a node is expanded; a node is still enqueued only once.
>
> **Update.** Claim a first arrival, add equal-distance contributions, ignore longer ones.

Break test. Stop the count when B first enqueues D; the equally short route A–C–D has not yet contributed.

Source notes: #39, #79.

18.6 BFS configurations: visit the whole future

A position alone may not describe a search state. Suppose a route may break *one* wall. Reaching cell X with the break already spent and reaching X with the break still available have different futures. The first can arrive in fewer moves yet be unable to pass the wall after X. The visited key must include both position and the resource used; distance is frontier metadata, not part of the key, because BFS already settles the earliest arrival for each complete state.

```
# Nonempty rectangular 0/1 grid; start (0, 0) is open.
queue = deque([(0, 0, 0, 0)])  # row, col, breaks used, steps
seen = {(0, 0, 0)}
while queue:
    row, col, used, steps = queue.popleft()
    if (row, col) == goal:
        return steps
    for nr, nc in neighbors(row, col, rows, cols):
        next_used = used + grid[nr][nc]
        state = (nr, nc, next_used)
        if next_used <= 1 and state not in seen:
            seen.add(state)
            queue.append((nr, nc, next_used, steps + 1))
return -1
```

Think of an implicit graph with vertices `(row, col, used)`. At X, a distance-one route may have `used = 1`; a distance-three detour may have `used = 0`. Both must remain eligible if the only onward edge breaks a wall. With a *fixed* allowance of k breaks, there are at most $V(k+1)$ states, where V is the number of cells. With collected keys, the third coordinate can be a bitmask (section 19.7); the potential state count then grows to $V2^k$. This is still ordinary BFS only when every step has the same cost; otherwise choose a cost-aware frontier.

> **State.** Queue entries with position, resource state, and distance; a seen set keyed by position plus resource.
>
> **Invariant.** Each complete state is claimed at its shortest distance; states at the same position may have different possible next moves.
>
> **Update.** Check bounds, compute the new resource state, reject illegal or previously claimed states, then enqueue.

Break test. Mark only X's coordinates when the fast route spends the break. The slower route carrying an unused break is then discarded, even if it is the only route that can continue past X.

Source notes: #39, #74, #79.

18.7 Multi-source BFS: start all waves at once

If the question is "distance to the nearest source", do not run one BFS per source. Seed the queue with *all* initial sources and mark each at distance zero. One shared FIFO queue interleaves the waves in increasing distance. Each cell's first wave is its nearest source. For Rotting Oranges (#78), count fresh cells first, and decrease the count when one is marked rotten and queued; the minute of that transition is its distance.

```
queue = deque()                    # put every initial source here
fresh = count_fresh(grid)
minutes = 0
for row, col in initial_sources(grid):
    queue.append((row, col, 0))
while queue:
    row, col, time = queue.popleft()
    for nr, nc in neighbors(row, col, rows, cols):
        if grid[nr][nc] == FRESH:
            grid[nr][nc] = ROTTEN   # mark at discovery
            fresh -= 1
            minutes = max(minutes, time + 1)
            queue.append((nr, nc, time + 1))
return minutes if fresh == 0 else -1
```

The conceptual source is a virtual node connected to every initial source by a zero-cost starting edge. If there are no fresh cells, the answer is zero even if no source exists; if fresh cells remain after the queue empties, some are unreachable. Counting every layer blindly can add a phantom final minute when the last processed layer infects nobody. Tracking the minute of actual infection avoids that ambiguity.

Break test. One rotten orange with no fresh neighbors takes zero minutes, not one. A loop that increments time for the final empty expansion produces the wrong answer.

Source notes: #32, #78, #79.

18.8 DFS colors: separate “on my path” from “done”

For directed cycles, plain `seen` is too coarse. White nodes have never been entered. Gray nodes are on the current recursion path. Black nodes and their descendants are finished. An edge to gray closes a cycle along the current path; an edge to black is a cross- or forward-edge into work already completed and proves nothing about a cycle.

```
def has_cycle_from(node, graph, color):
    color[node] = GRAY
    for nxt in graph[node]:
        if color[nxt] == GRAY:
            return True
        if color[nxt] == WHITE and has_cycle_from(nxt, graph, color):
            return True
    color[node] = BLACK
    return False
```

Picture the gray nodes as a rope unwinding behind the search. A cycle is an edge that hooks onto the rope. The diamond `0 -> 1 -> 3` and `0 -> 2 -> 3` does not have a cycle: by the time `2 -> 3` is examined, 3 is black, not on the rope. Marking black only after every child is explored is the critical condition placement.

If the graph may be a deep chain, choose iterative DFS with `(node, enter_or_exit)` events or use Kahn’s algorithm instead of risking Python’s recursion limit. Both detect cycles, but their state contracts are different.

For DFS topological order (#82), append a node *after* every child finishes and the node turns black, then reverse the completion list. The append site matters: an edge `u -> v` requires `v` to finish before `u` can be placed into reverse postorder. A gray back edge still invalidates the order by revealing a cycle.

Break test. In the diamond, a single `visited` bit reports a cycle when 3 is encountered for the second time. A gray test correctly ignores it.

Source notes: #37, #81, #82, #115, #116.

18.9 Low-link: can a subtree escape above its parent?

In an undirected graph, a bridge is an edge whose removal strands a subtree. DFS stamps each node's discovery time `disc[u]`. It also computes `low[u]`, the earliest discovery time reachable from `u`'s subtree by tree edges and at most one back edge. After exploring a child `v`, if `low[v] > disc[u]`, nothing below `v` can reach `u` or any ancestor except through the tree edge `u-v`; that edge is a bridge.

```
def bridges_in(graph):
    disc = [-1] * len(graph)
    low = [0] * len(graph)
    bridges = []
    clock = 0

    def visit(u, parent):
        nonlocal clock
        disc[u] = low[u] = clock
        clock += 1
        for v in graph[u]:
            if v == parent:
                continue
            if disc[v] == -1:
                visit(v, u)
                low[u] = min(low[u], low[v])
                if low[v] > disc[u]:
                    bridges.append((u, v))
            else:
                low[u] = min(low[u], disc[v])

    for start in range(len(graph)):
        if disc[start] == -1:
            visit(start, -1)
    return bridges
```

The essential placement is *after* returning from a child: its `low` is not final until its descendants and back edges have been explored. The condition is strict `>`, because a route from the child back to `u` itself forms a cycle that saves the edge. In the source note #116, parallel edges are excluded; if parallel edges are allowed, skip only the particular parent *edge id*, not every edge to the parent. The recursive version also needs an explicit stack on graphs whose depth exceeds Python's recursion limit.

> **State.** Discovery timestamps, low-link values, and the current DFS parent edge.
>
> **Invariant.** When a child returns, its low value is the earliest ancestor reachable without using its parent tree edge.
>
> **Update.** Combine the child's low with the parent, then test the child's low against the parent's discovery time.

Break test. For a triangle, each edge has an alternate route. Using `>=` instead of `>` reports a tree edge whose child can reach its parent by a back edge as a bridge.

Source notes: #81, #115, #116.

18.10 Topological order: a count of unfinished prerequisites

An indegree is a debt counter: `indegree[v]` equals the number of incoming prerequisites not yet removed. Build all edges and counts first. Seed a queue with every node owing nothing. Pop one, emit it, decrement the debts of its dependents, and enqueue a dependent precisely when its debt falls to zero. If the emitted order has fewer than `n` nodes, the remaining debts cannot be paid because some cycle blocks them.

```
ready = deque(v for v in range(n) if indegree[v] == 0)
order = []
while ready:
    node = ready.popleft()
    order.append(node)
    for nxt in graph[node]:
        indegree[nxt] -= 1
        if indegree[nxt] == 0:
            ready.append(nxt)
return order if len(order) == n else []
```

Unlike BFS visited checks, the admission test here is *indegree becomes zero*; a node with three prerequisites should not be queued after the first is removed. Duplicate edges require consistent counting and decrementing, or deduplication at construction. The order need not be unique: if several nodes are ready, FIFO selects one valid order; a heap can choose lexicographically smallest if requested.

State. Indegree of each node, queue of all debt-free nodes, and emitted order.

Invariant. Every queued node has no unfinished prerequisites; each emitted node appears once.

Update. Remove one node and one debt per outgoing edge; enqueue only new zeros.

Break test. Build `A -> C` and `B -> C`. If C is queued after A finishes, C can appear before B, violating its other prerequisite.

Source notes: #40, #82, #83.

18.11 Union-find: the root is the group identity

When edges arrive and the question is whether two endpoints are already connected, a traversal can be too expensive to repeat. Union-find stores each component as a parent-pointer tree. The root points to itself. `find(x)` climbs to that root and compresses the path; `union(a, b)` joins *roots*, not the original arguments, and returns whether a real merge occurred. A component count decreases only on a successful merge.

```
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path halving
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    if size[ra] < size[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    size[ra] += size[rb]
    return True
```

Initialize `parent = list(range(n))` and `size = [1] * n`. To track count, start at `n` and decrement only when `union` returns true. An edge joining nodes already under one root is redundant; in Kruskal's algorithm it would form a cycle, so skip it. Union-find handles merging components and connectivity queries, but not splitting or finding a shortest path within a component.

> **State.** Parent pointers and root sizes; optional component count.
>
> **Invariant.** Two nodes are connected exactly when their roots are equal.
>
> **Update.** Find both roots; if different, attach the smaller root below the larger and update only root metadata.

Break test. The parents of 3 and 1 can be different even while their *roots* are equal. Comparing immediate parents can mistake a cycle edge for a merge.

Source notes: #41, #84, #115, #118.

18.12 Dijkstra: propose, relax, reject stale routes

With nonnegative edge weights, the heap presents the cheapest known *proposal*. The distance map is the best known cost. Pop a proposal; if it is no longer equal to the recorded best, skip it. Otherwise, try each outgoing edge. "Relax" means replace a neighbor's known cost *only if* the new route is shorter, then push that improved proposal. The first nonstale pop for a node settles its shortest distance because every future route extends a cost at least that large by a nonnegative weight.

```
dist = [float("inf")] * n
dist[source] = 0
heap = [(0, source)]
while heap:
    cost, node = heapq.heappop(heap)
    if cost != dist[node]:
        continue
    for nxt, weight in graph[node]:
        candidate = cost + weight
        if candidate < dist[nxt]:
            dist[nxt] = candidate
            heapq.heappush(heap, (candidate, nxt))
```

Trace a three-edge detour: `1 -> 2 (4)`, `1 -> 3 (1)`, `3 -> 2 (2)`. After expanding 1, the heap holds `(1, 3)` and `(4, 2)`. Expanding 3 pushes `(3, 2)` and updates `dist[2]` to 3. The old `(4, 2)` will eventually be popped and rejected. Marking node 2 permanently when it was *pushed* at 4 would have forbidden its improvement to 3.

If edges can be negative, this settling proof fails. Bellman–Ford (#117) repeatedly relaxes every edge (up to `n - 1` passes for paths without repeated vertices) and performs another pass to detect a reachable negative cycle. That is not a `heapq` variant with a different comparison: it has a different reason for termination.

State. Best distances and a heap of proposed distances.

Invariant. Every best distance is an achieved route length; a proposal may be stale. With nonnegative weights, a nonstale minimum proposal can be safely expanded.

Update. Pop, reject if stale, relax outgoing edges and push only strict improvements.

Break test. Return the first cost on push for node 2 in the detour above: it is 4, although the shortest route is 3.

Source notes: #42, #79, #117.

18.13 MST: what has become connected?

Kruskal (#118) and Prim (#119) both build a minimum spanning tree, but they represent the frontier differently. Kruskal sorts all edges and asks whether an edge joins two previously disconnected components; union-find answers that question. Prim holds edges leaving the growing tree in a heap and asks whether the edge's destination is already inside. In either case, add a weight only when the edge creates a new connection. The number of accepted edges for a connected `n`-node tree is `n - 1`.

```
# Kruskal's decision; edges are sorted by weight.
for weight, u, v in edges:
    if union(u, v):
        total += weight
        chosen += 1
        if chosen == n - 1:
            break
```

If the input can be disconnected, this code builds a minimum *spanning forest*; a problem asking for a single tree should explicitly check `chosen == n - 1`. Choose Kruskal when edges are already listed and connectivity updates are natural. Choose Prim when adjacency lists and a growing local frontier feel simpler. Neither is a shortest-path algorithm: minimizing the sum of a spanning tree does not minimize the distance from a source to every vertex.

Break test. Take a sorted edge whose endpoints already share a root. Adding its weight without adding a new vertex creates a cycle and overstates the tree's weight.

Source notes: #41, #118, #119.

CHAPTER 19

Data shape decides the structure

When the algorithm is known, its data representation may still decide whether the code is short and correct or tangled and slow. Ask what operations the problem performs *repeatedly*: exact lookup, ordered predecessor, removal from both ends, minimum priority, prefix search, range aggregate, or approximate membership. Choose a structure whose *invariant* makes those operations cheap. The following is a decision guide, not an invitation to use the most sophisticated structure available.

19.1 Choose by operation, not by the noun in the story

"Store the numbers" could mean a list, set, map of counts, heap, bitset, or sorted array. Translate the request into verbs:

Recurring operation	First structure to test	Change your mind if...
Exact membership	set	Counts or associated values are needed: map
Lookup value by key	dict	Keys are tiny dense integers: list
Oldest/newest removal	deque	Arbitrary removals are required: linked nodes + map
Current smallest priority	heap	Fast arbitrary decrease-key is essential
Sorted predecessor/range	sorted list or tree	Queries interleave with many inserts
Prefix of a word	trie	Only whole-word equality matters: set or dict
Static range sum	prefix sums	Updates arrive: Fenwick or segment tree
Connectivity after merges	union-find	Edges are removed or paths are requested
Small bounded universe	bitset	Universe is huge or unknown: set
Huge set, cheap false yes	Bloom filter	False positives are forbidden: exact set

Each choice is a bundle of promises and costs. A Python dict is expected constant time for hashable keys, not a sorted map; adversarial collisions can degrade it. A binary heap gives constant-time access to its minimum but no fast arbitrary lookup without another map. A doubly linked list can splice a *known node* in constant time but finding that node costs linear time unless a map points to it. Start with the simplest structure whose guarantees cover the required verbs. Adding a second structure is justified when it satisfies a distinct verb, as in the LRU cache.

State. The required operations, acceptable error, key universe, and update pattern.

Invariant. The chosen structure's maintained order or index makes the dominant operation cheap.

Update. For each operation, update *all* representations of the same underlying fact.

Break test. Use a set to count how many earlier prefixes equal `total - k`. Membership tells you whether a match exists, but cannot tell you whether two different starts both match.

Source notes: #1, #2, #19, #43, #44, #107.

19.2 When a hash map is the right move

A hash map is natural when a current item needs a fact about an arbitrary earlier key: complement to index (#1), prefix sum to count (#2), word signature to group (#5), key to linked-list node (#44),

term to posting list (#150). The key should express the *equality relation* you care about. For anagrams, all words with the same 26 lowercase-letter counts share one immutable tuple key. For graph states, a pair `(row, col)` or `(node, mask)` can be a key. A mutable list cannot be a dict key; make a tuple or another stable representation.

Choose the stored *value* by the query: boolean existence needs a set, multiplicity needs a count, longest range needs an earliest index, reconstruction needs a parent or payload, grouping needs a list. State when entries are written. In Two Sum, current should not match itself, so query before insertion. In a cache, access must change recency, so successful `get` mutates the linked list even though it does not change the key's stored value.

Use a list instead when keys are dense small integers: `distance = [-1] * n` offers direct indexing with less object overhead than a dict and an explicit unseen marker. Use a bitset when keys lie in a bounded dense universe and you only need boolean membership. A hash map is not the default solution to every bookkeeping problem just because membership is easy to write.

Break test. Group anagrams with a key made from a mutable character-count list. Python rejects it as unhashable; changing the list after insertion would also invalidate its identity as a key. Convert counts to a tuple first.

Source notes: #1, #2, #5, #43, #44, #150.

19.3 Strings: choose the unit and the query

A Python string is both a sequence and a hashable key, but different questions require different state. For whole-word equality use a set or dict; for one common-prefix comparison scan columns (#121), but for repeated prefix queries share paths in a trie. For anagrams of a fixed width keep character counts; for a shortest covering substring keep deficits and shrink a variable window (#13). For one exact pattern use KMP when worst-case time matters; rolling hash is useful when many windows or lengths need fingerprints, but an exact answer still requires verification. Palindromes use yet another frontier, expanding from both character and gap centers (#87). Decide whether "character" means a code point, byte, or user-visible grapheme *before* choosing indices and counts.

Fixed-width windows keep a signed count delta

An anagram window is exactly as wide as its pattern. Let `need[ch]` be the copies still missing, negative for surplus, and `missing` the sum of its positive entries. Only pattern characters need entries: an unrelated character still occupies a window position but cannot fill a deficit. Add the new right character, evict the one just outside the width, *then* test a full window. After eviction, the deficits describe exactly the current window. On entry test the old deficit before decrementing; on exit test the new deficit after incrementing.

```
from collections import Counter

def anagram_starts(text, pattern):
    width = len(pattern)
    if width == 0 or width > len(text):
        return []                          # no empty-pattern matches
    need = Counter(pattern)
    missing = width
    starts = []
    for right, ch in enumerate(text):
        if ch in need:
            if need[ch] > 0:
                missing -= 1
            need[ch] -= 1
        if right >= width:
            old = text[right - width]
            if old in need:
                need[old] += 1
                if need[old] > 0:
                    missing += 1
        if right + 1 >= width and missing == 0:
            starts.append(right - width + 1)
    return starts
```

Trace `text = "BAAB"`, `pattern = "AB"`: after reading `"BA"`, `missing = 0`, so record start 0. The next A is surplus (`need["A"] = -1`), but evicting B makes `need["B"] = 1`; `"AA"` is *not* a match. Adding B and evicting the older A restores coverage at start 2. The answer is `[0, 2]`. The width makes zero missing sufficient; a *minimum* covering window instead shrinks while zero missing, because its width is not fixed.

Reuse a prefix, or verify a hash candidate

For KMP, `pi[i]` is the length of the longest *proper* prefix of `pattern[:i+1]` that is also its suffix. A mismatch may require several fallbacks, so use `while`, not `if`; neither table construction nor search rewinds its scanned position. During search, `matched` is the length of a pattern prefix equal to a suffix of the text read so far. After a complete match, fall back once more to retain overlaps.

```
def prefix_lengths(pattern):
    pi = [0] * len(pattern)
    matched = 0
    for i in range(1, len(pattern)):
        while matched and pattern[i] != pattern[matched]:
            matched = pi[matched - 1]
        if pattern[i] == pattern[matched]:
            matched += 1
        pi[i] = matched
    return pi

def kmp_starts(text, pattern):
    if not pattern:
        return []                           # no empty-pattern matches
    pi = prefix_lengths(pattern)
    starts, matched = [], 0
    for i, ch in enumerate(text):
```

```
        while matched and ch != pattern[matched]:
            matched = pi[matched - 1]
        if ch == pattern[matched]:
            matched += 1
        if matched == len(pattern):
            starts.append(i - matched + 1)
            matched = pi[matched - 1]       # keep the overlap
    return starts
```

For `pattern = "abab"`, the table is `[0, 0, 1, 2]`. In `"ababab"`, the hit at 0 leaves two matched characters; the final `"ab"` yields a second hit at 2. KMP takes $O(n + m)$ worst-case time for text length n and pattern length m. Rolling hash (#120) instead keeps a numeric summary of the current width-m window. With `high = pow(base, width - 1, modulus)`, its inner loop is:

```
if window_hash == pattern_hash and text[left:left + width] == pattern:
    starts.append(left)                    # confirm an exact match
if left + width < len(text):
    window_hash = ((window_hash - ord(text[left]) * high) *
                   base + ord(text[left + width])) % modulus
```

Compute both initial hashes with the same base and modulus; use a positive width. Equal hashes are only candidates: confirming every hit can cost $O(nm)$ if collisions or true matches are frequent. Prefer KMP for guaranteed single-pattern search, rolling summaries for many substring candidates.

Break test. With base 256 and modulus 7, `"ab"` and `"ai"` have the same rolling hash: their last code points differ by 7. Returning a match without comparing the underlying substrings gives a false positive.

Trie paths and token boundaries

A trie (#43) stores shared prefixes in child maps. The node reached after a prefix is the state for the *next* character; a missing child fails immediately. Test the node's word-end flag only after consuming the whole query:

```
def walk(root, prefix):
    node = root
    for ch in prefix:
        node = node.children.get(ch)
        if node is None:
            return None
    return node

node = walk(root, query)
is_prefix = node is not None
is_word = is_prefix and node.is_end
```

If the trie holds `"art"` and `"artist"`, the node for `"arti"` exists but is not a word; the node for `"art"` is a word *and* has children. In multiword search (#85), carry that node alongside the current position; do not delete its children when reporting the shorter word. A set is simpler for whole-word lookups with no prefix queries.

Tokenization needs a different state: whether the next symbol is data even if it looks like a delimiter. For a format in which backslash escapes the next code point, including a comma or another backslash, retain empty fields and reject an unfinished escape. Completed fields are in `fields`; `part` holds just the current field:

```
def split_escaped(text):
    fields, part = [], []
    escaped = False
    for ch in text:
        if escaped:
            part.append(ch)
            escaped = False
        elif ch == "\\":
            escaped = True
        elif ch == ",":
            fields.append("".join(part))
            part = []
        else:
            part.append(ch)
    if escaped:
        raise ValueError("dangling escape")
    fields.append("".join(part))
    return fields
```

Thus `r"a\,b„c"` gives `["a,b", "", "c"]`. For arbitrary string lists, a length prefix (#88) is often simpler than designing an escape grammar: read digits to the length separator, then take *exactly* that many symbols. Plain joining cannot distinguish `[]` from `[""]` or a separator inside a value.

Python `str` indexes Unicode code points, not UTF-8 bytes or user-visible graphemes. An accented letter can be one composed code point or an `"e"` plus a combining accent:

```
import unicodedata

composed = "\u00e9"             # one precomposed code point
decomposed = "e\u0301"          # e + combining accent
assert (len(composed), len(decomposed)) == (1, 2)
assert (len(composed.encode("utf-8")),
        len(decomposed.encode("utf-8"))) == (2, 3)
assert unicodedata.normalize("NFC", decomposed) == composed
raw = decomposed + "x"
assert (raw.index("x"),
        unicodedata.normalize("NFC", raw).index("x")) == (2, 1)
```

The `"x"` offset moves from 2 in raw input to 1 after normalization. Normalize both sides only if the contract treats canonically equivalent text as equal. Reporting a position in the original input requires a mapping from normalized offsets. Byte protocols count encoded bytes and slice bytes before decoding. A 26-slot count requires lowercase ASCII; a dict handles arbitrary code points, while grapheme clusters require separate segmentation.

Break test. Use normalized offset 1 to slice `raw` at `"x"`. It points to the combining accent, not `"x"`; the original code-point offset is 2.

> **State.** A named string unit and the summary the query needs: window deficits, matched prefix, trie node, or token cursor.
>
> **Invariant.** Each summary describes exactly its current span; a parser cursor points to the first unconsumed symbol.
>
> **Update.** Adjust counts at both window edges, reuse a surviving prefix on mismatch, and consume an escaped symbol as data.

Source notes: #5, #12, #13, #43, #85, #86, #87, #88, #120, #121.

19.4 Parsing: a cursor is part of the state

In a parser, "where am I in the input?" is as important as the value being computed. Make the cursor explicit. A recursive parser can return `(value, next_index)`; a class can retain a cursor field for a sequence of calls; an iterative parser can keep `index` in its main loop and store nested contexts on a stack. Do not let two helpers each silently advance the same cursor without a contract, or one consumes the other's character.

In a basic calculator (#63), multi-digit numbers require a loop that consumes *all* consecutive digits before applying an operator. A minus sign may be unary or binary depending on what preceded it. In decode-string (#25), the count before "[" may have multiple digits; on "]" the stack frame supplies the saved outer text and count. Parentheses, brackets, and length-prefix encodings all create an "inside" and "outside" with different ownership.

```
number = 0
while index < len(text) and text[index].isdigit():
    number = number * 10 + int(text[index])
    index += 1                  # first unconsumed character
```

Break test. Parse the repeat count in `"12[a]"` one character at a time without accumulating digits. Repeating `"a"` twice instead of twelve times is a cursor/number-state error, not an issue with the stack.

Source notes: #25, #36, #49, #63, #88, #122.

19.5 Grids: coordinates, neighbors, and marking policy

A grid is a graph with an implicit geometric neighbor rule. First decide whether diagonals count: islands usually use four directions, shortest path in a binary matrix uses eight. Second decide whether the algorithm may change the grid. If yes, sinking visited land avoids a separate `seen` set; if no, store coordinates as tuples in a set or a compact boolean grid. Third decide whether visits are permanent (flood fill) or path-local (word search). The same coordinate loop has three different state contracts.

```
for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
    nr, nc = row + dr, col + dc
    if not (0 <= nr < rows and 0 <= nc < cols):
        continue
    if grid[nr][nc] != OPEN or (nr, nc) in seen:
        continue
    seen.add((nr, nc))          # claim before queueing
    queue.append((nr, nc))
```

A matrix has two dimensions of indexing, and Python has an aliasing trap: `[[0] * cols] * rows` makes repeated references to the *same* inner list. Use `[[0] * cols for _ in range(rows)]` to create independent rows. When a grid is huge and sparse, storing only open or live cells in a set may be much smaller than a full matrix. When it is dense and bounded, a boolean array or bitset reduces overhead.

For spiral traversal (#126), keep top, bottom, left, and right boundaries and shrink each after sweeping its edge; recheck that boundaries still cross *before* walking a reverse edge. The grid is the same, but the state is four moving borders instead of a visited set.

Break test. Create a three-row `visited` grid with repeated inner lists and mark `visited[0][0]`. If all three rows change, the data representation is broken before BFS starts.

Source notes: #38, #74, #78, #79, #85, #126.

19.6 Simultaneous updates need an old state and a new state

In Game of Life (#89), all cells must be updated *as if* from the old board. Writing a new cell value directly makes later neighbor counts see the future. The straightforward solution reads the old board and writes to a separate new board. If in-place space is required and values are binary, use two bits per cell: bit zero is the old state throughout the first pass; bit one receives the new state; a second pass shifts bit one into place.

```
old = board[row][col] & 1       # stable even after neighbors write
live = sum(board[nr][nc] & 1 for nr, nc in nearby)
if live == 3 or (old == 1 and live == 2):
    board[row][col] |= 2         # set future bit
# After every cell has a future bit:
for row in range(rows):
    for col in range(cols):
        board[row][col] >>= 1
```

This is a transaction with a prepare phase and a commit phase. It generalizes beyond bit tricks: mark rows/columns to clear before changing a matrix, collect next-step frontier positions before replacing the current frontier, or apply batched updates only after all readers have finished. Encoding two states into one cell is safe only when every reader masks out the future bit; even one test `board[nr][nc] == 1` reads the wrong state when a neighbor has been marked as 3.

State. Original state that remains readable, plus pending next-state information.

Invariant. During preparation every decision reads only original values.

Update. Compute all new values without destroying old ones; commit only after the last old-state read.

Break test. Run a 2-by-2 all-live block. It should remain unchanged. Counting a prepared cell by equality to 1 rather than `& 1` can make later cells die.

Source notes: #89, #126, #127, #148.

19.7 Bitmasks and bitmaps: name the universe before shifting

A bitset is exact only when every possible item has a *distinct* position. For keys in `[low, low + width)`, position `i = key - low`; for arbitrary names, assign stable dense IDs first, or use a set if the universe is sparse or unknown. `hash(key) % width` is not an exact mapping: keys can collide. A subset mask uses the same representation, but its bits name chosen *decisions* instead of stored members. The two bits per Game-of-Life cell above (#89) instead name old and future *states* of one cell.

```
low, width = 10, 5                    # allowed keys: 10..14

def bit_for(key):
    i = key - low
    if not 0 <= i < width:
        raise ValueError("out of universe")
    return 1 << i

mask = 0
mask |= bit_for(12)                   # set bit 2
mask |= bit_for(14)                   # set bit 4
present = bool(mask & bit_for(12))    # test
mask &= ~bit_for(12)                  # clear bit 2
count = mask.bit_count()              # one member remains
```

OR adds idempotently; `mask ^= bit_for(key)` *toggles*, so using XOR to "add" the same key twice removes it. AND and OR give intersection and union only when both masks use the *same* mapping. For a larger bounded universe, `bytearray((width + 7) // 8)` is a byte-packed bitmap: position `i` lives in byte `i // 8`, with test/set bit `1 << (i % 8)`. A Python integer is convenient when the width is moderate; its bitwise operations are not constant-time for arbitrarily large integers.

For `n` labeled items, `range(1 << n)` visits all 2^n masks, including zero. Decode each by testing `mask & (1 << i)`. If a parent mask is already fixed, visit only *its* submasks:

```
items = ("a", "b", "c")
subsets = [[items[i] for i in range(len(items)) if mask & (1 << i)]
           for mask in range(1 << len(items))]

def submasks(mask):
    if mask < 0:
        raise ValueError("mask must be nonnegative")
    sub = mask
    while sub:
        yield sub
        sub = (sub - 1) & mask
    yield 0                             # include the empty submask once
```

For `mask = 0b1010`, this yields `0b1010, 0b1000, 0b0010, 0b0000`; `submasks(0)` yields just zero. The step clears a low set bit, fills lower positions, then discards positions outside the parent mask. With k set bits this takes 2^k steps; generating all masks takes 2^n steps *before* decoding them. If only the set positions matter, under `while remaining:` take `bit = remaining & -remaining`, extract its index as `bit.bit_length() - 1`, and clear it with `remaining &= remaining - 1`.

Python has arbitrary-precision signed integers, not an implicit unsigned machine word. `~0` is `-1`, and `-1 >> 1` stays `-1`; a shift-to-zero loop on a negative input will not terminate. Clearing a bit with `mask &= ~bit_for(key)` is safe because the nonnegative `mask` bounds the result; a bare complement does not. Fix the intended width *before* counting, shifting, or complementing a fixed-width value:

```
width = 8
full = (1 << width) - 1
value = -2
unsigned = value & full               # 11111110
ones = unsigned.bit_count()           # 7, not (-2).bit_count()
flipped = (~unsigned) & full          # 00000001
rotated = ((unsigned << 1) | (unsigned >> (width - 1))) & full
```

An exact bitset assigns a bit to each bounded key. A Bloom filter (#146) instead hashes *arbitrary* keys to several shared bit positions: with insertion-only bits and consistent hashes, any zero means definitely absent, but all ones mean only "might be present". Two different keys can probe the same positions, so adding one can make the other appear present. Never clear shared bits to delete a key; doing so can make another inserted key appear absent. Use a Bloom filter as a cheap rejection test before an exact lookup when false positives merely cost extra work, a set when exact arbitrary-key membership is required, and a direct bitset for an exact bounded universe. Persisted filters also need stable hashes across processes, not Python's randomized string `hash()`.

> **State.** A one-to-one key-to-position map, its bounds, and a nonnegative mask or byte array.
>
> **Invariant.** A set bit names exactly one present item; a submask has no bit outside its parent.
>
> **Update.** Check bounds before shifting, OR to insert and AND to remove; stop submask iteration at zero and include it once.

Break test. Use `while sub >= 0` with `sub = (sub - 1) & mask`. After sub reaches zero, the next step returns to `mask` and the loop never ends. For `mask = 0`, it never advances at all.

Source notes: #89, #135, #136, #137, #146.

19.8 Mutable range queries: precompute or maintain a tree

An immutable array with many range-sum questions needs prefix sums: $P[r+1] - P[l]$ answers the inclusive range from l to r in constant time. Once values change, every later prefix might need revision. A Fenwick tree maintains sums through indexed partial prefixes, supporting point updates and prefix queries in logarithmic time. A segment tree (#107) stores the aggregate for each interval; its parent combines two children, and a query handles three cases: no overlap (identity), full overlap (stored value), partial overlap (query both children).

```
def query(node, lo, hi, left, right):
    if hi < left or right < lo:
        return 0                # identity for sum
    if left <= lo and hi <= right:
        return tree[node]       # whole segment already known
    mid = (lo + hi) // 2
    return (query(2*node, lo, mid, left, right)
            + query(2*node+1, mid+1, hi, left, right))
```

Use a segment tree when range aggregation and point updates interleave, especially when the aggregate is not easily recovered by subtracting two prefixes (e.g., min/max). The no-overlap identity must match the operation: zero for sum, positive infinity for min, negative infinity for max. An interval module (#142) solves a different problem: it stores covered intervals and answers whether every point in a range is covered. Choose by the *query algebra*, not by the word "range".

State. Tree nodes owning precisely specified index ranges and their aggregate.

Invariant. A node's stored value is the combined value of its segment.

Update. Update the one root-to-leaf path; on query, reuse full-overlap nodes and combine partials with the correct identity.

Break test. Use zero as the no-overlap value for a minimum query of positive inputs. Every partial query then incorrectly returns zero.

Source notes: #3, #47, #92, #107, #142.

19.9 Approximation and indexing: state the answer contract

The exact-bitset/Bloom comparison above shows how a structure can change the meaning of an answer. Section 20.8 in the next chapter develops compact membership and frequency summaries and spatial/text indexes, including their error and memory tradeoffs. Here ask only: can the caller tolerate "maybe" or an overcount? Are deletion and stable behavior across restarts required? If membership must be exact, verify against authoritative state; a screening result alone is not an answer. Key placement (#139) addresses a different operation.

Source notes: #139, #146, #147, #148, #150.

CHAPTER 20

Under the hood: data structures by workload

"Use a hash map" is a recommendation about operations, not a spell that makes them cheap. In an interview or a design exercise, being able to explain how the structure maintains its promise helps you decide when to use it and when not to. First name the dominant workload: reads, writes, deletes, ordered queries, or streaming updates. Then ask what metadata is retained between operations, and what happens when a representation fills up.

20.1 Hashing: from arbitrary key to a bucket

A hash map computes a hash from a *stable* key and uses the hash to choose a bucket. If two different keys choose the same bucket, it must still check key equality. Separate chaining (#51) stores a small list of pairs in each bucket; a lookup scans only the chosen list. Under a good hash distribution and controlled load factor, those lists stay short on average. The expected operation cost is constant, but a badly distributed or adversarial set of keys can put many entries in one bucket and make a lookup linear.

```
bucket = buckets[hash(key) % len(buckets)]
for pair in bucket:
    if pair.key == key:
        pair.value = new_value     # update existing key
        break
else:
    bucket.append(Pair(key, new_value))
    count += 1
```

The invariants are that each live key occurs once and lives in the bucket named by its hash. If the table fills beyond a chosen load threshold, allocate more buckets and reinsert *every* entry, because the modulo result changes when bucket count changes. Resizing costs linear time on that operation but is amortized across many insertions. Do not mutate a key while it is in a map: the hash or equality relation may change while the entry remains in the old bucket. A tuple of anagram letter counts is a good immutable key; a mutable list is not.

Use a hash map when you repeatedly need arbitrary-key equality lookup, associated values, or counts. Prefer direct array indexing for dense small integer keys; prefer an ordered structure for predecessor/range questions. Dict insertion order in Python does not update on read, so it does not by itself implement LRU recency. A hash map is an *index*, not a complete policy for every query.

Break test. Insert two distinct keys that collide in one bucket. A lookup that returns the bucket's first value without checking key equality confuses them.

Source notes: #1, #5, #44, #51, #101.

20.2 Open addressing: a deleted slot is not an empty slot

Open addressing (#101) keeps entries in the table array itself. A collision probes more positions, for example successive slots with wraparound. A lookup stops at a slot that has *never* been used: no earlier

insertion of this key could have skipped it. Deletion cannot simply clear a slot to that empty state. It must leave a tombstone so lookups for keys that probed beyond it can continue.

Picture keys A and B both hashing to slot 2. A goes in slot 2; B probes onward to slot 3. Delete A. If slot 2 becomes "never used", a lookup for B stops there and falsely reports absence. A tombstone says "there used to be something here; keep probing". On insertion you may remember the first tombstone as a reusable slot, but you must keep probing until you find either the same key to update or a never-used slot. Otherwise you can insert a duplicate key farther along the probe sequence.

```
first_deleted = None
position = hash(key) % capacity
while slots[position] is not NEVER_USED:
    if slots[position] is DELETED:
        if first_deleted is None:
            first_deleted = position
    elif slots[position].key == key:
        slots[position].value = value
        return
    position = (position + 1) % capacity
target = position if first_deleted is None else first_deleted
slots[target] = Entry(key, value)
```

This sketch assumes a never-used slot always exists; maintain that invariant by resizing below full occupancy. Tombstones also build up after deletes, so count them in the rehash policy. Open addressing can be cache-friendly because entries sit in an array, but probe lengths deteriorate as occupancy grows. The lesson for algorithmic coding is broader: if a sentinel has two logical meanings, give those states distinct representations.

Break test. Insert A and B with the same initial slot, delete A, then look up B. Treating the tombstone as "not found, stop" loses B.

Source notes: #51, #101, #146.

20.3 Two-way indexing: a map and an array

Insert Delete GetRandom O(1) (#91) needs three operations: find a value, remove it, and pick a uniform value. A dict alone supports the first two but offers no constant-time random index; an array supports random indexing and append but deletion in the middle shifts a suffix. Keep both: `values` is a packed array and `position[value]` gives its current array index. Delete by swapping the target with the array's last value, updating that last value's map entry, then popping the last slot.

```
index = position[value]
last = values[-1]
values[index] = last
position[last] = index
values.pop()
del position[value]
```

If the removed value is already last, these assignments briefly rewrite its own map entry and then delete it; the result is still correct. This operation deliberately changes array order. If the contract requires preserving order, constant-time deletion by swap is not available without a different representation. The same map-to-object principle appears when cloning a graph (#80): map each original node to its clone *before* recursing into its neighbors so a cycle reaches the existing clone instead of recursing forever.

State. Dense array and map from each live value to its one array position.

Invariant. The two structures contain the same values, and every map index points to that value's actual array slot.

Update. Update the moved item's map position whenever an array slot is reused, then remove the old key.

Break test. Delete a non-last element without changing the last element's stored index. A later delete of that moved element looks at a position past the end.

Source notes: #44, #80, #91, #100.

20.4 Circular buffers: preserve order without shifting

A fixed-capacity queue can use an array with a `head` and a `size`. The logical element at offset i is physically at $(head + i)$ mod $capacity$. Enqueue writes at $(head + size)$ mod $capacity$ and increments size; dequeue reads head, increments head modulo capacity, and decrements size. Using size distinguishes full from empty even when head and tail indices happen to be equal.

```
class RingQueue:
    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be positive")
        self.items = [None] * capacity
        self.head = self.size = 0

    def enqueue(self, value):
        if self.size == len(self.items):
            raise OverflowError("queue full")
        tail = (self.head + self.size) % len(self.items)
        self.items[tail] = value
        self.size += 1

    def dequeue(self):
        if self.size == 0:
            raise IndexError("queue empty")
        value = self.items[self.head]
        self.items[self.head] = None
        self.head = (self.head + 1) % len(self.items)
        self.size -= 1
        return value
```

The `size` field, not a sentinel value in the array, determines whether a slot is occupied, so even `None` can be enqueued as data. If the queue should grow rather than reject a full write, resizing must copy values in *logical* order, starting at head, then reset head to zero. Design Hit Counter (#45) uses a different ring: time t mod $window$ selects a bucket. Store the full timestamp alongside its count so an old bucket at the same modulo index is recognized as expired rather than mistaken for current time.

Break test. With capacity 3, enqueue A, B, C, dequeue A, then enqueue D. The physical array wraps D into slot 0; a nonmodular tail index walks off the array.

Source notes: #45, #95, #110, #138.

20.5 Bounded keys: count instead of compare

Comparison sorting pays at least logarithmic work per item in the general case. If keys are integers from a small known interval, counting sort (#108) can allocate one counter per possible key and use $O(n+U)$ time and $O(U)$ extra space for universe size U. Negative values need an offset: index `value - minimum`. Stable counting sort makes counts into cumulative end positions and walks the input backward when placing items so equal keys keep their original order.

This is a *representation trade*: pay memory proportional to the key universe to avoid comparisons. If keys range over billions of integers and only a handful appear, the count array is the wrong structure; use a dict of observed counts, then sort its keys if ordered output is required. Bucket sort (#61) is useful when a bounded distribution makes buckets small, but its expected-time argument depends on that distribution. A bitmap is a particularly compact count array when membership alone matters.

State. Known key bounds and either a count array or a sparse map of observed keys.

Invariant. Count slot `value - minimum` equals the frequency of that exact key so far.

Update. Increment one slot per item; rebuild output in key order using the stored counts.

Break test. Values `-2` and `3` need six possible slots from `-2..3`, not four. Direct indexing by value turns a negative input into an unrelated Python list position.

Source notes: #61, #101, #108, #129.

20.6 Sorted array, heap, or range tree?

Each ordered structure supports a different question. A sorted array answers "first value at least `x`" in logarithmic comparisons but insertion must shift elements and costs linear time. A heap answers "current minimum" quickly but does not support general predecessor search. A segment tree answers "sum/min over indices `l..r`" under updates, but does not sort arbitrary keys. A Fenwick tree is a compact choice for point updates and prefix sums when the operation supports subtraction for range sums.

The choice changes with the *interleaving* of operations. If data is static and queries are many, sort once or build prefix sums. If points change after every query, update-friendly indexed aggregates earn their setup cost. If only one extreme is ever requested, a heap is simpler than maintaining a globally sorted list. A range module stores unions of covered intervals rather than numeric aggregates, so substituting a sum segment tree just because both APIs mention "range" misses the query contract.

Python's `bisect` finds positions but does not make list insertion logarithmic. A priority queue with replacement and lazy deletion can accumulate stale entries; its space is not automatically the number of live keys. This is why "what operations happen between queries?" must be answered before "which data structure?".

Break test. Use `bisect_left` for each of a million inserts into a Python list and claim $O(n \log n)$ overall. Locating the insertion point is fast; shifting later elements is still linear per insert.

Source notes: #14, #19, #47, #92, #107, #142.

20.7 A text cursor is a boundary between two stacks

Design a Text Editor (#144) asks for insertions and cursor moves. A simple string with a numeric cursor makes insertion in the middle copy a suffix. Instead, keep `left` and `right` as stacks of characters on

either side of the cursor. Inserting pushes onto left. Moving left pops a character from left and pushes it onto right; moving right does the reverse. The logical text is `left + right[::-1]` for character lists.

```
def move_left():
    if left:
        right.append(left.pop())

def move_right():
    if right:
        left.append(right.pop())
```

The two stacks store an *ordered partition*, not two copies of the same data. The current cursor is `len(left)`. This is a different choice from Browser History (#95), where the past is a list of visited pages and the cursor moves among whole entries; visiting a new page discards the forward suffix. If the editor must support arbitrary random access to very large text, a rope or balanced piece table may be preferable, but the two-stack invariant is a strong starting point for local cursor operations.

Break test. After typing "abc", move left once, type "X", then move right. The logical text should be "abXc", with cursor after c. Treating the right stack as left-to-right text instead of reversed text moves the wrong character.

Source notes: #49, #77, #95, #144.

20.8 Bounded memory changes the error contract

A direct bitset for a small universe is exact: each item has its own bit. A Bloom filter reuses bit positions across keys and can answer false positives. A Count–Min Sketch hashes a key to one counter per row and returns the minimum; collisions inflate estimates for insertion-only streams. A quadtree stores spatial regions, compressing repeated values when many neighboring cells are alike. An inverted index stores postings only for terms that actually appear in documents. All trade materialized data for different queries and guarantees.

For a Bloom filter with m bits, k hash probes, and n inserted items, the approximate false-positive probability is $(1 - e^{-kn/m})^k$. More probes are not always better: eventually most bits become one. A Count–Min Sketch uses more counter columns to reduce collision error and more independent rows to reduce the probability of a large overestimate. Neither structure offers a general exact `delete` just because a counter or bit can be changed: another key may share that state.

Ask whether a false yes, an overcount, or a missed historical state is permitted. If not, use an exact set, counter map, or log. Stable hash functions matter for persisted approximate indexes: Python's process-randomized `hash()` is not a durable on-disk hash contract.

Break test. Use an approximate membership test as the sole check for whether a user owns an item. A false positive becomes an incorrect authorization decision; add an exact lookup before deciding.

Source notes: #137, #139, #146, #147, #148, #150.

CHAPTER 21

Greedy, dynamic state, and exact arithmetic

A short loop can hide a large proof. Greedy code repeatedly commits to a local choice; dynamic programming saves a carefully defined subproblem; numerical algorithms preserve a mathematical identity. The bridge to Python is to turn that reason into a variable meaning, a progress rule, and an explicit way to handle an impossible case. When a choice is not irrevocable, keep it as a candidate rather than prematurely changing the answer.

21.1 Greedy: name the choice you can never regret

Interval scheduling (#70) chooses the available interval that ends earliest. The state is the end of the last chosen interval; scan intervals sorted by end and accept one when its start is at or after that end for half-open meetings. The exchange argument is a coding tool: replacing any first chosen interval with an earlier-finishing one can only leave *more* room for later intervals. It tells you why the sorting key must be end, not start.

```
last_end = float("-inf")
chosen = 0
for start, end in sorted(intervals, key=lambda pair: pair[1]):
    if start >= last_end:
        chosen += 1
        last_end = end
```

The `lambda` above is only a sort key, not part of the greedy proof. A named function or sorting `(end, start)` pairs works as well. Merge Intervals (#26) sorts by *start*, because it solves a different question: extend the active union. When choosing a greedy sort order, articulate the future option it protects.

> **State.** The boundary left by irrevocably chosen items.
>
> **Invariant.** No rejected earlier interval can yield more future choices than the chosen earliest-finishing one.
>
> **Update.** Consider in end-time order; accept only when the new interval does not conflict with the boundary.

Break test. Intervals `(1,10), (2,3), (3,4)`. Choosing the earliest start takes one; choosing the earliest finish takes two.

Source notes: #26, #67, #70, #71, #72.

21.2 Collapse a BFS layer to two integers

Jump Game II (#68) is a shortest-path problem on indices, but every layer of reachable positions is a contiguous range. You do not need a queue of those indices: `current_end` marks the end of the layer being scanned, and `farthest` is the end of the next layer discovered so far. When the scan reaches `current_end`, all launch points in this layer have been considered, so count one jump and advance the boundary. Stop before the final index; already standing at the target costs no extra jump.

```
jumps = current_end = farthest = 0
for index in range(len(nums) - 1):
    farthest = max(farthest, index + nums[index])
    if index == current_end:
        if farthest == current_end:
            return -1             # only if reachability is not promised
        jumps += 1
        current_end = farthest
return jumps
```

For `[2, 3, 1, 1, 4]`, layer 0 ends at 0, layer 1 ends at 2, and layer 2 ends at 4: the answer is 2. The update of `farthest` comes *before* deciding whether the current layer is finished, because the last index of that layer may be its best launcher. This is BFS without materializing the FIFO queue; the invariant still comes from BFS.

Break test. On a one-element array, the loop is empty and the answer is zero jumps. A version that counts one jump before scanning the layer overcounts.

Source notes: #39, #67, #68, #69.

21.3 A failed candidate can eliminate a whole prefix

In Gas Station (#69), the local tank balance tracks fuel gained minus fuel spent from the current candidate start. If it becomes negative at station `i`, *none* of the stations from that start through `i` can begin a full trip: each starts with no more fuel at that point than the failed candidate had. Restart at `i + 1`. A global balance makes a separate final claim: if total fuel is less than total cost, *no* start works. Both tests are needed.

```
start, tank, total = 0, 0, 0
for index in range(len(gas)):
    delta = gas[index] - cost[index]
    tank += delta
    total += delta
    if tank < 0:
        start = index + 1
        tank = 0
return start if total >= 0 else -1
```

Notice the two scopes: `tank` is since the most recent restart; `total` never resets. Kadane's algorithm (#28) makes a related restart when the previous subarray hurts a candidate extension, but its global answer must still handle negative values. A restart rule needs a proof that the discarded starts cannot become optimal later; otherwise it is a heuristic, not a greedy algorithm.

Break test. On `gas = [2,3,4]` and `cost = [3,4,3]`, a local scan identifies a last candidate, but total fuel is short. Returning it without the total check is wrong.

Source notes: #28, #29, #67, #69, #111.

21.4 DP: define the subproblem before the table

"Use DP" does not specify what a cell means. Say it literally: `dp[i]` is the best answer *ending at* index `i`, or the number of ways to reach exactly step `i`. Then write a recurrence using only smaller states, a base

case, and a fill order in which dependencies already exist. Memoization asks for the same subproblem on demand; tabulation visits them in an explicit order.

Fibonacci (#90) has state `F(i)` depending only on `F(i-1)` and `F(i-2)`. Two variables suffice:

```
previous, current = 0, 1    # F(0), F(1)
for _ in range(n):
    previous, current = current, previous + current
return previous             # F(n)
```

Longest Increasing Subsequence (#58) can instead keep `tails[k]`, the smallest end value seen for an increasing subsequence of length `k + 1`. `tails` is sorted and lower bound finds the first end at least as large as the new value; replace it or append. The array of tails is an *optimization frontier*, not necessarily an actual subsequence: on `[4,5,6,1]`, it ends as `[1,5,6]`, which is not in input order, but its length 3 is correct. If the caller needs the sequence itself, add predecessor indices rather than returning `tails`.

> **State.** Precisely defined subproblem state and only the predecessor states its recurrence uses.
>
> **Invariant.** Each saved entry is a correct answer for its own subproblem; dependencies exist before each new entry.
>
> **Update.** Compute a state from finished dependencies; compress history only when the recurrence no longer needs it.

Break test. For strictly increasing LIS, use the first tail *greater than* an incoming equal value. Four equal 7s then incorrectly build a length-four subsequence.

Source notes: #28, #58, #90, #107.

21.5 Write in place with a reader and a scribe

When an output fits inside its input, use a *read* pointer to finish a unit of input and a *write* pointer to place the compact result. String compression (#123) reads a whole run before writing its character and the digits of its length. It is safe to overwrite because encoding one run never needs more cells than the run occupied. That safety argument must be stated before relying on in-place writes.

```
read = write = 0
while read < len(chars):
    start = read
    char = chars[read]
    while read < len(chars) and chars[read] == char:
        read += 1
    chars[write] = char
    write += 1
    count = read - start
    if count > 1:
        for digit in str(count):
            chars[write] = digit
            write += 1
return write
```

For output strings not constrained to be in-place, append fragments to a list and `"".join(parts)` at the end instead of repeatedly concatenating immutable strings. Text justification (#122) chooses the words for a line *before* distributing spaces; line selection and line rendering are separate phases, like read and write. The last line has a different spacing policy, so identify it from its position, not from a guessed

width calculation. Multiplying decimal strings (#125) instead reserves up to `m + n` digit positions, adds each pairwise product into its place-value slot, and carries overflow leftward; the write positions encode powers of ten, not input indices.

Break test. A run of twelve `"b"` characters writes the characters `"b"`, `"1"`, `"2"`. Writing `"12"` into one list cell does not satisfy the output contract.

Source notes: #88, #122, #123, #124, #125.

21.6 Geometry: use exact signs, not approximate slopes

For integer points, orientation is the sign of a cross product. Positive means left turn, negative means right turn, zero means collinear. It uses only subtraction and multiplication; a slope quotient creates division-by-zero cases and floating-point equality problems. Convex hull (#133) keeps a stack of boundary candidates, popping while a new point invalidates its required turn. Whether collinear boundary points should be included decides whether the pop test uses `<= 0` or `< 0`.

```
def cross(origin, a, b):
    ax, ay = a[0] - origin[0], a[1] - origin[1]
    bx, by = b[0] - origin[0], b[1] - origin[1]
    return ax * by - ay * bx
```

For Max Points on a Line (#134), points sharing a slope from one anchor need an exact hash key. Reduce `(dy, dx)` by `gcd(abs(dy), abs(dx))` and normalize its sign; represent vertical and horizontal directions consistently. Segment intersection (#132) uses signs of orientations *and* separate bounding-box checks for collinear endpoints. As in every boundary problem, "lies on the line" does not imply "lies on the segment".

> **State.** Integer vector differences and a normalized orientation or direction key.
>
> **Invariant.** Cross-product signs reflect geometric side without rounding; equal rational slopes have equal normalized integer pairs.
>
> **Update.** Compute signs or normalized keys before deciding to keep, pop, or count a point.

Break test. Segments on the same infinite line may be disjoint. With endpoints `(0,0)-(2,2)` and `(3,3)-(5,5)`, all cross products are zero but the bounding boxes do not touch.

Source notes: #128, #132, #133, #134.

21.7 Sieve: spend work only where it proves something

To find all primes below `n`, do not trial-divide every number independently. Keep a boolean table: an uncrossed number is a prime candidate until smaller primes have had a chance to mark it. For each uncrossed `p`, cross its multiples starting at `p*p`. Smaller multiples `2*p`, `3*p`, and so on were already crossed by their smaller factors. The outer loop needs to run only while `p*p < n`: every composite below `n` has a factor below that square-root boundary.

```
crossed = [False] * n
p = 2
while p * p < n:
    if not crossed[p]:
        for multiple in range(p * p, n, p):
            crossed[multiple] = True
    p += 1
primes = [i for i in range(2, n) if not crossed[i]]
```

The comprehension only collects survivors after the marking invariant is established; it is not part of the sieve's reasoning. For a single number rather than a whole range, trial division through `d*d <= n` is simpler and avoids the table. Choose the precomputation only when many related queries amortize its cost.

Break test. To test whether 49 is prime, stop trial divisors at `d*d < n` instead of `<=`. You skip exactly its factor 7.

Source notes: #108, #128, #129.

21.8 Halving: make progress by consuming a bit

Fast exponentiation (#130) takes the exponent's bits from least significant to most significant. `base` represents the power for the current bit; `result` holds the product selected by bits already consumed. If the bit is 1, multiply `result` by `base`; square the base, then shift the exponent right. Each loop step removes one bit, so a billion-sized exponent takes only about 30 steps of multiplication.

```
def power(base, exponent):
    if exponent < 0:
        base = 1 / base          # zero raises explicitly here
        exponent = -exponent
    result = 1
    while exponent:
        if exponent & 1:
            result *= base
        base *= base
        exponent >>= 1
    return result
```

For a modular version, reduce both `result * base` and `base * base` modulo `m` at each step, not just at the end. This preserves the invariant that the products represent the same power modulo `m` while keeping intermediates bounded in a fixed-width language. The same "discard half of the remaining problem" pattern appears in binary search, Euclid's GCD, and fast Fibonacci, but each has a different state identity; do not copy their updates. For Euclid's algorithm (#128), the invariant is $\gcd(a, b) = \gcd(b, a \bmod b)$; repeatedly replace the pair until $b = 0$, then a is the answer. The numerical identity, not the presence of a loop, dictates the update.

> **State.** Unconsumed exponent, power of the current bit, and product of powers already selected.
>
> **Invariant.** The selected product times the appropriate powers of the remaining base still equals the original requested power.
>
> **Update.** Consume the lowest bit, square the base, halve the remaining exponent.

Break test. Only reduce the running result modulo `m`; the squared base can grow astronomically while the result appears small.

Source notes: #90, #128, #130.

21.9 Sampling and weights: respect the boundary of a draw

A streaming reservoir of size `k` needs memory independent of stream length. After observing item number `i` (counting from one), draw uniformly from `1..i`; if the draw is at most `k`, replace that reservoir slot. The reservoir is a uniform sample of the prefix after every iteration. Keeping `i` one-based makes the probability `k / i` and the replacement slot easy to state.

```
sample = []
for i, value in enumerate(stream, start=1):
    if i <= k:
        sample.append(value)
    else:
        ticket = random.randint(1, i)   # inclusive endpoints
        if ticket <= k:
            sample[ticket - 1] = value
```

Weighted random pick (#106) uses a different boundary: build positive weight prefixes, draw a uniform integer from `1..total`, and find the *first prefix at least* that ticket. Shifting either draw by one biases the result even though examples still look plausible. State the sample space and inclusivity before calling a random API.

Break test. With `k = 1`, two stream items should each survive with probability one half. A draw from `0..i` instead of `1..i` gives the second item one chance in three.

Source notes: #106, #131, #137.

CHAPTER 22

Design problems: invariants that survive every call

A function can start with fresh locals. A design class cannot: each public method inherits the state left by all previous calls. Begin by writing an operation table with the result, the state it reads, and the state it changes. Define what happens on empty state, missing keys, repeated writes, ties, and capacity zero. Only then decide which helpers and instance fields are needed.

22.1 Make the operation table the class blueprint

Consider an LRU cache of capacity two:

Operation	Reads	Must update
`get(k)` hit	map to node	recency order; return node value
`get(k)` miss	map	nothing; return defined miss value
`put(k,v)` existing	map to node	value and recency
`put(k,v)` new	capacity, map, list tail	map, list; evict if needed

This table catches a classic omission: a `get` hit is a use and changes recency. The fields must be shared by the methods, so they belong on `self`; temporary locals such as `evicted` belong in the method. A helper such as `_move_front(node)` owns one invariant-preserving transition; it should not silently alter capacity or return a cache hit value.

The same blueprint works for a browser history: visit truncates the forward suffix and appends a page, back/forward move a cursor within bounds, and current page is `history[cursor]`. For a Tic-Tac-Toe tracker, each move changes one row, one column, and possibly diagonal counters; a winner check reads only those counters. A good design starts with the operation semantics, not a favorite data structure.

> **State.** Fields whose meaning must persist across calls, plus one event per public method.
>
> **Invariant.** After every public call, all fields agree on the same logical object state.
>
> **Update.** Implement private methods for small transitions, then make every public branch use them consistently.

Break test. Capacity zero: call `put` then `get`. The cache must not keep a real entry or evict a sentinel; make the behavior explicit before implementing `put`.

Source notes: #44, #48, #95, #99, #143.

22.2 Fast lookup plus fast order requires two structures

An LRU cache needs key-to-node lookup and recency changes. A dict gives the first; a doubly linked list of nodes gives constant-time removal, front insertion, and eviction of the tail node. The map points to the *same objects* that appear in the list. Two sentinels, head and tail, are always present so an empty cache and an edge node use the same pointer operations.

```
def remove(node):
    node.prev.next = node.next
    node.next.prev = node.prev

def add_front(node):
    node.prev = head
    node.next = head.next
    head.next.prev = node
    head.next = node

def touch(node):
    remove(node)
    add_front(node)
```

On a hit, `touch` updates recency. On a new insert, add the node, then if over capacity evict `tail.prev` and delete *its key* from the map. The node stores both key and value so eviction can clean both representations. On an existing key, update the same node instead of creating a duplicate. Python's `OrderedDict` can implement the same behavior with library operations, but the hand-built version exposes the invariant the interview is testing.

> **State.** Map key-to-node and linked order newest-to-oldest between permanent sentinels.
>
> **Invariant.** Every map entry points to exactly one real list node, and every real list node appears in the map.
>
> **Update.** All mutations change both representations as one logical operation; evict the node at the old end.

Break test. Update an existing key by inserting a second node. When the ghost old node is evicted, it can delete the live key from the map.

Source notes: #44, #46, #93, #141.

22.3 Frequency buckets: exploit unit-size changes

LFU (#93) needs the least use count and, within that count, the least recent key. A map finds a key's node; a map from count to a recency-ordered bucket stores peers; `min_freq` names the smallest nonempty count. Touching a key moves it from bucket `f` to `f + 1`. If bucket `f` empties and it was the minimum, the minimum becomes `f + 1` because the touched key has just arrived there. A newly inserted key enters bucket 1 and resets the minimum to 1. Evict *before* inserting the new key, or it may evict itself immediately.

All O(1) Data Structure (#141) also groups keys by counts, but supports both min and max. Link the nonempty count buckets in increasing order and map each key to its bucket. A `+1` or `-1` change only needs the adjacent bucket; if it does not exist, splice one in. Remove an empty bucket immediately, or the first and last buckets stop representing the actual extremes. The possibility of moving by exactly one is what makes constant-time local repairs possible; if counts jumped arbitrarily, a tree or heap would be more natural.

Break test. In LFU, `put` to an existing key counts as a use under the stated contract. Updating its value without moving it to the next bucket can evict the wrong key.

Source notes: #21, #44, #93, #140, #141.

22.4 Time is an input and an expiration boundary

Time-dependent classes should accept a timestamp in each call if the problem permits it. Then a test can replay a sequence exactly; no wall clock advances between assertions. A TTL cache (#94) stores `expire_at = now + ttl`. If the contract says "alive until, but not including, the deadline", then `get` expires when `now >= expire_at` and a min-heap purge removes entries with `deadline <= now`. Both methods must agree at equality.

The map is current truth and a deadline heap is a schedule of possibly stale events. A repeated `put` leaves its old event in the heap, which must compare against the map before deleting the key. A job scheduler (#149) needs a generation or unique sequence as well as a name: cancel and reschedule under the same name must not reactivate the old job. A hit counter (#45) or rate limiter (#138) can use a ring buffer or queue if timestamps are nondecreasing; without that promise, expiration-by-front is not valid.

> **State.** Caller-supplied time, authoritative current records, and optional heap/queue of future events.
>
> **Invariant.** Each record's deadline has one exact inclusive/exclusive meaning; an event is live only if it still matches the authoritative record.
>
> **Update.** Expire or discard at the agreed boundary before answering, and version replacements to reject old scheduled entries.

Break test. Put a key with `now = 7`, `ttl = 3`; at time 10 one method returns its value while another counts it as expired. The public API disagrees about whether the deadline is inclusive.

Source notes: #45, #94, #138, #140, #149.

22.5 History: make "as of" a search in sorted writes

Snapshot Array (#47) should not copy the whole array on each snapshot if only a few indices changed. Give each index a sorted history of `(snap_id, value)` pairs; a write before the next snapshot replaces the last pair for the current id, and `snap()` increments the id. A read for an old id uses upper-bound search and returns the last value written *at or before* that id; if none, return the array's initial value.

```
# Pairs for one index: [(0, 5), (3, 8), (7, 2)]
lo, hi = 0, len(history)
while lo < hi:
    mid = (lo + hi) // 2
    if history[mid][0] <= asked_id:
        lo = mid + 1
    else:
        hi = mid
answer = history[lo - 1][1] if lo else 0
```

Time-Based Key-Value Store (#92) uses the same per-key sorted timeline, with timestamps rather than snapshot IDs. The entire optimization is in the representation: writes append at increasing times, so there is no sort, and an old read touches only one key's history. State whether the API guarantees monotone writes; if not, insertions must maintain ordering explicitly.

Break test. History `[(0,5), (3,8)]`, query id 2. A lower-bound search for the first id *at least* 2 returns 8 from the future; upper-bound minus one returns 5.

Source notes: #14, #47, #92, #140.

22.6 Undo logs: preserve the first before-image

A nested transactional store (#50) keeps one live dictionary and a stack of undo logs. `begin()` pushes an empty log. Before the first write to a key at the current level, record the value it had when that level began; later writes to the same key must not replace that saved value. Rollback pops and restores exactly the logged keys. Under the source contract, `commit()` ends *all* open levels and retains the live map; another API might define commit of only the innermost level, requiring a different log merge rule.

```
MISSING = object()              # distinct from a real stored None

def remember(key):
    if not logs:
        return
    undo = logs[-1]
    if key not in undo:
        undo[key] = data.get(key, MISSING)
```

The source note uses `null` as an absent marker and forbids storing it as a value. A more general Python class can use the unique object above so a caller may store `None`. Restoring a `MISSING` key means delete it if present; restoring any other value means assign it. In either case, the log records one *before-image* per key per level, not a list of writes.

> **State.** Live map and one map of before-images per open level.
>
> **Invariant.** Each log entry equals the key's value at its level's begin, even after several writes.
>
> **Update.** Remember before mutating, only the first time per level; rollback the innermost log or discard logs according to commit semantics.

Break test. Set a value to 40 and then 45 inside one transaction after it started at 30. If the second write logs 40, rollback restores 40 instead of 30.

Source notes: #24, #47, #50.

22.7 Iterators: normalize before answering

A lazy iterator must pause work between calls. Flatten Nested List Iterator (#49) stores a stack of `(list, next_index)` frames, one per nesting level. `has_next` normalizes the stack until its top points at an integer or the stack is empty: exhausted list frames are popped, sublists are entered after advancing the parent frame. `next()` calls `has_next` and then consumes that ready integer.

The important API contract is that repeated `has_next()` calls without a `next()` do not skip a value. "Prepare" is allowed to advance through empty lists, but it must stop before consuming the next integer. A BST iterator (#77) uses an analogous stack of pending left branches; a browser history (#95) instead keeps a cursor because backward and forward navigation are part of its API.

Break test. Input `[[], [[]]]` has no integers. Checking only whether the stack has a frame makes `has_next()` lie; normalization must exhaust empty nested lists.

Source notes: #25, #49, #77, #95, #144.

22.8 Identity, placement, and distributed counters

Some "design an object" exercises make the representation itself the contract. A Snowflake-style ID (#145) packs time, machine, and a per-time sequence into fixed-width bit fields; uniqueness depends on nonoverlapping field widths, unique machine identifiers, and refusing or handling sequence overflow. Consistent hashing (#139) maps machines and keys onto a ring; successors of key positions own keys, and virtual nodes smooth uneven load. A URL shortener (#100) combines a key generator, two-way maps, and collision policy.

When a class crosses a system boundary, name what guarantees come from outside it. A unique machine id cannot be conjured locally; a timestamp may run backward; persisted hash positions need stable hashing across processes. Within the class, the same state/invariant/update discipline still applies, but do not claim a global guarantee from a local invariant alone.

Break test. A packed ID generator wraps its 12-bit per-millisecond sequence after 4,096 calls without changing its time field. The next ID repeats the first one minted in that millisecond.

Source notes: #100, #139, #145, #146, #149.

CHAPTER 23

Prove the code with a trace, then try to break it

A pattern is learned when you can recover it from a contract and can catch a plausible wrong variant without looking at an answer key. Do not treat a few passing examples as proof. For each loop, find a tiny input that exercises the boundary it claims to keep: first and last item, duplicate, tie, empty input, unreachable target, expired timestamp, or a branch that returns early.

23.1 The trace is a test of the invariant

A good trace writes down state *at the same point* in every iteration. For BFS, write the queue and seen set just before each pop; for a window, write the edges and counts just after repair; for a heap, write the popped item and authoritative map at pop. Changing the observation point mid-table can hide a one-step discrepancy.

For Dijkstra on edges `A->B (5)`, `A->C (1)`, `C->B (2)`, use this ledger:

Pop	Authority after relaxations	Pending proposals
`(0,A)`	A=0, B=5, C=1	(1,C), (5,B)
`(1,C)`	A=0, B=3, C=1	(3,B), (5,B)
`(3,B)`	unchanged	(5,B)
`(5,B)`	B=3: reject stale 5	empty

This table catches a code path that marks B visited on *push* at cost 5. The popped item `(5,B)` is still physically real; it simply no longer has the authority to expand. The same trace format works for a job heap, with generation instead of distance in the authority column.

For backtracking, trace entry and exit of a call: `path = [1]` at entry; choose 2 gives `[1,2]`; after recursion, undo brings it back to `[1]`; only then may a sibling try 3. If the before and after snapshots of *working state* differ, a restoration was missed. If two recorded results are the same list object, a snapshot was missed.

Break test. Write a trace of two iterations with the queue pictured just *before* the first pop but just *after* the second. The diagram can look consistent while an item has actually been enqueued twice.

Source notes: #23, #39, #42, #74.

23.2 Find the smallest input that distinguishes two orders

When unsure between two lines, construct the smallest case on which they do different things:

Competing moves	Tiny input	Fact it tests
Hash lookup before/after insert	`[3]`, target 6	No item pairs with itself
Mark BFS on enqueue/pop	A–B, A–C, B–D, C–D	D enters queue once
Strict/non-strict warmer test	`[7,7]`	Equal is not warmer
Record window before/after drop	`"AB"`, need `"AB"`	Coverage can vanish
Seed best with 0/first value	`[-4]`	Nonempty answer can be negative
Use latest/earliest prefix index	`[0,0,0]`	Longest span needs earliest
Expire on `>=` or `>`	deadline exactly now	Endpoints need one policy

An empty input, a single item, two equal items, and a diamond graph often reveal more than a hundred ordinary examples. For a result that reuses mutable state, make *two* calls on the same object or input. Some bugs only show up when the first call leaves a mark or old heap entry behind.

The aim is not to memorize test cases; it is to ask "What does this line claim?" and invert that claim. If code says all heap entries reflect current map state, replace one key and look at its old entry. If code says a backtracking call restores the board, run a second search. If code says every candidate start is considered, make a run that begins at the very first position.

Break test. Change a "<" to "<=" without changing the stated invariant. If you cannot explain whether equal values remain on the stack, the equality case was never specified.

23.3 Check output shape, not only a value

A shortest *distance*, a shortest *path*, and a list of all shortest paths require different stored evidence. An answer of 2 may be numerically correct while the caller asked for `[0, 5, 4]`. An empty path, unreachable marker, and missing optional value are different API choices. Write one assertion about the shape before reasoning about performance.

For a reconstructed BFS path:

1. It begins at the source and ends at the target.
2. Each consecutive pair is a real directed edge.
3. Its number of edges equals the recorded distance.
4. If the target was never discovered, the method returns exactly the specified absent-path value.

For topological order, check *every* edge `u -> v` satisfies "position of u before position of v", not just that the result contains `n` different nodes. For a subsets generator, check both cardinality and uniqueness when input elements are distinct; then check each subset's contents against allowed indices.

Break test. A topological sort can emit all the nodes but in reverse dependency order if the graph construction reversed the edges. Output length alone would not catch it.

Source notes: #30, #39, #40, #47.

23.4 Use a deliberately slow oracle on small cases

Many fast patterns have a tiny obvious brute force. For prefix-sum subarray counting, enumerate all nonempty index ranges on arrays of length at most six, including negative values and zeros. Compare that answer with the hash-map version. The slow algorithm is not production code; it is a compact definition of the result that can check the optimized update order.

```python
def slow_count(nums, target):
    answer = 0
    for left in range(len(nums)):
        total = 0
        for right in range(left, len(nums)):
            total += nums[right]
            if total == target:
                answer += 1
    return answer

# Test random small arrays and compare:
# assert fast_count(nums, target) == slow_count(nums, target)
```

To test systematically, generate short inputs from a small alphabet, e.g. every length-four array over `-1, 0, 1`; this automatically includes duplicates, sign changes, and all-zero inputs. Or use a seeded random generator and retain a failing seed and input. For a sliding window that relies on positivity, compare only on inputs satisfying that promise; testing outside its contract is useful for learning why the assumption matters, not for claiming the correct implementation is broken.

Other good pairs: BFS against exhaustive shortest paths on tiny graphs, merge intervals against an endpoint coverage oracle, binary search against a linear scan, and convex-hull membership against small exact cross-product checks. Testing is a way to pressure the invariant, not a substitute for stating it.

Break test. A prefix counter passes all strictly positive examples. Add `[1,-1,1]` with target 1; a sliding window substitute can now miss matches that the brute-force oracle finds.

Source notes: #2, #14, #26, #39.

23.5 Python details that change the proof

Python conveniences have precise costs and semantics. Use `collections.deque` for FIFO: `list.pop(0)` shifts later elements and turns an otherwise linear BFS into a potentially quadratic one. Use `heapq` for minimum priority: the rest of the list is *not* sorted. Make nested mutable arrays with a comprehension rather than multiplying one row; copies of a path must be explicit. A tuple priority may compare its later fields on a tie; insert a numeric sequence before an unorderable node object.

Integers do not overflow at 32 or 64 bits in ordinary Python, but bitwise complements of negative numbers need an explicit width mask. Slicing a string or list costs time proportional to the slice length; a loop that makes a large slice on each step may invalidate a claimed linear bound. Recursive DFS on a long chain can hit the recursion limit even when its mathematical space bound is linear. `None` is not the same as an absent dict key when storing `None` is allowed; use membership or an object sentinel to distinguish them.

If the invariant says...	Python habit that preserves it
Front pops are constant time	`deque.popleft()`, not `list.pop(0)`
Each row has its own marks	Independent inner lists, not repeated references
Every saved result is a snapshot	`path.copy()` before later mutation
No arbitrary object comparison	Heap tuple has a numeric tie-breaker
A bit field has fixed width	Mask after a complement or shift

Break test. Create a 1,000-wide BFS frontier with `list.pop(0)`. The algorithm may still return correct answers, but the advertised $O(V + E)$ time argument no longer matches the operations in the program.

23.6 Count transitions, not loop nesting

For a monotonic stack, the inner while loop pops many items once, but each item is pushed once and popped at most once: total work is linear. For a sliding window, both boundaries advance at most `n` times. For a heap, each push or pop costs logarithmic time *in the heap size*; Top-K keeps that size at `k`, while a lazy Dijkstra heap may hold entries proportional to the edges.

State space as clearly as time. A recursive tree walk takes stack space proportional to tree height, which is `n` on a chain. BFS stores a frontier as wide as a layer and a visited structure up to `V`; a path result adds output space. Backtracking may use only `O(n)` working space but return exponentially many answers. A probabilistic sketch may bound memory while changing the correctness contract.

When a claim says "constant time", say whether it is worst case, amortized, or expected. Hash maps give expected constant-time lookups. A deque's expired events are each removed once, so expiry is amortized. A segment tree query has logarithmic worst-case time. A cache with lazy deletion can hold stale heap entries that make its memory larger than its number of live keys.

Break test. Call each operation of a lazy-deletion job scheduler "$O(1)$" because cancel only deletes from a map. Later `run_due` must still pop the cancelled entry at heap cost; the work did not vanish.

Source notes: #19, #23, #39, #42, #107, #149.

23.7 The final blank-page checklist

Before typing a solution, answer these questions in order:

1. What is returned, and what is mutated? What do empty, impossible, duplicate, and tied cases mean?
2. What is the unit of work? What part of the input has been settled at the start of a step?
3. What state summarizes that part, and who owns each piece of state?
4. What exact invariant is true before the step? What line restores it after the step?
5. Which event makes a candidate an answer? Is the condition tested before or after mutation?
6. Why is the next step guaranteed to progress? What small case would expose a wrong order?

Write a five-line skeleton next: initialize, loop or helper, guard, transition, return. Draw one trace with state before and after. Only then fill in details such as neighbor generation, count updates, or heap priority tuples. If you cannot state the invariant, do not disguise the uncertainty with more syntax. Go back to the operation that the data structure must support.

In the next chapter, each lab presents a familiar algorithm with one missing implementation decision. Try to write the S/I/U answers before reading the worked bridge. The goal is to make "I know the algorithm" turn into "I know the next line and why it belongs there".

CHAPTER 24

Translation labs: from idea to the next line

For each lab, cover the "Bridge" paragraph. Write three lines: *state, invariant, update*. Then place the condition at the event where its truth is meaningful. These are deliberately small implementation decisions, not a repeat of the full algorithm entries. Compare your skeleton with the worked bridge and run its break test.

24.1 The BFS diamond: who claims D?

Try. Edges are A–B, A–C, B–D, C–D. Visit from A and return a shortest path to D. At which exact line does D become visited, and when do you set its parent?

Bridge. The queue holds discovered but unprocessed nodes. The visited mark means "already has a queue entry". Claim D as soon as B discovers it, assign `parent[D] = B`, and enqueue D in the *same* branch. When C later sees D, the mark rejects it. Reconstruct backward D–B–A and reverse the list; the path has two edges. BFS may stop upon discovery of the target if all it needs is one shortest path, or upon pop if the processing step owns the return.

```
if nxt not in seen:
    seen.add(nxt)            # claim before anyone else expands
    parent[nxt] = node
    distance[nxt] = distance[node] + 1
    queue.append(nxt)
```

Break test. Mark at pop and both B and C enqueue D. If you rewrite `parent[D]` on the second enqueue, the first-discovery invariant is lost. *Transfer:* tree levels #32, shortest path #39, Word Ladder #79.

24.2 DFS: what is waiting while a child runs?

Try. Edges `0-1, 0-4, 0-2, 1-3, 3-2` appear in that order. The required DFS preorder from 0 is `[0, 1, 3, 2, 4]`. If an iterative version pushes all of 0's neighbors and marks them immediately, can 3 still enter 2? When should 3 appear in postorder?

Bridge. A DFS frame does not push its entire neighbor list at once. It remembers the next neighbor index. At 0, enter 1 and suspend 0; at 1, enter 3; at 3, enter 2. Now 2 exits, then 3 exits. Only after 1 finishes does 0 resume and enter 4. The resulting postorder is `[2, 3, 1, 4, 0]`. The *exit* is the point where a child can return an aggregate or a directed node can turn black.

```
node, index = stack[-1]
if index == len(graph[node]):
    postorder.append(node)
    stack.pop()
else:
    nxt = graph[node][index]
    stack[-1] = (node, index + 1)
    if nxt not in seen:
        seen.add(nxt)
        stack.append((nxt, 0))
```

Break test. Push and mark 2 from 0. Then 3 sees an already marked 2 and cannot reproduce the specified DFS order. *Transfer:* graph DFS #37, cycle colors #81, postorder topological sort #82.

24.3 Two fires, one queue

Try. A grid contains two fire sources, several unburned cells, and walls. Return the last minute at which a cell burns, or `-1` if one is unreachable. What do you enqueue *before* the loop? Does a source with no unburned neighbors consume a minute?

Bridge. Seed both sources at minute zero. Every newly reached cell gets `time + 1` when claimed and enqueued. Track an initial count of unburned cells and subtract exactly once at that admission event. The answer is the maximum *successful infection time*, initialized to zero. After the queue empties, a positive unburned count means unreachable. No successful infection means zero minutes, not one phantom round.

```
for source in sources:
    queue.append((source, 0))
while queue:
    cell, time = queue.popleft()
    for nxt in open_neighbors(cell):
        if nxt not in burned:
            burned.add(nxt)
            remaining -= 1
            last_minute = max(last_minute, time + 1)
            queue.append((nxt, time + 1))
```

Break test. Start with no unburned cells. The result is zero even if the sources have no neighbors. *Transfer:* Rotting Oranges #78, nearest-source distance, rippling updates in a grid.

24.4 One cell, two futures

Try. A route may cross one wall. A direct route reaches X in one step *after* using that privilege. An open detour reaches X in three steps with the privilege unused. The only edge from X toward the goal crosses a wall. Is marking only X's location at first arrival safe?

Bridge. No: the earlier arrival and the later one cannot take the same next move. Queue `(position, used_break, steps)` and key the visited set by `(position, used_break)`. At X, `(X, 1)` and `(X, 0)` are different states; the latter can cross the final wall. Mark each complete state at enqueue. Do not include `steps` in the key: the first arrival at a given complete state has its shortest step count.

```
next_used = used_break + is_wall(next_cell)
state = (next_cell, next_used)
if next_used <= 1 and state not in seen:
    seen.add(state)
    queue.append((next_cell, next_used, steps + 1))
```

Break test. Mark X by location alone: the direct route blocks the viable detour, and the search reports no route. *Transfer:* shortest path #39, Word Ladder's implicit-state graph #79, keys encoded as a bitmask.

24.5 Two prerequisites, one ready course

Try. A requires no prerequisites; B requires none; C requires both A and B. When may C enter a queue of ready courses? What check tells you there was a cycle?

Bridge. `indegree[C]` starts at two: it is the number of unpaid dependencies. Seed *all* zero-indegree nodes, A and B, before popping anything. Finishing A decreases C's counter to one; C is still blocked. Finishing B decreases it to zero and queues it once. After the queue empties, an order shorter than the number of courses proves that some nodes were permanently blocked by a cycle.

```
for follower in graph[completed]:
    indegree[follower] -= 1
    if indegree[follower] == 0:
        ready.append(follower)
if len(order) != n:
    return []                # incomplete order: cycle
```

Break test. Queue C when its indegree first *decreases*; it can be emitted before B. *Transfer:* Kahn's algorithm #40, Alien Dictionary #83, dependency scheduling.

24.6 A cheaper proposal arrives late

Try. Routes A–B cost 9, A–C cost 1, and C–B cost 2. A heap contains a proposal for B at 9 before it learns about the route through C. Which record is authoritative?

Bridge. After popping A, `dist[B] = 9` and `dist[C] = 1`. Pop C next; relax C–B to 3, write `dist[B] = 3`, and push `(3, B)`. The older `(9, B)` remains in the heap. Upon popping it, compare 9 with the current map value 3 and skip expansion. B was not final when first *pushed*. This settling argument requires nonnegative weights.

```
cost, node = heapq.heappop(heap)
if cost != dist[node]:
    continue                # proposal lost to a better one
for nxt, weight in graph[node]:
    candidate = cost + weight
    if candidate < dist[nxt]:
        dist[nxt] = candidate
        heapq.heappush(heap, (candidate, nxt))
```

Break test. Stop on the first *push* of B; the answer is 9 instead of 3. *Transfer:* Dijkstra #42, price tracker #140, scheduler #149.

24.7 Connectivity: compare roots, not neighbors

Try. Union the pairs (0,1), (2,3), (0,2), then ask whether an edge (3,1) joins two components. `parent[3]` and `parent[1]` can differ. Should union succeed?

Bridge. The key fact is that `find(3) == find(1)`: both roots are 0 after earlier merges. The edge is redundant. Initialize each parent to itself and size to one. Union *roots* by size, and decrease the component count only on a successful merge. Path compression can alter intermediate parents without altering group membership.

```
ra = find(a)
rb = find(b)
if ra == rb:
    return False
if size[ra] < size[rb]:
    ra, rb = rb, ra
parent[rb] = ra
size[ra] += size[rb]
return True
```

Break test. Compare direct parent values instead of roots and a two-level tree can be mistaken for two components. *Transfer:* Union-Find #41, Kruskal #118, incremental connectivity.

24.8 Warmer day: pop to finish, not to forget

Try. For temperatures `[73, 74, 71, 75]`, tell how many days each day waits for a strictly warmer day. What do the indices remaining in the stack mean just before reading 75?

Bridge. They are unfinished questions: indices 1 (74) and 2 (71). The stack's temperatures fall as you read it from bottom to top. The new 75 answers index 2 first (wait 1), then index 1 (wait 2). Index 0 was answered at day 1. The result is `[1, 2, 1, 0]`; day 3 stays at the default zero because no later day appears.

```
while stack and temps[stack[-1]] < temperature:
    old = stack.pop()
    answer[old] = today - old
stack.append(today)
```

Break test. Use `[70,70]`; pop on `<=` instead of `<` and equality is falsely treated as warmer. *Transfer:* next greater #23, histogram #62, convex hull turns #133.

24.9 Histogram: the popped bar finds its width

Try. Heights are `[2, 1, 2]`. The middle bar of height 1 supports a width-three rectangle. At what event does it get measured, and where does its left boundary come from?

Bridge. A stack keeps bars with nondecreasing heights. Append a virtual zero at the end to force unresolved bars to finish. When a shorter height arrives, pop a bar: its right limit is the new index, and its left limit is one more than the *new* stack top, or zero if no lower bar remains. At the sentinel, the height-1 bar sees width three, for area 3.

```
while stack and bars[stack[-1]] > bars[right]:
    height = bars[stack.pop()]
    left = stack[-1] + 1 if stack else 0
    best = max(best, height * (right - left))
stack.append(right)
```

Break test. On `[1,2,3]`, no natural shorter bar arrives. Without a sentinel or cleanup the scan never measures the remaining bars. *Transfer:* monotonic stack #23, histogram #62.

24.10 Top-K: why a min-heap keeps large values

Try. Read values 4, 9, 1, 7 and keep the two largest. Which value should sit at the heap's top after all four? Why not keep a max-heap instead?

Bridge. A min-heap of at most two values makes the *weakest selected* item cheap to evict. After 4 and 9, the top is 4. Push 1, pop it; selection remains 4 and 9. Push 7, pop 4; selection is 7 and 9 and the top is 7, the second-largest overall. A max-heap would make the *best* item cheap to evict, the opposite of the required update.

```
heapq.heappush(chosen, value)
if len(chosen) > k:
    heapq.heappop(chosen)
# after all items: chosen[0] is the kth largest
```

Break test. Read a third value smaller than both selected ones. A correct update ejects that new value, not the current maximum. *Transfer:* Top-K #19, running kth largest, fixed-size candidate frontiers.

24.11 Return an aggregate, then send context forward

Try. On the four-node chain `0-1-2-3`, compute the total distance from each node to every other node. What can child 2 return to parent 1? After learning node 0's total, how does node 1 obtain its own without starting another traversal?

Bridge. From the leaf upward, subtree sizes become `[4, 3, 2, 1]`. For an edge from parent `u` to child `v`, the child contributes its inside distance plus its size: every descendant is one step farther from `u`. So node 0's inside total is 6. Now reroot across `0-1`: three nodes become one step closer, and one becomes farther, giving `total[1] = 6 + 4 - 2 * 3 = 4`. Continue forward to obtain `[6, 4, 4, 6]`. The two passes use opposite orders on the same tree.

```
for u in reversed(order):        # children before parents
    if u != 0:
        p = parent[u]
        size[p] += size[u]
        inside[p] += inside[u] + size[u]
for u in order[1:]:              # parents before children
    total[u] = total[parent[u]] + n - 2 * size[u]
```

Break test. Reroot before finishing the reverse pass. The child size is not final, so the "closer vs farther" adjustment cannot yet be computed. *Transfer:* tree path returns #33, low-link propagation #116, prefix/suffix contributions #3.

24.12 Backpropagation through a shared input

Try. Set `x = 2`, `y = 3`, `p = x * y`, and `f = p + x`. If the output `f` changes when `x` changes, how many routes carry that change? What are the gradients of `f` with respect to `x` and `y`?

Bridge. The forward values give local slopes: `p` responds to `x` with slope 3, and to `y` with slope 2. Reverse the operation order. The addition sends one unit each to `p` and `x`; the multiplication then adds three more to `x` and two to `y`. Thus `gradient[x] = 4` and `gradient[y] = 2`. A shared input needs an accumulator, not a single assigned parent.

```
# Nodes 0=x, 1=y, 2=p, 3=f; slopes use forward values.
inputs = [[], [], [(0, 3), (1, 2)], [(2, 1), (0, 1)]]
gradient = [0.0] * 4
gradient[3] = 1.0
for node in (3, 2, 1, 0):
    for arg, slope in inputs[node]:
        gradient[arg] += gradient[node] * slope
# gradient[:2] is [4.0, 2.0]
```

Break test. Replace the addition into `gradient[x]` with assignment after visiting `p`. The direct `+ x` path is lost and the result becomes 3. Unlike backtracking, this reverse pass accumulates contributions instead of restoring a working path.

24.13 Subsets or permutations: which siblings are distinct?

Try. Input `[1,2]`. Subsets include `[1,2]` but not `[2,1]`; permutations include both. Which variable restricts the next choice? At what depth do you record output?

Bridge. A subset frame has a `start` index: descendants may choose only later positions. Every frame, including the empty one, is a valid subset and records a *copy*. A permutation frame may try any not-yet-used index; a `used` array forbids using one input position twice, and only a path of length `n` is complete. Both implement *choose, recurse, undo*; their state and answer event differ.

```
# Subsets:
answer.append(path.copy())
for i in range(start, len(nums)):
    path.append(nums[i])
    visit(i + 1)
    path.pop()
```

Break test. Append `path` instead of `path.copy()`: the output holds several references to the same list. *Transfer:* subsets #30, permutations #31, combination sum #73.

24.14 Word Search: visited means “on this route”

Try. A word-search grid may need to visit a cell in one candidate route after another route failed. May a helper leave `seen` set after it returns? What if it found the whole word early?

Bridge. A mark means this cell is on the current route, not that it is forbidden for all future starts. First check whether the word is complete, then check bounds and letter, then mark. Try adjacent cells, remember whether any succeeded, *restore*, and return that boolean. Restoration must occur after success as well as failure if the grid is expected to be unchanged.

```
old = board[row][col]
board[row][col] = "#"
found = False
for nr, nc in adjacent(row, col):
    if search(nr, nc, index + 1):
        found = True
        break
board[row][col] = old
return found
```

Break test. Search `"ABCB"` then `"ABCCED"` on the grid from #74. Old marks from the failed query can make the second query falsely fail. *Transfer:* word search #74, trie-guided search #85, N-Queens #113.

24.15 N-Queens: which conditions can prune a branch?

Try. Place one queen per row. Without checking every earlier queen, how can you decide in constant-time set operations whether column `col` is legal for row `row`? What state must be undone?

Bridge. A queen attacks along its column and both diagonals. Store used columns, differences `row - col`, and sums `row + col`. Before choosing, test membership in all three sets. If legal, add all three identifiers, recurse on the next row, then remove all three. The row is implicit in the call frame; there can be no second queen in that row.

```
down, up = row - col, row + col
if col in columns or down in diag_down or up in diag_up:
    continue
columns.add(col)
diag_down.add(down)
diag_up.add(up)
place(row + 1)
diag_up.remove(up)
diag_down.remove(down)
columns.remove(col)
```

Break test. Forget one diagonal removal; a later sibling sees a ghost queen. *Transfer:* row-by-row constraint search #113, pruning before mutation #73.

24.16 A spare A cannot stand in for B

Try. Find the smallest window of `"AAB"` covering `"AB"`. A counter reaches zero after the second A in a buggy implementation. What should `missing` mean?

Bridge. `need[ch]` is the signed deficit of each required character; positive means missing, negative means surplus. `missing` is the sum of positive deficits, including repeated required characters. At the second A, the old deficit is zero, so `missing` stays one. At B, it falls to zero. Record `"AAB"`, shrink past the first A, then record better `"AB"`. Dropping the next A makes the window invalid and stops the shrink.

```
if need.get(ch, 0) > 0:
    missing -= 1
need[ch] = need.get(ch, 0) - 1
while missing == 0:
    record_window(left, right)
    old = text[left]
    need[old] += 1
    if need[old] > 0:
        missing += 1
    left += 1
```

Break test. A version that decreases `missing` on *every* A can claim `"AA"` covers `"AB"`. *Transfer:* minimum window #13, fixed-width anagrams #12, replacement window #53.

24.17 The empty prefix counts as a start

Try. Count subarrays summing to 1 in `[1,-1,1]`. Why seed `counts[0] = 1` and why query before adding the current prefix?

Bridge. Prefix sums after items are 1, 0, 1. The first 1 pairs with the empty prefix; the final 1 pairs with both earlier zeros. Total three subarrays. A map stores how many *earlier* prefixes had each value. Pair current total with `total - target`, then add the current total for later iterations.

```
counts = {0: 1}
total = answer = 0
for value in nums:
    total += value
    answer += counts.get(total - target, 0)
    counts[total] = counts.get(total, 0) + 1
```

Break test. For `[0]` and target zero, recording the current zero before the query counts a zero-length range as well as the real one. *Transfer:* prefix map #2, subarray counting with negatives and zeros.

24.18 First true, not just any equal position

Try. Find the first 3 in `[1,3,3]`. Then find where 4 would be inserted. Can one search return both answers?

Bridge. Write the predicate `nums[i] >= x`. It reads false, then true forever. Maintain unknown `[lo, hi)`, starting at `[0, len(nums))`. If `mid` fails, discard it with `lo = mid + 1`; if it passes, *keep* it with `hi = mid`. When bounds meet, `lo` is 1 for target 3 and 3 for target 4.

```
lo, hi = 0, len(nums)
while lo < hi:
    mid = (lo + hi) // 2
    if nums[mid] < target:
        lo = mid + 1
    else:
        hi = mid
return lo
```

Break test. With target greater than every value, the answer is `len(nums)`. Always check bounds before indexing that position for exact search. *Transfer:* lower bound #14, feasibility #15, snapshot histories #47.

24.19 LIS tails are state, not a subsequence

Try. Process `[4, 5, 6, 1]` for longest strictly increasing subsequence. Why is the final `tails = [1, 5, 6]` not itself an output subsequence?

Bridge. `tails[k]` is the *smallest possible ending value* for a length-`k+1` increasing subsequence seen so far. It is a summary of possibilities from different histories, not a single path. Lower-bound the first tail at least as large as the new number and replace it; append if none exists. The final length is 3, realized earlier by 4,5,6. Reconstruction requires parent pointers and indices in addition to tails.

```
for value in nums:
    pos = lower_bound(tails, value)  # first tail >= value
    if pos == len(tails):
        tails.append(value)
    else:
        tails[pos] = value
```

Break test. On `[7,7]`, a first tail *greater* than 7 would append the second 7 and falsely claim strict increase. *Transfer:* LIS #58, sorted frontier states, output reconstruction.

24.20 A shared endpoint means two different things

Try. Compare `[1,4]` with `[4,5]` as closed intervals, then compare meetings `[1,4)` and `[4,5)` as half-open intervals. What changes in the code?

Bridge. Closed intervals share point 4, so merging uses `start <= end`. Half-open meetings do not overlap at time 4, so a room whose previous `end <= start` may be reused. The same numbers are involved; the data contract determines the equality comparison. Once sorted by start, merge can compare only with the active last result; meeting rooms can compare with the earliest end-time heap entry.

```
# Closed ranges: touching means union.
if start <= merged[-1][1]:
    merged[-1][1] = max(merged[-1][1], end)
# Half-open meetings: touching means reuse.
if end_times and end_times[0] <= start:
    heapq.heappop(end_times)
```

Break test. Replace `max(old_end, end)` with `end`. An interval nested inside another shrinks the merged answer. *Transfer:* merge #26, meeting rooms #27, calendars #98.

24.21 Strings that contain your delimiter

Try. Encode `["a|b", "", "c"]` into one string and recover all three values. Why is `"|".join(items)` not a reversible format?

Bridge. A delimiter is data inside the first value and cannot mark its end unambiguously. Store a length before each value, then a separator between length digits and content: `"3|a|b0|1|c"` if lengths count Python code points. The decoder reads digits to the separator, then *exactly* that many characters, even if some are separators. Its cursor is the first unconsumed character throughout.

```
length_end = encoded.index("|", pos)
length = int(encoded[pos:length_end])
start = length_end + 1
item = encoded[start:start + length]
pos = start + length
```

Break test. The empty string needs its own `"0|"` frame; omitting it loses one item. For a byte protocol, lengths must count encoded bytes instead. *Transfer:* encode/decode #88, parsing frames #25, text processing.

24.22 Old grid and new grid in one cell

Try. Game of Life updates all cells simultaneously. A 2-by-2 block of live cells is stable. How do you update the matrix in place without later neighbors reading new values?

Bridge. Reserve bit zero for the old state, bit one for the new state. Neighbor counts always read `cell & 1`, even after a new state has been written. The first pass sets bit one on next-live cells, leaving bit zero alone. Only after every cell has been decided does the second pass shift right by one to commit.

```
old = board[row][col] & 1
neighbors = count_old_live_neighbors(row, col)
if neighbors == 3 or (old == 1 and neighbors == 2):
    board[row][col] |= 2
# commit after all decisions:
board[row][col] >>= 1
```

Break test. Counting a neighbor by `cell == 1` misses an old live cell that has been marked `3`. *Transfer:* simultaneous grids #89, in-place matrix updates #127.

24.23 A range query's three possible overlaps

Try. A segment tree stores sums. A query range may miss a node's segment, contain it, or partly overlap it. What does each case return?

Bridge. No overlap contributes the identity for addition, zero. Full overlap returns the stored node sum without visiting children. Partial overlap asks both child segments and adds their results. An update touches one root-to-leaf path and recomputes sums on the way back up; old branches need no repair.

```
if hi < left or right < lo:
    return 0                    # no overlap
if left <= lo and hi <= right:
    return tree[node]           # full overlap
mid = (lo + hi) // 2
return query(2 * node, lo, mid, left, right) + \
       query(2 * node + 1, mid + 1, hi, left, right)
```

Break test. Use identity zero for a *minimum* tree of positive values: every partial query may falsely report zero. *Transfer:* segment tree #107, Fenwick sums, range aggregation.

24.24 Bitset or Bloom filter?

Try. One system tracks membership among 64 possible flags; another checks whether a URL *might* have been seen among billions. Would you use the same bitmap scheme for both?

Bridge. For 64 known flags, bit `i` directly names flag `i`. This is an exact bitset: `mask & (1 « i)` tests membership, OR adds, AND with a complemented bit removes. For arbitrary URLs, a Bloom filter hashes each string to several bit positions. Different strings share bits, so all-one means only "might be present". Any zero means definitely absent.

```
# Exact universe:
mask |= 1 << flag
present = bool(mask & (1 << flag))
# Approximate universe:
present = all(bits[pos] for pos in hash_positions(url))
```

Break test. Clear a Bloom filter's bits to delete one URL. Shared bits may vanish for a different URL, creating a false negative. *Transfer:* bitmask subsets #137, Bloom filter #146, count-min sketch #147.

24.25 Enumerating submasks without cycling forever

Try. Only flags 0, 1, and 3 are available: `mask = 0b1011`. Visit every subset of those flags, including the empty set, without looping through all four-bit integers. What happens after the empty set?

Bridge. A subset's bits are contained in the original mask. Starting from the full mask, `(sub - 1) & mask` drops to the next smaller allowed combination. The eight submasks are `1011`, `1010`, `1001`, `1000`, `0011`, `0010`, `0001`, and `0000`. Visit zero once and break *before* decrementing it: in Python, `(-1) & mask` is the full mask again.

```
sub = mask
while True:
    consider(sub)
    if sub == 0:
        break
    sub = (sub - 1) & mask
```

Break test. Omit the zero guard. The transition `0 -> mask` repeats the same eight states forever. *Transfer:* bitmask subsets #137, subset-state search, bounded-universe membership.

24.26 LRU: a read is an update

Try. Capacity two: insert A, insert B, read A, then insert C. Which key is evicted, and which representations must agree?

Bridge. Reading A makes it most recent, so B is evicted. A dict maps keys to exact linked-list nodes; a doubly linked list orders nodes from newest to oldest between sentinels. A hit removes A's node and adds the *same* node at the front. New C is inserted in both map and list; evict B from both.

```
node = nodes.get(key)
if node is not None:        # read hit
    remove(node)
    add_front(node)
    return node.value
```

Break test. Leave B in the dict after unlinking it from the list: `get(B)` now returns a value the cache supposedly evicted. *Transfer:* LRU #44, LFU #93, All O(1) #141.

24.27 Snapshots: latest write not after the query

Try. One index has history `[(0,5), (3,8)]`. Return its value as of snapshot 2. Which boundary does binary search find?

Bridge. Search for the first history entry whose id is *greater* than 2. That position is entry 1, so step back to entry 0 and return 5. Histories are sorted because snap IDs increase. Multiple writes before the next `snap()` replace the last pair for that same ID instead of appending redundant entries.

```
while lo < hi:
    mid = (lo + hi) // 2
    if history[mid][0] <= asked_id:
        lo = mid + 1
    else:
        hi = mid
return history[lo - 1][1] if lo else 0
```

Break test. Return the first entry with id at least 2; it is the future value 8. *Transfer:* snapshot array #47, time-based store #92.

24.28 Expired, replaced, and still present

Try. A key expires at time 5. At time 4, put it again with a new deadline of 9. The heap still holds its old time-5 entry. What happens at time 6?

Bridge. The map is current truth and stores the new deadline 9. Purging pops the time-5 entry but checks it against the map. Its deadline no longer matches, so it is discarded without removing the key. A read at time 9 expires the new record because the deadline is exclusive.

```
while deadlines and deadlines[0][0] <= now:
    deadline, key = heapq.heappop(deadlines)
    if key in current and current[key].deadline == deadline:
        del current[key]
```

Break test. Delete the key whenever any deadline for it is popped. The old entry wrongly kills the live replacement at time 5. *Transfer:* TTL cache #94, job scheduler #149, stock tracker #140.

24.29 Exact turns, no floating-point slope

Try. Decide whether `(0,0) -> (4,0) -> (2,3)` turns left. Decide whether two collinear segments must intersect. Why avoid a slope quotient?

Bridge. Use the integer cross product `(ax-ox)*(by-oy) - (ay-oy)*(bx-ox)`. Here it is `4*3 - 0*2 = 12`, positive, so the turn is left. Two collinear segments have cross product zero but can lie apart; check whether an endpoint falls within the other's bounding box. Slopes divide by zero on vertical lines and introduce rounding for large integers.

```
turn = (ax - ox) * (by - oy) - (ay - oy) * (bx - ox)
if turn > 0:
    direction = "left"
elif turn < 0:
    direction = "right"
else:
    direction = "collinear"
```

Break test. Segments `(0,0)-(2,2)` and `(3,3)-(5,5)` are on one line but do not meet. *Transfer:* orientation #132, convex hull #133, normalized slopes #134.

24.30 Reservoir: the current item gets a fair ticket

Try. A stream has unknown length. Keep one uniform random item. After two items, what probability should each have? Which inclusive random endpoints give it?

Bridge. Item number `i`, counting from one, replaces the current selection with probability `1 / i`. Keep the first item unconditionally; for the second, draw uniformly from `1..2` and replace on one outcome. Then each item has probability one half. For a reservoir of size `k`, draw from `1..i`; if the draw is at most `k`, use it to choose one of the `k` replacement slots.

```
for i, item in enumerate(stream, start=1):
    if i <= k:
        sample.append(item)
    else:
        ticket = random.randint(1, i)
        if ticket <= k:
            sample[ticket - 1] = item
```

Break test. Draw from `0..i` instead of `1..i`; the newest item is overweighted when the `<= k` test is unchanged. *Transfer:* reservoir #131, weighted pick #106.

24.31 Undo: store the state before the first write

Try. A key has value 30. Begin a transaction, set it to 40, then to 45, and rollback. Which value must the transaction log contain?

Bridge. The log remembers the value the key had *before* this level began to change it: 30. It records a key on the first mutation in that level only. The live map holds 45 until rollback uses the log to restore 30. Each nested level owns its own log; an inner rollback restores the outer transaction's live state.

```
undo = logs[-1]
if key not in undo:          # first write at this level
    undo[key] = data.get(key, MISSING)
data[key] = new_value
```

Break test. Recording 40 on the second write restores 40, not 30. *Transfer:* transactional store #50, undo/redo state, snapshot histories #47.

24.32 Jump boundaries: do not count arrival twice

Try. In `[2,3,1,1,4]`, the last index is reachable. Why does the minimum-jumps loop stop *before* the last index, and when does the jump count increase?

Bridge. The current BFS layer ends at `current_end`. While scanning it, each index may extend `farthest`. After reaching that layer's end, one more jump is needed to enter the next layer. The last index is the destination, not a launch point; processing it can count a nonexistent third jump. The boundaries move from 0 to 2 to 4, for two jumps.

```
for index in range(len(nums) - 1):
    farthest = max(farthest, index + nums[index])
    if index == current_end:
        jumps += 1
        current_end = farthest
```

Break test. Continue over the last index and count again when it closes a layer. *Transfer:* Jump Game II #68, interval-shaped BFS frontiers.

24.33 A collision is not an equal key

Try. Two distinct strings hash to the same bucket. A hash-map lookup reaches that bucket. What additional test does it need before returning an associated value?

Bridge. A bucket number narrows the search; it does not prove key equality. In a chained map, scan the bucket's pairs and compare keys. Updating an existing key replaces its value in place; adding a distinct key appends a new pair. If the load factor grows, rehash into a larger bucket array: an old bucket number need not be the new one. Choose the key's representation to preserve its hash and equality while stored.

```
bucket = buckets[hash(key) % len(buckets)]
for stored_key, value in bucket:
    if stored_key == key:
        return value
raise KeyError(key)             # not in this bucket
```

Break test. Return the first pair in the bucket without equality checking. Both colliding strings appear to have one value. *Transfer:* chaining #51, canonical tuple keys #5, general dict-backed indexes.

24.34 Tombstones: a deletion is not a never-used slot

Try. A and B hash to slot 2. A occupies 2; B probes into 3. Delete A. When searching for B, may lookup stop at slot 2?

Bridge. No. Slot 2 is a tombstone, meaning the insertion path continues through it. Only a *never-used* slot proves an absent key. An insert may remember a tombstone to reuse, but it must continue probing in case the same key already appears later in the chain. Use three slot states: never used, live, and deleted.

```
position = hash(key) % capacity
while slots[position] is not NEVER_USED:
    if slots[position] is not DELETED:
        if slots[position].key == key:
            return slots[position].value
    position = (position + 1) % capacity
return MISSING
```

Break test. Change the tombstone back to `NEVER_USED`; the lookup for B returns missing although B still occupies slot 3. *Transfer:* open addressing #101, stateful sentinel design.

24.35 A queue wraps while its logical order does not

Try. An array queue has capacity three. Enqueue A, B, C; dequeue A; enqueue D. Where is D written and what is the logical queue?

Bridge. Start with `head = 0, size = 0`. After three enqueues, the physical cells are A,B,C. Dequeuing A makes `head = 1, size = 2`. The next tail is $(head + size) \bmod 3 = 0$, so D fills the vacated physical slot zero. The logical order, starting at head, is B,C,D. The size distinguishes full from empty even when wrapped positions look alike.

```
tail = (head + size) % capacity
items[tail] = value
size += 1
# dequeue:
value = items[head]
head = (head + 1) % capacity
size -= 1
```

Break test. Use the next physical index without modulo and the fourth enqueue attempts slot 3, beyond capacity. *Transfer:* circular queue #110, time buckets #45.

24.36 A found word can still be a useful prefix

Try. A trie contains `"art"` and `"artist"`. During a grid search, you reach the node for `"art"` and emit that word. May you delete the node to prevent reporting it twice?

Bridge. No. The node also owns child letters for `"artist"`. Clear or remember only the short word's end marker after emitting it, leaving the child links intact. The current trie node is the matched prefix; path-local grid marks are restored after the DFS returns. The child test prunes branches whose letters do not continue *any* remaining prefix.

```
child = trie.children.get(letter)
if child is None:
    return                # no word starts with this route
if child.word is not None:
    answer.append(child.word)
    child.word = None     # emit once; keep child.children
```

Break test. Erase the entire "art" node; the longer word becomes invisible. *Transfer:* trie #43, word search II #85, autocomplete #97.

24.37 LFU: the minimum moves only after its bucket empties

Try. A capacity-two LFU cache inserts key 1, reads it, then inserts key 2. Which frequency is the eviction target when key 3 arrives?

Bridge. Key 1 has frequency 2; new key 2 has frequency 1, so `min_freq = 1` and key 2 is evicted before key 3 enters the cache. A `get` hit moves a node from bucket `f` to `f + 1`. If bucket `f` becomes empty and was the minimum, the minimum becomes `f + 1`. Every new key starts at frequency 1, resetting `min_freq`.

```
old_bucket.remove(node)
if old_bucket.empty() and self.min_freq == node.freq:
    self.min_freq += 1
node.freq += 1
self.bucket_for(node.freq).add_front(node)
```

Break test. Evict *after* inserting key 3 when all older keys have higher frequencies. The new key can evict itself. *Transfer:* LFU #93, frequency buckets #141.

24.38 KMP: a mismatch does not rewind the text

Try. The pattern starts with `"ababa"` and its next expected letter is `"c"`. The text matched that five-letter prefix but its next letter is `"b"`. Must the text cursor go backward?

Bridge. No. The matched prefix's own suffix `"aba"` is also a pattern prefix. A prefix table entry gives length 3 as the next plausible matched length. Compare the *same* text `"b"` again against the pattern letter at index 3; it matches and the count grows to 4. The invariant is "the last `matched` text characters equal the first `matched` pattern characters". For `text = "aaab"` and `pattern = "aab"`, the third `a` falls back from two matched letters to one, then matches again; the following `b` completes a match at index 1.

```
while matched and pattern[matched] != char:
    matched = prefix[matched - 1]
if pattern[matched] == char:
    matched += 1
```

Break test. Reset `matched` to zero and advance the text without retrying the current character: the match at index 1 in `"aaab"` is lost. Restarting a full comparison at each text position can preserve correctness, but loses KMP's linear-work guarantee. After a full match, report it and fall back to the final prefix-table length before reading the next character. *Transfer:* KMP #86, reusable prefix summaries.

24.39 Rate limit: burst or exact sliding window?

Try. A service allows at most three requests in *every* six-second window. Three arrive at time zero and one at time two. Which structure implements this exact promise?

Bridge. A sliding log holds the timestamps of admitted requests in a queue. At time `t`, first remove times `<= t - 6`. If three remain, reject. The request at time two is rejected; at time six the three zero timestamps expire, so new requests may pass. A token bucket instead permits an initial burst and then a steady refill; it need not enforce the exact count in every sliding window.

```
while log and log[0] <= now - window:
    log.popleft()
if len(log) < limit:
    log.append(now)
    return True
return False
```

Break test. Use `<` instead of `<=` at time six and old requests remain one tick too long. *Transfer:* rate limiter #138, hit counter #45.

24.40 Sieve: why mark at the square?

Try. In a sieve, 2 and 3 have already crossed their multiples. When 5 is reached, why begin at 25 rather than 10?

Bridge. A smaller multiple of 5 is `5 * k` for `k < 5`. It has a smaller prime factor and was already crossed. The first useful new mark is `5 * 5`. Continue only while `p * p < n` when listing primes strictly below `n`. For a single number, try divisors `d` while `d * d <= n` instead of building a table.

```
for p in range(2, n):
    if p * p >= n:
        break
    if not crossed[p]:
        for multiple in range(p * p, n, p):
            crossed[multiple] = True
```

Break test. For testing 49 alone, using `d*d < n` skips divisor 7 and incorrectly calls it prime. *Transfer:* sieve #129, start-only work.

24.41 Exponentiation: carry exactly one binary rung

Try. Compute `3**13` without thirteen separate multiplications. Which three values capture what has been done so far?

Bridge. Keep `exponent` as unconsumed bits, `base` as the power represented by the current bit, and `result` as the product selected by previous bits. Since 13 has bits for 1, 4, and 8, multiply by those three successive squares and halve the exponent after each decision.

```
result = 1
while exponent > 0:
    if exponent & 1:
        result *= base
    base *= base
    exponent >>= 1
```

Break test. In modular power, reduce only `result` but not `base` after squaring; the intermediate base becomes unnecessarily huge. *Transfer:* fast power #130, fast Fibonacci #90.

24.42 Compression: the scribe must not outrun the reader

Try. Compress characters `["a","a","b","b","b"]` in place. How can a write pointer safely reuse cells the read pointer has passed?

Bridge. The read pointer walks to the end of one equal-letter run. Only then write its character and, if length exceeds one, its length digits. Encoding a run never uses more cells than the run itself, so the write pointer cannot pass the read pointer. After the two runs, the meaningful prefix is `["a","2","b","3"]`; the value after the returned length is irrelevant.

```
start = read
while read < len(chars) and chars[read] == char:
    read += 1
chars[write] = char
write += 1
if read - start > 1:
    for digit in str(read - start):
        chars[write] = digit
        write += 1
```

Break test. Compress a run of 12 repeated letters but write `"12"` into a single list cell. The output contract requires two digit characters. *Transfer:* string compression #123, read/write frontiers.

CHAPTER 25

Transfer: make the next variant your own

The strongest signal of understanding is a small change in the problem statement. A new constraint should trigger a deliberate change to state, invariant, admission rule, or data structure, not a complete rewrite from memory. These worked transfers use familiar algorithms; the emphasis is the coding bridge that changes when the data or API changes.

25.1 A grid cell may have more than one state

Imagine a grid whose first row is `S . D G` and whose second row is `## a ## ##`. The cell `a` is a key; `D` is a door that opens only after picking it up. From `S`, the shortest route goes through the dot, down to the key, back up to the *same* dot, then through the door to `G`. A visited set containing just `(row, col)` blocks the necessary return visit. The graph's node is the *whole situation*: `(row, col, key_mask)`.

Step	Cell	Keys	Why this state is new
0	S	none	source
1	dot	none	first arrival at dot
2	a	a	acquire key
3	dot	a	same cell, different future moves
4	D	a	door is now passable
5	G	a	target reached

The POPREXEN loop itself does not change. What changes is the identity used for "already discovered". At each neighbor, check walls and doors using the *current* key mask, compute the *next* mask after entering the cell, then test and claim the resulting state before enqueueing it.

```
queue = deque([(start_row, start_col, 0, 0)])
seen = {(start_row, start_col, 0)}
while queue:
    row, col, keys, steps = queue.popleft()
    if grid[row][col] == "G":
        return steps
    for nr, nc in neighbors(row, col):
        cell = grid[nr][nc]
        if cell == "#" or (cell == "D" and not (keys & 1)):
            continue
        next_keys = keys
        if cell == "a":
            next_keys |= 1
        state = (nr, nc, next_keys)
        if state not in seen:
            seen.add(state)
            queue.append((*state, steps + 1))
return -1
```

For multiple keys, assign a bit per key and test each matching door bit. The possible state count is cells times key configurations, not just cells. This is why a grid problem that looks like ordinary BFS may

need far more memory. A visited set keyed by too *little* state incorrectly merges different futures; one keyed by too *much* state needlessly repeats equivalent work.

State. Position, acquired-key mask, distance, and a queue of situations.

Invariant. Every queued situation has a shortest arrival distance for that exact position and mask.

Update. Apply the movement rule, derive the new mask, then claim and enqueue the full new state.

Break test. Mark only the dot coordinate as visited. The route back from the key is discarded, so the goal looks unreachable despite a length-five solution.

Source notes: #39, #74, #79, #137.

25.2 An event queue uses POPREXEN with a new priority

Change BFS's uniform-cost queue to a scheduler's min-heap. The spoken moves still help: *pop* the next event by time, *process* it if it is still live, *expand* any follow-up events, and *enqueue* their future appointments. But the invariant is different: a FIFO queue guarantees nondecreasing hop count; a heap guarantees minimum key among *currently scheduled* events. A cancelled or rescheduled event can remain in the heap. The authoritative map, not the heap, decides whether it still exists.

For a repeating job `ping` due at 5 every 5 units, running due at time 16 must process 5, 10, 15 and re-arm at 20 if the API promises catch-up. Each processed event schedules one successor. If the API instead coalesces missed runs, the successor must be the first appointment *after* 16. The data structure alone cannot choose between those policies.

```
while heap and heap[0][0] <= now:
    due, generation, name = heapq.heappop(heap)
    if live.get(name) != generation:
        continue                  # cancelled or replaced
    output.append((due, name))    # this event really happened
    if interval[name] > 0:
        schedule_next(name, due + interval[name])
    else:
        del live[name]
```

Stop and ask which result is needed: all due events, a count, or the next appointment. Change the processing step to match it; do not accidentally drop an event because the loop happens to be written like BFS. If cancellation is frequent and deadlines are distant, rebuild the heap from live records when stale entries become too numerous.

Break test. Check only the job name after cancel and reschedule. An old event with the same name is mistaken for the new one; the generation is essential.

Source notes: #42, #94, #138, #149.

25.3 Trie search: a prefix is unfinished work

Change Word Search from one word to thousands of words. Running a separate grid search for each word repeats the same prefix paths. A trie shares those prefixes. The DFS frame holds a grid cell, a trie node representing the prefix matched so far, and path-local visited marks. A neighbor is admitted only when its letter is a child of the current trie node.

There are *two* kinds of state: the temporary path mark in the grid, restored on unwind, and the persistent trie that names every word still worth looking for. Arriving at a trie node with a word-end marker records an answer. If each word should be returned once, clear or record that end marker *after* finding it, but do not discard the node if it has children: a longer word may share the same prefix.

```
letter = board[row][col]
if letter not in trie_node.children:
    return
next_node = trie_node.children[letter]
if next_node.word is not None:
    found.append(next_node.word)
    next_node.word = None      # avoid duplicates, keep descendants
board[row][col] = "#"
for nr, nc in neighbors(row, col):
    search(nr, nc, next_node)
board[row][col] = letter
```

This sketch assumes the search helper checks bounds and marked cells before the excerpt; the marker must not be a valid input letter. The trie prunes impossible prefixes *before* exploring their entire grid paths. That is a data-structure choice driven by repeated prefix questions, not a different backtracking algorithm.

Break test. Clear the whole trie node on finding a short word. A longer word sharing that prefix becomes unreachable even though its grid path still exists.

Source notes: #43, #74, #85, #97.

25.4 Sorted postings: the frontier is two positions

An inverted index stores document IDs for each term in increasing order. An AND query for two terms asks for their intersection. The state is a pointer into each postings list; everything before a pointer has already been settled. If the current IDs match, emit that ID and move both pointers. If they differ, the smaller current ID cannot appear later in the other sorted list, so move *only* its pointer.

```
left = right = 0
matches = []
while left < len(a) and right < len(b):
    if a[left] == b[right]:
        matches.append(a[left])
        left += 1
        right += 1
    elif a[left] < b[right]:
        left += 1
    else:
        right += 1
```

For OR, emit the smaller value when different and one value when equal, then append the remaining tail of whichever list survives. For many AND terms, start with the shortest list so intermediate results cannot be longer than that first list. For many OR terms, a heap of list fronts can avoid repeatedly merging a growing result with the next list. The algorithmic move is still “discard only what sorted order proves cannot match”; the selected data structure controls query cost.

Break test. When IDs are 2 and 4, advance both pointers. A later 4 in the first list may be the actual match and is now skipped.

Source notes: #7, #20, #66, #150.

25.5 Compose a TTL with a capacity limit

Add expiration to an LRU cache. Now a key can leave for two reasons: its deadline passes or its recency loses a capacity contest. The map and doubly linked list still need identical live keys. A min-heap of deadlines is an additional *schedule*, not another source of truth. The operation order matters: before evicting a live key for capacity, purge expired keys whose deadlines have passed. On a `get` hit, check its deadline before moving it to the front; an expired read is a miss, not a use.

The helper boundaries follow the invariants. `unlink(node)` changes list pointers only. `delete(key)` removes the map entry *and* its list node. `purge(now)` pops due heap entries and calls `delete` only when the entry's generation still matches the current key. `touch(node)` moves a live node to the front. Public methods invoke these helpers in an order that maintains the agreed deadline and capacity policy.

If a key is rewritten, push a new `(deadline, generation, key)` heap record; the old one becomes stale. The heap can temporarily contain more entries than the map and list. Claiming $O(capacity)$ total memory is false without a stale-entry rebuild policy.

> **State.** Map and linked list of live keys, deadline heap of scheduled checks.
>
> **Invariant.** The map and list agree exactly; a heap entry acts only if its generation matches the map's current record.
>
> **Update.** Purge due records, then perform the read or write and its recency/ capacity change.

Break test. Capacity two holds expired A as the most recent key and live B as the least recent. Insert C without purging A: the cache evicts B even though reclaiming A would have made room.

Source notes: #44, #93, #94, #149.

25.6 The same algorithm, a changed output contract

Change "is there a path?" to "return one shortest path." A BFS visited set still solves reachability, but now it must keep a parent per first-discovered node. Change to "count shortest paths": a first discovery sets a count from its parent; another arrival at the *same shortest distance* adds to that count rather than discarding the edge. Change to positive weighted edges: use a heap and relaxation; marking permanently on first enqueue no longer works.

Change "is this string in the dictionary?" to "list all dictionary words with this prefix." A hash set no longer answers the dominant query; a trie or sorted word list can. Change "sum of a fixed range" to "sum after point updates" and immutable prefix sums no longer have cheap maintenance. Change "membership exactly" to "rule out most nonmembers without storing every key", and a Bloom filter becomes viable *only* if a false yes can be checked later.

The most productive question to ask after finishing a solution is "What single word in the output or operation contract would force a different state?" For each change, name the new state and the old invariant that is no longer true. This is how a memorized algorithm becomes a repertoire of implementation patterns.

Break test. Keep a boolean visited set while changing the BFS output to the *number* of shortest paths. Dropping every later arrival at the same distance loses valid distinct paths.

Source notes: #39, #42, #43, #107, #146.

CHAPTER A

Every source entry, by coding move

This catalog includes all 150 pseudocode entries, in the source's rank order. The rightmost number points back to the pattern page; the title and number identify the source note. One algorithm may use several moves; this is the first implementation decision to recall, not a claim that the algorithm has only one.

No.	Source entry	Coding move	p.
1	Two Sum	Query earlier values before storing	78
2	Prefix Sum + Hash Map	Count earlier prefix sums	78
3	Product of Array Except Self	Exclusive left and right passes	80
4	Longest Consecutive Sequence	Start only at a run boundary	79
5	Group Anagrams	Use an immutable canonical key	116
6	Reverse a Linked List	Save next before rewiring	86
7	3Sum	Fix one and converge two pointers	83
8	Container With Most Water	Discard the shorter wall	83
9	Trapping Rain Water	Maintain left and right walls	80
10	Floyd's Tortoise and Hare	Advance pointers at two speeds	86
11	Longest Substring Without Repeating Characters	Advance the valid window	83
12	Find All Anagrams in a String	Add and evict one character	85
13	Minimum Window Substring	Track signed coverage deficits	84
14	Binary Search	Keep the first true position	87
15	Binary Search on the Answer	Search a monotone feasibility test	87
16	Search in a Rotated Sorted Array	Identify the sorted half	89
17	Merge Sort	Merge ordered halves stably	90
18	Quick Sort	Keep partition regions valid	89
19	Top-K Elements	Evict the smallest chosen item	94
20	Merge K Sorted Arrays	Keep one front per source	94
21	Find Median from Data Stream	Balance two ordered halves	95
22	Valid Parentheses	Close the latest obligation first	91
23	Monotonic Stack	Pop to finish pending answers	92
24	Min Stack	Carry prior minima with entries	91
25	Decode String	Save one frame per bracket	121
26	Merge Intervals	Extend only the active interval	85
27	Meeting Rooms II	Define the meeting tie policy	90
28	Kadane's Algorithm	Separate best ending from best seen	80
29	Best Time to Buy and Sell Stock	Track prior minimum and best profit	80
30	Subsets	Record every search-tree node	100
31	Permutations	Record only complete arrangements	102
32	Binary Tree Level-Order Traversal	Snapshot the current BFS layer	69
33	Diameter and Max Path Sum	Return one arm and record both	97
34	Lowest Common Ancestor	Return a found subtree witness	97
35	Validate a Binary Search Tree	Carry legal bounds downward	97

No.	Source entry	Coding move	p.
36	Serialize and Deserialize a Binary Tree	Return node and next token position	121
37	Depth-First Search on a Graph	Resume each DFS frame before its next neighbor	105
38	Number of Islands	Choose permanent flood-fill marks	121
39	BFS Shortest Path	Claim at enqueue and save parents	106
40	Kahn's Algorithm	Admit a node when indegree is zero	113
41	Union-Find	Merge roots rather than parents	113
42	Dijkstra's Algorithm	Relax edges and skip stale proposals	114
43	Trie	Index shared prefixes with a trie	117
44	LRU Cache	Keep lookup and recency synchronized	137
45	Design Hit Counter	Expire time buckets at one boundary	139
46	Design Twitter	Merge each followee's next post	94
47	Snapshot Array	Binary search per-index histories	139
48	Design Tic-Tac-Toe	Update row column and diagonal counts	137
49	Flatten Nested List Iterator	Normalize a paused traversal	140
50	Key-Value Store with Transactions	Log the first before-image	140
51	Hash Map with Separate Chaining	Resolve collisions in hash buckets	126
52	Dutch National Flag	Classify the swapped-in item	89
53	Longest Repeating Character Replacement	Repair the validity condition	83
54	Minimum Size Subarray Sum	Shrink only under a monotone sum	83
55	Median of Two Sorted Arrays	Certify both sides of a cut	88
56	Search a 2D Matrix	Translate matrix position to index	87
57	Find Peak Element	Discard a slope-proven half	87
58	Longest Increasing Subsequence	Keep minimal tails by length	132
59	Heapify	Build a heap bottom-up	94
60	Quickselect	Search only the selected partition	89
61	Bucket Sort	Allocate buckets for a bounded domain	129
62	Largest Rectangle in Histogram	Flush unresolved bars with a sentinel	92
63	Basic Calculator	Make the parser cursor explicit	121
64	Sliding Window Maximum	Expire age and remove dominated values	93
65	Insert Interval	Absorb the new sorted interval	85
66	Interval List Intersections	Advance the interval ending first	83
67	Jump Game	Keep the farthest reachable boundary	131
68	Jump Game II	Count one jump per completed layer	131
69	Gas Station	Restart after a proven deficit	132
70	Interval Scheduling	Choose the earliest finishing interval	131
71	Task Scheduler	Schedule equal priority deliberately	90
72	Partition Labels	Cache each label's last occurrence	116
73	Combination Sum	Choose whether an index may repeat	102
74	Word Search	Restore every path-local mark	100
75	Construct Binary Tree from Preorder and Inorder	Return each constructed subtree root	97
76	BST Search, Insert, Delete	Reattach nodes after structural edits	86
77	BST Iterator	Keep a stack of next BST nodes	140
78	Multi-Source BFS	Seed every source at minute zero	110
79	Word Ladder	Claim shortest edit paths once	106

No.	Source entry	Coding move	p.
80	Clone Graph	Map originals to clones once	127
81	Cycle Detection in a Directed Graph	Distinguish gray from finished nodes	111
82	Topological Sort with DFS	Append after all DFS descendants	111
83	Alien Dictionary	Order characters from constraints	113
84	Connected Components	Maintain component identity	113
85	Word Search II	Prune paths with a shared trie	117
86	KMP String Matching	Fall back to a matched prefix	117
87	Longest Palindromic Substring	Expand from each possible center	117
88	Encode and Decode Strings	Frame strings with exact lengths	117
89	Game of Life	Read old bits and write new bits	122
90	Fibonacci	Define a recurrence and base state	132
91	Insert Delete GetRandom O(1)	Map values to array slots	127
92	Time-Based Key-Value Store	Search the last historical write	139
93	LFU Cache	Move keys between frequency buckets	138
94	Cache with Expiring Keys	Check expiration against current truth	139
95	Design Browser History	Move a cursor through mutable history	129
96	Design Underground System	Aggregate route totals by key	116
97	Design Search Autocomplete System	Choose trie prefix over whole-word lookup	117
98	My Calendar I and II	State interval endpoint semantics	85
99	Design Snake Game	Apply one complete move per call	137
100	URL Shortener	Choose a reversible compact key	141
101	Hash Map with Open Addressing	Probe past deleted slots	126
102	Boyer-Moore Majority Vote	Verify a provisional candidate	81
103	Next Permutation	Rearrange a classified suffix	89
104	Reorder List	Split reverse and weave chains	86
105	Median of K Sorted Arrays	Search values using a count test	87
106	Random Pick with Weight	Draw within cumulative weight bounds	136
107	Segment Tree	Cache and combine interval aggregates	124
108	Counting Sort	Count only a bounded key universe	129
109	Heap Sort	Repair a heap after root replacement	94
110	Circular Queue	Wrap queue indices modulo capacity	128
111	Candy	Combine two directional constraints	80
112	Hand of Straights	Consume the earliest required group	131
113	N-Queens	Reject occupied columns and diagonals	103
114	Bipartite Check	Assign two colors at discovery	106
115	Cycle Detection in an Undirected Graph	Detect an already-connected edge	113
116	Critical Connections	Propagate the earliest reachable ancestor	112
117	Bellman-Ford	Repeat edge relaxations for negative weights	114
118	Kruskal's Minimum Spanning Tree	Accept edges only between components	115
119	Prim's Minimum Spanning Tree	Keep cheapest edge leaving the tree	115
120	Rabin-Karp	Verify a rolling-hash candidate	117
121	Longest Common Prefix	Advance only matching character positions	117
122	Text Justification	Separate line selection from rendering	133
123	String Compression	Keep write behind the read frontier	133

No.	Source entry	Coding move	p.
124	Reverse Words in a String	Reverse or join without repeated copies	133
125	Multiply Strings	Write digits into reserved positions	133
126	Spiral Matrix	Maintain four shrinking boundaries	121
127	Rotate Image	Rearrange cells in complete cycles	122
128	GCD and LCM	Preserve a number-theoretic identity	80
129	Sieve of Eratosthenes	Mark multiples starting at the square	134
130	Fast Exponentiation	Consume one exponent bit per step	135
131	Reservoir Sampling	Use an inclusive one-based draw	136
132	Orientation and Segment Intersection	Use exact orientation and bounds	134
133	Convex Hull	Pop nonconvex turns from a chain	134
134	Max Points on a Line	Normalize slope as an integer pair	134
135	Single Number	Use XOR to cancel paired bits	122
136	Count Set Bits	Clear one set bit per step	122
137	Bitmask Subset Enumeration	Encode a subset as a mask	122
138	Rate Limiter	Choose exact window or refill policy	139
139	Consistent Hashing	Map keys to ring successors	141
140	Stock Price Tracker	Version priorities against current values	95
141	All O(1) Data Structure	Remove empty frequency buckets	138
142	Range Module	Keep disjoint covered intervals	124
143	In-Memory File System	Define path ownership in a tree	137
144	Design a Text Editor	Keep text left and right of cursor	129
145	Unique ID Generator	Pack time machine and sequence fields	141
146	Bloom Filter	Allow false yes but never false no	125
147	Count-Min Sketch	Estimate counts with hashed rows	125
148	Quadtree	Subdivide only useful spatial regions	125
149	Job Scheduler	Version names in the event heap	95
150	Inverted Index	Intersect sorted posting lists	168